

Programação Funcional

Uma introdução em Clojure



© Casa do Código

Todos os direitos reservados e protegidos pela Lei nº9.610, de 10/02/1998.

Nenhuma parte deste livro poderá ser reproduzida, nem transmitida, sem autorização prévia por escrito da editora, sejam quais forem os meios: fotográficos, eletrônicos, mecânicos, gravação ou quaisquer outros.

Edição

Vivian Matsui
Sabrina Barbosa

[2021]

Casa do Código

Rua Vergueiro, 3185 - 8º andar

04101-300 – Vila Mariana – São Paulo – SP – Brasil

casadocodigo.com.br



Este livro possui a curadoria da Casa do Código e foi estruturado e criado com todo o carinho para que você possa aprender algo novo e acrescentar conhecimentos ao seu portfólio e à sua carreira.

A Casa do Código é a editora do Grupo Alura, grupo que nasceu da vontade de criar uma plataforma de ensino com o objetivo de incentivar a transformação pessoal e profissional através da tecnologia.

Juntas, as empresas do Grupo constroem uma verdadeira comunidade colaborativa de aprendizado em programação, negócios, design, marketing e muito mais, oferecendo inovação na evolução dos seus alunos e alunas através de uma verdadeira experiência de encantamento.

Venha conhecer os cursos da Alura e siga-nos em nossas redes sociais.

 alura.com.br

 [@casadocodigo](https://www.instagram.com/casadocodigo)

 [@casadocodigo](https://twitter.com/casadocodigo)

ISBN

Impresso e PDF: 978-85-7254-001-8

EPUB: 978-85-7254-002-5

MOBI: 978-85-7254-003-2

Caso você deseje submeter alguma errata ou sugestão, acesse
<http://erratas.casadocodigo.com.br>.

PREFÁCIO

Poucos anos atrás, eu me vi entrando no mundo do Clojure para resolver o problema de processar muitos dados diferentes ao mesmo tempo e fazer o *merge* deles em uma estrutura de dados só. Naquela época, era difícil encontrar coisas de Clojure em português e/ou com tanta qualidade quanto o livro do Gregório. Minha trajetória foi divertida, mas a partir de hoje vou certamente passar a indicar este livro como o primeiro contato com Clojure.

Quando o Gregório pediu para eu ler o livro e dar um feedback, tive de apagar minhas opiniões centenas de vezes, pois parecia que ele já tinha previsto o que eu ia falar. Foi uma dinâmica incrível ler pensando que seria legal adicionar algo, e encontrá-lo lá, no próximo capítulo. Creio que o livro tem todo o conteúdo que eu gostaria de ver em um livro de Clojure.

Agora, se você já sabe um pouco de Clojure, vá para a segunda e terceira partes e tente aplicar as ideias do livro em seu projeto ou dia a dia. Espero que você goste deste livro tanto quanto eu!

Julia Naomi Boeira

ESTE LIVRO É PARA VOCÊ

O intuito deste livro é ser um material introdutório à Programação Funcional. Mesmo sendo introdutório, ele não tem a intenção de ser o seu primeiro livro de programação. Portanto, saber programar um pouquinho será muito útil. Se você já escreveu algumas linhas de código na sua vida, este livro pode ser

para você.

Aqui você aprenderá novos conceitos que vão ajudá-lo a escrever melhores programas de computador. Estes conceitos serão aprendidos ao longo do livro e, assim que apresentados, já podem ser aplicados no seu trabalho ou estudo. Tenho a expectativa de que, enquanto lê este livro, você procure aplicar tudo o que aprender, e demonstre para sua equipe as vantagens do que viu aqui. E, ainda, quando concluir a leitura, que você veja o tanto que terá se desenvolvido na escrita de software.

Veremos um bocado de código em Clojure. Se você tem a mente aberta para uma nova linguagem de programação e acredita que uma sintaxe e paradigma novos o levarão a novos horizontes, este livro é para você.

Por ser introdutório, não abordaremos conceitos como *monads* e *applicatives*, que têm assustado muita gente ao entrar neste mundo (inclusive eu mesmo), nem equações matemáticas.

COMO ESTE LIVRO ESTÁ ORGANIZADO

Nossa jornada será muito interessante. Introduziremos princípios da Programação Funcional, e aos poucos veremos algumas comparações com linguagens historicamente consideradas orientadas a objeto (como Java e Ruby).

Na primeira parte do livro, veremos um pouco de Clojure, o suficiente para que nos familiarizemos com a linguagem. Isto deve facilitar a compreensão dos exemplos e conceitos de Programação Funcional, apresentados na parte seguinte, que é onde vamos conhecer e nos aprofundar em seus princípios. Algumas ideias

muito bacanas aparecerão, trazendo um vocabulário novo para nós, como *preguiça*.

Na parte 3, veremos como construir aplicações em Clojure. Utilizando o domínio e as funções da parte 2, produziremos um serviço para processar requisições HTTP, fazendo uso de uma abordagem de desenvolvimento guiada por testes. E tudo isso acompanhado de muitos exemplos de código.

Participe da discussão

Os códigos de exemplo utilizados neste livro podem ser encontrados em <https://gitlab.com/programacaofuncional/>.

SOBRE O AUTOR

Natural de João Pessoa, Paraíba, Gregório Melo é desenvolvedor de software na ThoughtWorks Brasil. Estudou Desenvolvimento de Sistemas para Internet no IFPB e iniciou a carreira na área de desenvolvimento de software estagiando em uma grande empresa nacional de varejo, gerando Strings imensas a partir de um formulário gigante para criação de cartões de crédito.

Em 2011, mudou-se para Porto Alegre, onde teve seus primeiros contatos com Programação Funcional. Após um período de contatos esporádicos com linguagens funcionais, foi durante um engajamento profissional em Xi'an, na China, que Programação Funcional virou presença constante no seu dia a dia. O foco, na época, era Scala. Hoje, dedica-se a Clojure, sem querer discutir qual das duas é melhor. Foi gerente do escritório da ThoughtWorks em São Paulo entre junho de 2016 e dezembro de

2017, e hoje é, de novo, consultor de desenvolvimento na mesma.

Adora jogar futebol, mesmo com um talento um tanto quanto questionável, e talvez passe mais tempo do que deveria jogando videogames.

AGRADECIMENTOS

Este livro é um dos frutos de todo o carinho e amor que Diana e eu nutrimos entre nós. Obrigado por todo o incentivo e apoio! Não existiria livro algum sem você por perto.

Dentre os diversos privilégios que tenho, considero o mais significativo ter convivido num lar onde a educação sempre foi muito valorizada. Agradeço a minha mãe e a meu pai por me proporcionar uma educação diferenciada, e a minha irmã e irmão pelo respeito e compaixão que temos entre nós.

Agradeço à ThoughtWorks por tantas oportunidades fantásticas de carreira, onde pude conviver com pessoas maravilhosas e inspiradoras!

Agradeço à Julia Naomi pela revisão técnica e pelo prefácio que me deixou emocionado. Quanto conhecimento! Também ao Paulo Caroli pela mentoria e todo o incentivo com o livro e tudo mais!

À Vivian, minha editora, que tem uma paciência exemplar! Vivian, obrigado por me acompanhar nessa jornada.

E gostaria de agradecer a todos os computadores que já rodaram os programas que eu criei. E também àqueles que pelo menos tentaram.

POR QUE LER ESTE LIVRO?

Se você se interessa em aprender a escrever programas melhores, Programação Funcional tem muito a lhe ajudar. Com ela, você pode produzir códigos mais robustos, menos suscetíveis a erros e expandir sua forma de pensar. Por exemplo, a imutabilidade, um conceito que veremos ao longo do livro, minimiza a possibilidade de encontrarmos defeitos oriundos da manipulação de estado em lugares desconhecidos. Você verá os principais conceitos que trazem à tona as vantagens da programação funcional.

Uma grande razão para aprender Programação Funcional é fazer uso dos benefícios do paralelismo, e veremos como fazer isso mais à frente. Escrever código paralelizável em uma linguagem de paradigma funcional é muito fácil. A ausência de efeitos colaterais (ou o estímulo para que eles sejam contidos e limitados) nas funções de um programa permite que estas funções sejam executadas sem uma ordem definida e em paralelo. Isso nos faz tirar proveito da nova aplicação da Lei de Moore, que agora não mais se refere à quantidade de transistores, mas sim de núcleos na máquina (<http://www.gotw.ca/publications/concurrency-ddj.htm>). Assim, nossa vida fica mais simples com a linguagem de programação nos ajudando. Você vai perceber como é fácil escrever código para rodar em múltiplas *threads*!

Ainda, para algumas pessoas, as origens na matemática fazem da Programação Funcional um paradigma mais simples de ser compreendido do que a Orientação a Objetos. Para estas pessoas, será muito natural a ideia de que, sempre que você chamar uma

função com um mesmo parâmetro, o resultado será o mesmo, sem que nada mais no universo do programa seja alterado. Claro que isso não é exclusivo da Programação Funcional, já que podemos alcançar o mesmo resultado em outros paradigmas, mas a Programação Funcional vai ajudá-lo a pensar sempre em construir funções sem efeitos colaterais.

Por fim, talvez você queira entender um pouco mais sobre como escrever melhor código na plataforma que você utiliza atualmente. As ideias que foram citadas nos parágrafos anteriores podem ajudar seus programas a serem mais robustos. Você pode fazer com que seu código em JavaScript fique mais idiomático, e, de brinde, você vai entender melhor o processo de escrita de código para frameworks JavaScript como o D3, Ramda, Lodash, ou tantos outros. Seu código em Java 8, por exemplo, pode ficar muito mais sucinto e com melhor desempenho, tendo mais facilidade no uso do paralelismo provido pela máquina virtual.

POR QUE CLOJURE?

Existem muitas linguagens de programação que trazem consigo os princípios de Programação Funcional. Muitas delas têm conseguido uma boa adoção no mercado, como Scala, Clojure e Elixir. E linguagens cujas origens estão atreladas à Orientação a Objetos, como Java, C#, Ruby e Python, já há algum tempo possuem recursos encontrados em Linguagens Funcionais.

Então, por que Clojure? Além de ser a minha linguagem favorita do momento, ela possui recursos interessantes para nos ajudar a manter o foco nos aspectos inerentes à linguagem e à Programação Funcional: a sintaxe, que é simples e muito diferente

das linguagens mais populares, e um sistema dinâmico de tipos, o que vai nos permitir evitar pensar em Orientação a Objetos por um tempo.

Clojure é um dialeto de Lisp (<https://pt.wikipedia.org/wiki/Lisp>). Lisp é uma linguagem que trouxe a ideia de que é possível utilizar apenas funções matemáticas para representar estruturas de dados básicas, aliado ao conceito de código como dado (do inglês *Homoiconicity*). Lisp tem muita história na computação, principalmente no mundo das linguagens funcionais e no campo da inteligência artificial. Sendo Clojure um dialeto de Lisp, ela traz uma sintaxe bem diferente das linguagens derivadas de Algol (como C e afins). Você vai perceber que, além de diferente do tradicional, a sintaxe é extremamente simples, com poucos símbolos ou palavras-chaves reservadas.

Clojure, principalmente, roda na máquina virtual Java, o que permite que programas escritos nesta linguagem usem bibliotecas escritas em Java. A ferramenta de automação de tarefas e gerenciamento de dependências, Leiningen, é bastante flexível e completa, e provê suporte ao repositório de bibliotecas do Maven. Ela também roda em outras plataformas, sendo JavaScript a mais notória (através do ClojureScript, que compila seu código Clojure para JavaScript), e no motor do Unity (para desenvolvimento de jogos) com o Arcadia (<http://arcadia-unity.github.io/>).

Existe uma comunidade muito legal por trás da linguagem, além de casos de sucesso de adoção de Clojure, que lhe dão um respaldo muito forte.

Fui convincente o bastante? Que tal começar a pôr a mão na massa e instalar as ferramentas necessárias?

INSTALANDO CLOJURE

Você precisará instalar o kit de desenvolvimento do Java, JDK, na versão 8 ou 11. O OpenJDK deve bastar (<http://openjdk.java.net>). Se você já tem a JDK da Oracle instalada (versão 8 ou 11), não precisa se preocupar em instalar o OpenJDK.

Com o JDK instalado, basta instalar o Leiningen conforme instruções no site: <http://leiningen.org/>.

Para verificar se a instalação foi feita com sucesso, abra um novo terminal (ou prompt de comando), e digite `lein -v`. Deve aparecer uma mensagem dizendo qual é a versão instalada do Leiningen, o que significa que a instalação foi um sucesso. Talvez demore um pouco para o resultado deste comando aparecer, já que é a primeira execução do Leiningen e ele deve buscar na grande rede algumas bibliotecas essenciais para que nossos códigos em Clojure rodem com sucesso.

Neste link você vai encontrar um guia atualizado de como instalar Clojure no Windows, no Mac e no Linux: <http://bit.ly/instalando-clojure>.

Nosso primeiro código em Clojure (e o REPL)

Esta combinação de letras passará a ser comum no seu vocabulário, se já não for. REPL é um ambiente interativo onde escrevemos códigos e eles são interpretados de imediato, gerando resultados muito rápidos. É ideal para trechos pequenos e

validação de ideias.

REPL é um acrônimo para *read, evaluate, print e loop*. Isto quer dizer que ele vai ler nossos comandos, interpretá-los, exibir na tela o resultado e repetir o processo. Se você programa em Ruby, talvez já tenha se deparado com `irb` ou `pry`. Python também oferece uma ferramenta para o mesmo propósito: é só digitar `python` no terminal e você terá um interpretador. Scala e Erlang são exemplos de linguagens cujas plataformas também oferecem tais ferramentas. Dica: o REPL do Clojure é o mais bacana.

Programar no REPL é algo bem comum entre quem tem familiaridade com Clojure. Claro, em algum momento nossos códigos vão parar em arquivos e serão empacotados. Mas o REPL fornece um ambiente muito prático para experimentação e testes. Algumas pessoas substituem desenvolvimento guiado por testes pelo constante uso do REPL. Eu prefiro continuar a criar casos de testes, e mais para a frente nós veremos como aplicar desenvolvimento guiado por testes aos nossos programas em Clojure. Agora, vejamos este tal de REPL na prática.

Em uma janela no terminal, digite:

```
lein repl
```

Um terminal interativo deve aparecer para você, com uma mensagem mais ou menos assim:

```
nREPL server started on port 59100 on host  
127.0.0.1 - nrepl://127.0.0.1:59100
```

E, esperando você fornecer alguma informação, o terminal diz:

```
user=>
```

Isso significa que já podemos escrever código! Digite o seguinte:

```
"Valeu, Clojure!"
```

Aperte `Enter` e veja o texto `Valeu, Clojure!` aparecer na tela mais uma vez. Não sei você, mas eu acho que os passos para um *Hello, world!* como este foram práticos e rápidos (bem, talvez não tão rápidos por causa da primeira execução do Leiningen, quando ele baixa as dependências mínimas para começarmos a utilizar Clojure).

Digite `exit` para sair do REPL. Ou `quit`, ou aperte `control + d`. Já já voltaremos para ele, já que lá praticaremos as partes 1 e 2 do livro.

Ferramentas alternativas

O OpenJDK tem crescido bastante ao longo dos anos, sendo o suficiente para o desenvolvimento de aplicações em Clojure. Mas fique à vontade para utilizar a JDK da Oracle, se preferir.

O Leiningen é a ferramenta mais comum no meio Clojure. Mas existe uma alternativa que tem ganhado muita tração e interesse da comunidade, o Boot (<http://boot-clj.com>). Após acostumar-se com o Leiningen, recomendo que você pesquise um pouco mais o Boot, que pode ser mais apropriado para você e/ou seu time.

Conclusão

Agora temos tudo que precisamos para metermos a mão na

massa e começarmos a praticar Programação Funcional com Clojure! Estou aqui imaginando sua empolgação para correr para as próximas páginas. Nos vemos lá!

Sumário

O suficiente de Clojure para começarmos a brincadeira	1
1 Primeiros contatos com Clojure	2
1.1 As primeiras linhas de código	2
1.2 Nossas próprias funções	4
1.3 O que é a verdade? Sobre os condicionais	6
1.4 Conclusão	9
2 Resolvendo o FizzBuzz	11
2.1 Conclusão	14
3 Estruturas de dados em Clojure	15
3.1 Listas	15
3.2 Vetores	19
3.3 Sets	22
3.4 Conclusão	23
4 Controle financeiro e novas estruturas de dados	24
4.1 Keywords	25
4.2 Mapas	25

4.3 Conclusão	28
Embarque no mundo da Programação Funcional	29
5 Programação Funcional, o começo	30
5.1 Funções: primeira classe e grandeza superior	31
5.2 Funções de grandeza superior e nossas finanças	33
5.3 Conclusão	43
6 Composição de funções e aplicação parcial de funções	44
6.1 Composição de funções	53
6.2 Aplicação parcial	55
6.3 Conclusão	59
7 Pureza e imutabilidade	61
7.1 Pureza	61
7.2 Imutabilidade	65
7.3 Imutabilidade no nosso domínio	72
7.4 Recursão	73
7.5 Conclusão	84
8 Preguiça	85
8.1 Sequências preguiçosas e listas infinitas de transações	90
8.2 A macro lazy-seq	95
8.3 Conclusão	99
Programação Funcional na prática	101
9 Criando uma aplicação com Leiningen	102

9.1 O conversor de moedas	106
9.2 Interoperabilidade com Java	107
9.3 Dependências	110
9.4 Consultando o serviço	113
9.5 Interpretando JSON	118
9.6 Conclusão	124
10 Controle financeiro pessoal via HTTP	125
10.1 O esqueleto com o Leiningen	125
10.2 Testes	132
10.3 Conclusão	140
11 Evolução da API: saldo inicial	142
11.1 Começando com testes de aceitação	142
11.2 JSON como formato de representação de dados	156
11.3 O toque final	162
11.4 Conclusão	165
12 Uma API mais rica com transações	166
12.1 Criando transações	166
12.2 Conclusão	187
13 Extrato das transações e seus filtros	189
13.1 Receitas ou despesas?	189
13.2 Conclusão	209
14 Considerações finais	210
14.1 Ferramentas para o dia a dia da programação	210
14.2 Oportunidades de refatoração	211
14.3 Funcionalidade a implementar	212

14.4 Outras formas de testes	212
14.5 Ports & Adapters (hexagonal) e funcional	213
14.6 Despedida	213

Versão: 26.2.22

O suficiente de Clojure para começarmos a brincadeira

PRIMEIROS CONTATOS COM CLOJURE

1.1 AS PRIMEIRAS LINHAS DE CÓDIGO

Tudo preparado para escrevermos código loucamente? Agora que você já instalou as ferramentas necessárias, vamos praticar um pouco. No terminal, volte a executar o REPL:

```
lein repl
```

A ideia é vermos um pouco da sintaxe do Clojure para pegarmos um pouco de familiaridade, para, então, partirmos para coisas bem legais. Vamos ver algumas operações aritméticas primeiro. No REPL, digite a seguinte operação:

```
(+ 1 2)
```

O resultado, como você já deve estar esperando, é `3`. Ufa, a máquina virtual ainda sabe fazer contas matemáticas. Mas a sintaxe parece bem diferente das linguagens com as quais estamos acostumados, não é? Vale vermos um pouco de explicação do que acontece na linha que acabamos de digitar.

Esta linha parece uma lista, composta de `+`, `1` e `2`. E é isso mesmo. Acontece que o primeiro elemento desta lista é especial.

Este elemento é uma função que é executada, e os demais elementos desta lista são argumentos para esta função. O mesmo se aplica a outras operações aritméticas:

```
(* 2 3)
(/ 2 2)
(- 0 2)
```

E se for uma operação um pouco mais complexa? Olhe só:

```
(* 2 (+ 3 3))
;; isto é um comentário em Clojure e vamos
;; utilizá-lo para demonstrar o retorno das funções
;; 12
```

Este exemplo é onde Clojure mostra mais um diferencial no sentido de sintaxe, fruto de sua herança de Lisp. Do ponto de vista de precedência, é claro que a soma será executada primeiro. O diferencial, porém, é que a linguagem exige os parênteses, o que não deixa margem alguma para dúvidas do que precede o quê. Portanto, a ordem de execução de código é sempre de dentro para fora. Então, primeiro veremos o código `(+ 3 3)` ser executado, e o resultado, `6`, é argumento para o seguinte, `(* 2 6)`.

Vamos ver como podemos concatenar Strings? Assim:

```
(str "Oi, " "mundo!")
```

Como podemos verificar se duas Strings são iguais? Assim:

```
(= "Oi" "Olá")
;; false

(= "Oi" "Oi")
;; true
```

É importante notar que, diferente de muitas linguagens, `=` é uma função em Clojure que verifica se duas coisas são iguais. O `=`

não é um operador de associação, normalmente utilizado na construção de variáveis.

Vamos explorar mais. Como verificamos se um número é par?

```
(even? 2)
;; true
```

Se um número é múltiplo de 3?

```
(= 0 (mod 9 3))
;; true
```

Neste último exemplo, o que acontece é que vamos verificar o módulo da divisão entre 9 e 3. O resultado será utilizado como o segundo argumento na função que verifica igualdade, =.

1.2 NOSSAS PRÓPRIAS FUNÇÕES

Agora que já sabemos como chamar funções, que tal criarmos as nossas? Vamos começar do básico. Que tal uma função que receba um nome e dê um "oi"? Um exemplo de execução seria algo assim:

```
(oi "galera")
```

E o resultado esperado seria Oi, galera! .

Sabemos que precisamos criar uma função que se chame oi e que receba um só argumento, uma String. E que o coloque entre as Strings Oi, e ! .

Para criar esta função, fazemos o seguinte:

```
(defn oi [nome]
  (str "Oi, " nome "!"))
```


NAMESPACES

Ao criar uma função no REPL, você verá que a saída é a seguinte:

```
#'user/oi
```

Isto quer dizer que alguma coisa com o nome `oi` acabou de ser criada, e encontra-se no *namespace* padrão, `user`. Namespaces em Clojure representam a mesma ideia que em outras linguagens, como pacotes em Java, sendo uma forma de agrupar funções. A combinação do namespace e do nome da função forma o identificador de tal função.

A função `+`, por exemplo, é encontrada no namespace `clojure.core`, sendo seu identificador `clojure.core/+`. Como o namespace `clojure.core` é disponibilizado por padrão, a função `+` está sempre disponível. Funções em outros namespaces precisam ser incluídas no nosso código antes de serem utilizadas. Veremos como fazer isto em um outro momento.

Com a função criada, vamos aplicá-la ao nome desejado:

```
(oi "galera")  
;; "Oi, galera!"  
  
(oi "Margaret Hamilton")  
;; "Oi, Margaret Hamilton!"
```

O `defn` nos indica que vamos criar uma função. Depois, damos um nome a ela; neste caso, `oi`. Logo a seguir, vem a lista

de argumentos, cercada por `[ e ]` . Neste caso, temos apenas um só argumento, então fica `[nome]` .

Em seguida vem o que realmente é executado: a concatenação de Strings. Note que, assim como em algumas linguagens, não precisamos definir o que será retornado. A última instrução é o que será retornado; neste caso, o resultado da concatenação. Note que o parêntese que foi aberto em `defn` só é fechado no final. Isto nos ajuda muito a entender até onde vai o escopo do código que criamos.

O trecho que concatena Strings tem uma particularidade: é aplicado em 3 argumentos. Esta é uma de várias funções que são aplicáveis em uma quantidade indeterminada de argumentos.

E como seria uma função que verifica se um número é múltiplo de 3? Precisamos verificar se, na divisão deste número por 3, o resto é 0.

```
(defn multiplo-de-3? [dividendo]
  (= 0 (mod dividendo 3)))
```

Então podemos chamar a função assim:

```
(multiplo-de-3? 9)
;; true

(multiplo-de-3? 20)
;; false
```

1.3 O QUE É A VERDADE? SOBRE OS CONDICIONAIS

Há pouco aprendemos a criar nossas próprias funções e a colocar um pouco de lógica nelas. Mas e se quisermos dar um

rumo diferente a esta lógica, dependendo de alguma condição? Clojure também tem condicionais (novidade!). Que tal incrementarmos nossa caixa de ferramentas e colocarmos uns condicionais à nossa disposição?

De novo, vamos aplicar algo simples na jogada. Que tal exibirmos a palavra `sim` se um número for par, e, caso contrário, a palavra `não` ? Pegando carona com a função `even?` , nosso código fica assim:

```
(defn par? [numero]
  (if (even? numero)
      "sim"
      "não"))
```

Vamos ver como nos saímos?

```
(par? 5)
;; não

(par? 4)
;; sim
```

`if` tem muito a cara de uma função também. Até podemos enxergá-la como uma, mas ela é na verdade o que é chamado de *forma especial*: um recurso base do Clojure. Veremos outras (que são poucas, na verdade) ao longo do livro.

O `if` recebe 3 argumentos, começando com uma verificação que retorna verdadeiro ou falso (`true` ou `false`). Os demais argumentos representam algo a ser executado de acordo com o resultado da verificação. Em caso de verdadeiro, o argumento 2 é avaliado e retornado. Caso contrário, o argumento 3 é que é avaliado e retornado.

É importante salientar que **apenas** `nil` e `false` são

considerados realmente falsos para a verificação de condições. Outros, como `0` e String vazia, que são comuns em outras linguagens, serão avaliados como verdadeiro!

Vale repetir: **apenas** `nil` e `false` são considerados realmente falsos!

CONSULTANDO A DOCUMENTAÇÃO

Sempre que você quiser saber mais sobre algum recurso do Clojure, você pode experimentar a documentação dentro do REPL! Para isso, use a função `doc` :

```
(doc if)
```

E se houver outra opção na nossa lógica, além de apenas duas? Por exemplo, dizer quando um saldo é positivo, negativo ou zerado. Nestes casos, a macro `cond` pode nos ajudar (veremos mais sobre macros no capítulo 5). Ela recebe pares de condicionais e expressões. Segue um exemplo:

```
(defn saldo [valor]
  (cond (< valor 0) "negativo"
        (> valor 0) "positivo"
        :else "zero"))
```

Vamos testar?

```
(saldo 900)
;; "positivo"

(saldo -900)
;; "negativo"
```

```
(saldo 0)
;; "zero"
```

Neste exemplo, quando o valor é menor que zero, o primeiro teste, logo de cara, retorna verdadeiro, e a expressão que o segue, "negativo", é avaliada (neste caso, não faz nada, apenas retorna a String "negativo"). Quando o valor é maior que zero, o primeiro teste (`(< valor 0)`) falha, retornando falso, mas o teste seguinte retorna verdadeiro, e a expressão correspondente é avaliada. Agora, quando o valor é exatamente 0, as duas primeiras verificações retornam falso, e a última verificação (`:else`) é validada e "zero" é retornado.

O que significa este `:else` ? Nada demais, na real. Como qualquer coisa diferente de `nil` e `false` é considerada verdadeira, qualquer coisa que colocássemos ali no lugar do `:else` funcionaria como uma forma de fazer com que "zero" fosse retornado por padrão. Se quiser, teste lá com valores como 1 ou "milhão". Bem, não é realmente *qualquer* coisa: números (mesmo que 0), Strings (mesmo que vazias), caracteres, coleções... O `:else` é apenas uma convenção adotada por algumas pessoas as quais eu andei copiando. Veremos um pouco mais sobre essa "coisa" que começa com `:` no capítulo 4.

1.4 CONCLUSÃO

Ufa! Acho que agora temos a capacidade de aumentar um pouco mais a complexidade dos problemas que podemos tratar. Aqui aprendemos como interagir com o REPL do Clojure, chamar funções, criar nossas próprias funções e trabalhar com condicionais. No próximo capítulo, resolveremos um problema bastante comum no mundo da programação: o FizzBuzz. Você

verá como a solução para este problema fica bem sucinta em Clojure.

RESOLVENDO O FIZZBUZZ

Com tudo o que aprendemos até aqui, podemos resolver um problema simples e bem comum em eventos de programação: o FizzBuzz (https://en.wikipedia.org/wiki/Fizz_buzz).

Explicando brevemente, devemos criar uma função que nos retorne:

- `fizz` , se o número for divisível por 3;
- `buzz` , se divisível por 5;
- `fizzbuzz` , se divisível por 3 e por 5; ou
- o próprio número, caso contrário.

Quer tentar primeiro, antes que eu mostre uma possível solução?

Se não, vamos lá. Podemos começar pensando em como seria o nome ideal para uma função que fizesse o cálculo, e que recebesse um argumento, que é o número que buscamos. Que tal uma função que se chame `fizzbuzz` ? Seria algo assim:

```
(defn fizzbuzz [numero]
  numero)
```

Desse jeito, chamaríamos a função assim: `(fizzbuzz 8)` . Porém, claro, ainda não há lógica suficiente ali; o próprio número

seria sempre retornado. Vamos primeiro criar uma função que verifica se um número é divisível por outro:

```
(defn divisivel-por? [dividendo divisor]
  (= 0 (mod dividendo divisor)))
```

Podemos dar uma melhoria de leve, substituindo o `= 0` pela função nativa que verifica se um número é exatamente 0 : `zero?` . Então a função de verificar divisibilidade fica assim:

```
(defn divisivel-por? [dividendo divisor]
  (zero? (mod dividendo divisor)))
```

Agora podemos chamar esta função dentro dos condicionais que checam a lógica do FizzBuzz, começando com a primeira verificação, a divisão por 3:

```
(defn fizzbuzz [numero]
  (cond (divisivel-por? numero 3) "fizz"
        :else numero))
```

O primeiro passo foi dado. Agora podemos ver o resultado para os múltiplos de 3

```
(fizzbuzz 3)
;; "fizz"

(fizzbuzz 6)
;; "fizz"

(fizzbuzz 20)
;; 20

(fizzbuzz 8)
;; 8
```

Agora precisamos ver o caso dos múltiplos de 5:

```
(defn fizzbuzz [numero]
  (cond (divisivel-por? numero 3) "fizz"
        (divisivel-por? numero 5) "buzz"
```



```

        :else numero))

(fizzbuzz 3)
;; "fizz"

(fizzbuzz 6)
;; "fizz"

(fizzbuzz 20)
;; "buzz"

(fizzbuzz 8)
;; 8

```

E agora, e qual seria o resultado para o 15?

```

(fizzbuzz 15)
;; "fizz"

```

O resultado foi "fizz" , o que é compreensível. O que acontece é que a primeira condição é válida, uma vez que 15 é divisível por 3. A saída é verificarmos, antes de tudo, se o número é divisível por 3 e por 5, como diz o enunciado do problema:

```

(defn fizzbuzz [numero]
  (cond (and (divisivel-por? numero 3)
            (divisivel-por? numero 5)) "fizzbuzz"
        (divisivel-por? numero 3) "fizz"
        (divisivel-por? numero 5) "buzz"
        :else numero))

```

Vamos testar?

```

(fizzbuzz 15)
;; "fizzbuzz"

```

Agora sim! A função `and` verifica a veracidade de todos os seus argumentos. Se todos eles forem `true` , `and` retorna `true` . Se algum deles for `false` , `and` retorna `false` . No caso do número 15, `and` recebe `true` e `true` , portanto, retorna `true` e "fizzbuzz" é o resultado.

Dado que já aprendemos sobre a função `and` , que tal praticar no REPL uma outra função interessante, a função `or` ?

2.1 CONCLUSÃO

Esta é uma das várias possíveis soluções para este problema. Aqui vimos um pouco mais a aplicabilidade de condicionais e uma nova função, `and` . Logo mais daremos nosso primeiro passo no mundo funcional através da função `map` .

ESTRUTURAS DE DADOS EM CLOJURE

Agora que temos o FizzBuzz resolvido, que tal aplicar esta lógica a uma lista de números? Até agora vimos um pouco sobre estruturas de dados em Clojure e chegou a hora de vermos listas e mapas! Vejamos primeiro as listas.

3.1 LISTAS

```
;; é assim que criamos uma lista em Clojure
(list 1 2 3 4 5)
;; (1 2 3 4 5)

;; ou assim
'(1 2 3 4 5)
;; (1 2 3 4 5)
```

POR QUE O APÓSTROFO NA CONSTRUÇÃO DA LISTA?

Bem observado! Lembre-se de que a primeira coisa que vem depois de um parêntese (é normalmente uma função. E esta é uma característica de linguagens baseadas em Lisp, onde tudo é uma lista, inclusive quando aplicamos uma função. Sem o apóstrofo, a interpretação seria de que 1 seria compreendido como uma função, e seria aplicada aos demais argumentos. Com o apóstrofo, instruímos que a lista seja interpretada como uma simples lista, sem execução de função alguma.

Podemos até dar um nome a esta lista:

```
(def um-ate-5 '(1 2 3 4 5))  
;; #'user/um-ate-5  
  
(count um-ate-5)  
;; 5
```

O `def` nos permite dar um nome a alguma coisa e esta coisa pode, então, ser referenciada mais adiante. É como a declaração de variáveis em outras linguagens, como o uso de `const` em JavaScript. E aí podemos passar o valor referenciado com `def` para uma função, como fizemos ao aplicar a função `count` em `(count um-ate-5)`, que nos diz quantos elementos há em uma lista. E como estamos lidando com uma sequência de números, podemos usar mais uma função nativa para nos ajudar a criar a lista:

```
;; range é não-inclusivo, por isso o 16  
(def um-ate-15 (range 1 16))
```

```
;; #'user/um-ate-15
```

FizzBuzz até 15, em Clojure

Agora podemos ver como fica o FizzBuzz aplicado a cada número desta sequência.

```
(map fizzbuzz um-ate-15)
;; (1 2 "fizz" 4 "buzz" "fizz" 7 ... 13 14 "fizzbuzz")
```

O que acontece aqui é que, com `map`, vamos aplicar uma função (`fizzbuzz`) a cada elemento de uma lista (`um-ate-15`). Confesso que, na primeira vez que vi uma sentença assim, achei que o código era sucinto demais para ser verdade. O grande ponto aqui é que nós descrevemos no código *o que* precisa ser feito, e não *como*. O compilador e a JVM vão cuidar para que a operação ocorra da forma mais otimizada possível.

Na próxima parte do livro, no capítulo 5, vamos conversar em mais detalhes sobre `map` e funções afins. Mas se você quiser ver um pouco mais sobre a função `map`, sugiro olhar a documentação.

```
(doc map)
```

Que tal um ligeiro comparativo de como seria o mesmo procedimento em Java?

FizzBuzz até 15, em Java

Antes de Java ter *closures* (antes do Java 8), seria mais ou menos assim que varreríamos uma coleção para aplicar a função FizzBuzz para os números de 1 a 15:

```
// pseudo-java, OK?
// suposta implementação do FizzBuzz
```

```

public String fizzBuzz(Integer numero) { ... }

// uma lista com números de 1 a 15
List<Integer> numeros = ...

// uma lista para o resultado
List<Integer> resultado = new ArrayList<>();

// varre cada número, verifica qual o fizzbuzz e adiciona na
// lista de resultado
for (Integer numero : numeros) {
    resultado.add(fizzBuzz(numero));
}

System.out.println(resultado);

```

Nas versões 7 e anteriores do Java, precisamos descrever como é o processo de aplicar uma função: dizemos explicitamente que devem ser varridos todos os elementos, e o resultado de FizzBuzz no elemento deve ser copiado para uma outra lista. Já no Java 8, as coisas mudam um pouco: somente dizemos *o que* precisa ser feito, e não *como*:

```

public String fizzBuzz(Integer numero) { ... }
List<Integer> numeros = ...

List<String> resultado = numeros.stream()
                                .map(FizzBuzzeria::fizzBuzz)
                                .collect(Collectors.toList());

System.out.println(resultado);

```

Brevemente, o que acontece nesse último trecho de código é:

- Utilizamos o método `stream()` para gerar uma versão funcionalmente navegável da lista `numeros`.
- Pedimos que o método `fizzBuzz`, presente na classe `FizzBuzzeria`, seja aplicado a cada elemento do que foi gerado no item acima.

- Por fim, pedimos que a JVM colete os resultados da operação anterior e os traga como uma lista.

Mas, como este não é um livro de Java, não precisamos entrar em detalhes de tudo isso. Recomendo a leitura do livro *Java 8 Prático: Lambdas, Streams e os novos recursos da linguagem* (<https://www.casadocodigo.com.br/products/livro-java8/>).

3.2 VETORES

Até agora falei muito em listas, mas Clojure também traz a ideia de vetores:

```
;; para criar um vetor:  
(vector 1 2 3 4 5)  
;; [1 2 3 4 5]  
  
;; ou apenas  
[1 2 3 4 5]  
;; [1 2 3 4 5]  
  
(def numeros-vetorizados [1 2 3 4 5])  
;; #'user/numeros-vetorizados  
  
;; muita coisa funciona tanto para listas quanto vetores  
(map fizzbuzz numeros-vetorizados)  
;; (1 2 "fizz" 4 "buzz")
```

Existem diferenças sutis entre ambas as estruturas de dados, principalmente no aspecto de desempenho. Mas não recomendo que você se preocupe com desempenho agora, apenas com quão diferentes elas são do ponto de vista de manipulá-las. Os vetores são como os famosos *arrays*, começando da posição 0, e podem guardar elementos de qualquer tipo, permitindo a busca dos elementos através do índice. Vetores são eficientes para adicionar itens ao final, enquanto listas, para adicionar itens no início. O

livro *Clojure Applied: From Practice to Practitioner*, de Ben Vandgrift e Alex Miller, elabora muito bem sobre quando melhor usar cada estrutura de dados.

```
(def cantor-arretado (vector "Chico César" "Catolé do Rocha"
                              26 "janeiro" 1964))
;; #'user/cantor-arretado

(get cantor-arretado 0)
;; "Chico César"

(get cantor-arretado 4)
;; 1964

(last cantor-arretado)
;; 1964

;; vamos incluir um novo elemento no vetor
(conj cantor-arretado "MPB")
;; ["Chico César" "Catolé do Rocha" 26 "janeiro" 1964 "MPB"]

;; e agora ver o estado atual do vetor
cantor-arretado
;; ["Chico César" "Catolé do Rocha" 26 "janeiro" 1964]
```


Clojure trata todo espaço em branco como um simples espaço. Isto quer dizer que tabulação (devido ao uso da tecla TAB) e quebra de linha (ENTER) são tratados da mesma forma que um espaço em branco. Perceba que os valores passados para a construção do vetor em cantor-arretado tem uma quebra de linha depois de "Catolé do Rocha" . Devido ao limite de caracteres para exibição de código no livro, encontraremos mais algumas quebras de linha como esta. Funcionalmente, é o mesmo que um espaço em branco, então você não precisa se preocupar em quebrar a linha no seu código.

Aqui criamos um vetor que possui Strings e números. Por ser um vetor, podemos pegar itens de posições variadas, como em (get cantor-arretado 4) . Perceba que, ao digitar cantor-arretado (sem os parênteses), o resultado não contém "MPB" . O que acontece é que, por padrão, as estruturas de dados em Clojure são imutáveis - também tema da próxima parte do livro.

Diferentemente dos vetores, com listas não conseguimos pegar diretamente um elemento de uma posição específica. Para isso, utilizamos a função nth , que nos permite pegar elementos em uma determinada posição. O problema aqui é que esta função precisa percorrer elemento por elemento na lista, então tem um pouco de perda de desempenho.

```
;; listas também podem conter elementos de tipos variados
(def cantora-arretada (list "Renata Arruda" "João Pessoa"
                             23 "dezembro" 1967))
;; #'user/cantora-arretada
```

```

(nth cantora-arretada 0)
;; "Renata Arruda"

(nth cantora-arretada 4)
;; 1967

(last cantora-arretada)
;; 1967

(first cantora-arretada)
;; "Renata Arruda"

;; conj, aqui, adiciona elementos ao início da lista
(conj cantora-arretada "MPB")
;; ("MPB" "Renata Arruda" "João Pessoa" 23 "dezembro" 1967)

```

3.3 SETS

Assim como em quase toda linguagem de programação mundo afora, Clojure também traz uma estrutura de dados que mantém uma lista de valores únicos: os *sets*.

```

;; Podemos criar sets assim:
(hash-set "Chico César" "Renata Arruda")
;; #{ "Renata Arruda" "Chico César" }

;; ou assim:
#{ "Chico César" "Renata Arruda" }
;; #{ "Renata Arruda" "Chico César" }

(def artistas #{ "Chico César" "Renata Arruda" })
;; #'user/artistas

;; Que tal incluir um artista novo na lista?
(conj artistas "Jackson do Pandeiro")
;; #{ "Renata Arruda" "Jackson do Pandeiro" "Chico César" }

;; E um que já existe?
(conj artistas "Chico César")
;; #{ "Renata Arruda" "Chico César" }

```

Como esperado, "Jackson do Pandeiro" entra no conjunto artistas , mas em uma posição que é difícil de prever. E "Chico César" não aparece duplicado no conjunto, porque já tem um lá.

3.4 CONCLUSÃO

Neste capítulo, vimos um pouco das estruturas de dados que Clojure oferece: listas, vetores e conjuntos. Mas ainda tem um pouco mais para vermos no próximo capítulo: mapas. Além disto, descreveremos o exemplo que vamos trabalhar ao longo do resto do livro, que é um controle financeiro pessoal. Quem sabe não vira um produto de sucesso?

CONTROLE FINANCEIRO E NOVAS ESTRUTURAS DE DADOS

Até aqui nós trabalhamos com alguns simples exemplos de como utilizar alguns recursos de Clojure. A partir deste capítulo, vamos trabalhar com um exemplo um pouco mais elaborado, de modo que possamos ver, aos poucos, tanto os conceitos de Programação Funcional, quanto novos recursos da linguagem. A ideia é criarmos uma ferramenta que nos ajude a controlar nossas finanças, registrando transações de entrada e saída de dinheiro e relatórios diversos. Um apanhado das funcionalidades seria:

- Registrar transações de receita e despesa
- Agrupar transações por categoria (alimentação, lazer, moradia)
- Agrupar transações por período
- Agrupar transações por rótulos (*tags* como *desnecessária*, *inesperada* e *cara*)

E, claro, todo desenvolvimento será de forma iterativa. A começar pelas estruturas de dados que vamos utilizar.

4.1 KEYWORDS

Conhecidas como símbolos em outras linguagens (Ruby, por exemplo), *keywords* são estruturas de dados muito simples, utilizadas principalmente como chaves de mapas. Lembra quando usamos `:else` no capítulo 1? Ali utilizamos este símbolo para declarar alguma coisa não nula para que fosse sempre executada no fluxo da avaliação de condições.

Existe um tipo em Clojure chamado *Symbol*, que não deve ser confundido com keywords. Recomendo o livro *The Joy of Clojure*, de Michael Fogus e Chris Houser, se você quiser saber mais sobre este tipo.

```
;; uma keyword  
:a  
;; :a
```

Não parece fazer muita coisa, não é? Mas vamos agora conhecer os mapas, e você verá o poder das keywords!

4.2 MAPAS

Pensando no exemplo que descrevemos há pouco, seria normal já partirmos para a modelagem de classes. No mundo de Clojure, no entanto, trabalhamos com mapas para representar as entidades com as quais vamos trabalhar. Logo, em vez de uma classe com atributos `tipo` e `valor`, nós trabalhamos com mapas com chaves `tipo` e `valor`. Assim:

```
;; um mapa pode ser criado assim
(hash-map :valor 200 :tipo "receita")
;; {:valor 200, :tipo "receita"}

;; ou assim
(def transacao {:valor 200 :tipo "receita"})
;; #'user/transacao

;; adicionando mais um par chave-valor ao mapa transacao
(assoc transacao :categoria "Educação")
;; {:valor 200, :tipo "receita", :categoria "Educação"}

;; mas o mapa original é mantido intacto
transacao
;; {:valor 200, :tipo "receita"}

;; para pegar elementos do mapa
(get transacao :valor)
;; 200
```

Vírgulas podem ser usadas também para separar elementos de um conjunto, mas são dispensáveis. Clojure vai tratá-las como um espaço em branco também. Eu não as uso, mas você as verá com frequência na saída do REPL, quando um mapa é impresso. Perceba que não há vírgula na definição de `transacao`, mas elas aparecem quando o REPL a imprime, separando pares.

E é com estruturas de dados simples assim que muitos programas complexos são feitos em Clojure. No código de exemplo anterior, declaramos um mapa com dois pares de chave-valor. Logo a seguir, adicionamos um outro par, cuja chave é `:categoria`.

Normalmente as chaves de mapas são keywords, mas não

precisam ser:

```
"chave muito louca" "de verdade"}  
;; {"chave muito louca" "de verdade"}
```

Mas quando utilizamos keywords como chaves, temos um poder especial de utilizá-las para facilitar a busca pelo valor:

```
;; podemos pegar o valor assim:  
(get transacao :valor)  
;; 200  
  
;; ou assim:  
(:valor transacao)  
;; 200
```

O que acontece aqui é que keywords podem ser utilizadas como função para pegar valores em mapas, o que simplifica muito o trabalho com esta estrutura de dados. Podemos também ter valores opcionais caso a chave não seja encontrada no mapa:

```
(def transacao-desnecessaria {:valor 34  
                              :tipo "despesa"  
                              :rotulos '("desnecessária"  
                                          "cartão")})  
  
;; #'user/transacao-desnecessaria  
  
(:rotulos transacao-desnecessaria)  
;; ("desnecessária" "cartão")  
  
(:rotulos transacao)  
;; nil  
  
(:rotulos transacao '())  
;; ()
```

Nesse código, criamos um mapa que tem como um dos valores uma lista (cuja chave é `:rotulos`), e dele obtemos tal lista usando a chave. Tentamos fazer o mesmo com `transacao`, que não possui a chave `:rotulos`, e, portanto, não retorna nada (`nil`).

Em seguida tentamos obter o valor de `:rotulos` novamente, mas desta vez indicamos que teremos uma lista vazia (`'()`) caso a chave `:rotulos` não seja encontrada.

E por que retornar alguma coisa em vez de nulo quanto uma chave não é encontrada em um mapa? Trabalhar com a possibilidade de valores nulos permite a introdução de diversos problemas, vide a apresentação do erro bilionário: <https://www.infoq.com/presentations/Null-References-The-Billion-Dollar-Mistake-Tony-Hoare/>. Tanto que algumas linguagens de programação mais novas nem trazem mais este conceito. Sempre que possível, evite a possibilidade de ter nulos como retorno.

4.3 CONCLUSÃO

E é tudo isso que temos sobre o básico de linguagem! Clojure é uma linguagem de programação muito simples, e é comum ver pessoas creditando o seu poder a esta simplicidade. Aqui encerramos esta parte do livro, focada em trazer o básico da linguagem para que possamos dar vida a ideias.

Na próxima parte, focaremos em princípios da Programação Funcional, elaborando os conceitos em cima do que aprendemos da linguagem e no exemplo que mencionamos neste capítulo. Prepare-se, um novo paradigma de programação espera por você!

Embarque no mundo da Programação Funcional

PROGRAMAÇÃO FUNCIONAL, O COMEÇO

Acabamos de nos familiarizar com Clojure e agora está na hora de começarmos a conversa sobre os princípios da Programação Funcional. A ideia é abordar o assunto de forma um tanto pragmática. Ao fim desta parte, espero que você tenha uma boa compreensão do essencial deste paradigma, já levando algumas ideias para o seu dia a dia. E, claro, que aproveite este conhecimento para escrever programas melhores. Além disso, caso você queira se aprofundar em teorias e conceitos matemáticos, há referências esperando por você no fim do livro.

Programação Funcional não é novidade alguma. De fato, é baseada no cálculo *lambda*, desenvolvido em meados dos anos 1930. Mas, por muito tempo, linguagens de Programação Funcional eram tidas como lentas e exigiam maior consumo de memória se comparadas às linguagens de Programação Imperativa. Hoje, estes argumentos são infundados, já que compiladores de linguagens funcionais, há bastante tempo, trazem recursos como *tail call optimization* (tema para o capítulo 7), que melhoram, e muito, o uso de recursos computacionais.

Por um lado, poderemos tirar proveito de novidades

interessantes. Como dito na introdução do livro, o aumento do número de núcleos computacionais pede por programas que lidem bem com concorrência. Dada a natureza de linguagens funcionais, que prezam por dados imutáveis, elas se tornam ideais para cenários assim, já que é minimizado o risco de alterações indesejadas no estado de um sistema.

Por outro lado, precisaremos sair da nossa zona de conforto. Abriremos mão de alguns conceitos do mundo de linguagens imperativas que não estarão mais presentes, como *loops* com *for* ou até mesmo variáveis. E assim precisaremos pensar de uma forma diferente para resolver alguns problemas. Um exemplo disto são os próprios *loops*, como vimos no capítulo 3, no exemplo do uso da função *map*. E aqui fica o convite para a introdução do primeiro conceito: a importância de funções.

5.1 FUNÇÕES: PRIMEIRA CLASSE E GRANDEZA SUPERIOR

Por mais óbvio que soe, preciso dizer que funções são muito relevantes em Programação Funcional, não é à toa este nome! Funções são tão importantes que são consideradas de primeira classe. Isto quer dizer que funções são tratadas como valores: assim como você passa um número ou uma String para uma função, você também pode passar uma função como argumento. Em JavaScript, por exemplo, temos o seguinte:

```
// código JavaScript
var autor = "George Orwell";
var nomeDoLivro = "1984";
var anoDePublicacao = 1949;
// variáveis 'normais'
```

```
let descricaoCompleta = (autor, nomeDoLivro, anoDePublicacao) =>
{
    return nomeDoLivro + ", " + autor + ", " + anoDePublicacao;
}
// variável que é uma função
```

Em JavaScript, podemos declarar variáveis cujo tipo são funções, e isso faz com que JavaScript seja uma linguagem que possui funções como cidadãs de primeira classe. Java é um exemplo de linguagem em que funções (métodos) não são consideradas cidadãs de primeira classe, apesar de ser possível a passagem de funções como argumento para um método (a partir do Java 8). Um recurso que falta ao Java, nesse sentido, é a capacidade de um método retornar uma função.

Com toda esta importância, ganhamos um recurso a mais para lidar com funções, que são as funções de grandeza superior (em inglês, *higher-order functions*). Funções de grandeza superior são funções que recebem funções como argumento, ou que retornam uma função como resultado. Por exemplo, veja este trecho de código para lidar com o registro de senhas, usando a técnica (simplificada) de *salting* (uma técnica que anexa uma String aleatória a uma senha antes que ela seja criptografada e salva):

```
// código JavaScript
let salgarSimples = (valor) => {
    return valor + "sal"
};
// função que aplica um sal para um texto

salgarSimples("senhainsegura");
// "senhainsegurasal"
// perceba que "sal" foi anexado ao final da senha, como esperado

let gravarCliente = (nome, senha, comoSalgar) => {
    console.log("Salvando cliente de nome '" + nome + "'");
    console.log("Senha salva: " + comoSalgar(senha));
```

```
};
// função que 'salva' cliente no console

gravarCliente("Gregório", "senhasegurademaiss", salgarSimples);
// Salvando cliente de nome 'Gregório'
// Senha salva: senhasegurademaissal
```

Repare que o argumento `salgar` é uma função e é utilizada como argumento para `gravarCliente`. Isso permite que `gravarCliente` não se importe com a estratégia que será utilizada para salvar a senha.

E agora veremos como utilizaremos esse recurso no nosso domínio.

5.2 FUNÇÕES DE GRANDEZA SUPERIOR E NOSSAS FINANÇAS

Já vimos um exemplo de uso de função de grandeza superior no capítulo 3, utilizando a função `map`. `map` é o recurso que Clojure provê (função) para que iteremos sobre uma coleção, e, a cada iteração, aplica-se uma função. Por fim, é retornada uma outra coleção como resultado. Vamos ver um exemplo?

Iterando sobre transações

Imagine que temos uma coleção de transações e, para a aplicação cliente, nosso serviço deve retornar um conjunto reduzido de dados: o valor, o tipo da transação (receita ou despesa) e a data. Nosso programa, então, deve varrer cada mapa e pegar só chaves e valores que nos interessem:

```
(defn resumo [transacao]
  (select-keys transacao [:valor :tipo :data]))
;; função que cria um resumo de uma transação
```

```

(def transacoes
  [{:valor 33.0 :tipo "despesa"
    :comentario "Almoço" :data "19/11/2016"}
   {:valor 2700.0 :tipo "receita"
    :comentario "Bico" :data "01/12/2016"}
   {:valor 29.0 :tipo "despesa"
    :comentario "Livro de Clojure" :data "03/12/2016"}])
;; as transações

(map resumo transacoes)
;; ({:valor 33.0, :tipo "despesa", :data "19/11/2016"}
;;  {:valor 2700.0, :tipo "receita", :data "01/12/2016"}
;;  {:valor 29.0, :tipo "despesa", :data "03/12/2016"})

```

Perceba que o resultado foram transações sem o campo de comentário. A função nativa `select-keys` permite que peguemos só alguns valores de um mapa. Daí, com `map`, pegamos o resumo de cada item de `transacoes`. E se quisermos procurar algum padrão nas despesas, e filtrando as transações que são do tipo "despesa" ?

Filtrando transações

A função `filter` pode nos ajudar aqui. Assim:

```

(defn despesa? [transacao]
  (= (:tipo transacao) "despesa"))
;; função que verifica se uma transação é uma despesa,
;; verificando o valor para a chave :tipo

(filter despesa? transacoes)
;; ({:valor 33.0, :tipo "despesa",
;;   :comentario "Almoço", :data "19/11/2016"}
;;  {:valor 29.0, :tipo "despesa",
;;   :comentario "Livro de Clojure", :data "03/12/2016"})

```

Das 3 transações que temos, apenas uma é uma receita. E `filter` separou para nós só as que queremos, deixando a receita

de fora.

Agora que as despesas estão separadas, que tal somar o valor delas?

Condensando resultado de iterações

Depois de aplicarmos `filter`, conseguimos uma outra coleção com os valores que queríamos. E agora precisamos somar todos eles. Usando uma versão do Java anterior à 8, teríamos que fazer algo assim:

```
public Integer somaDosValores(List<Integer> valores) {
    Integer soma = 0;

    for (Integer valor : valores) {
        soma += valor;
    }

    return soma;
}
```

Linguagens de programação com recursos funcionais facilitam essa operação através de uma função conhecida por diversos nomes, de acordo com a linguagem. Em Clojure, ela se chama `reduce`, mas pode ser encontrada com os seguintes nomes em outras linguagens: `fold`, `aggregate`, `accumulate`, `inject`, dentre outros nomes.

Então, vamos utilizar o `reduce` para fazer a soma de todos os valores da coleção de valores das despesas?

```
(defn so-valor [transacao]
  (:valor transacao))
;; função que pega só o valor de uma transação

(map so-valor (filter despesa? transacoes))
```

```
;; (33.0 29.0)
;; pegamos só os valores das despesas, que serão
;; condensados com reduce

(reduce + (map so-valor (filter despesa? transacoes)))
;; 62.0
;; a soma dos valores das despesas
```

`reduce` é uma função que combina todos os itens de uma coleção em um valor só. `reduce` aplica uma função que vai resultando em um valor acumulado. Aqui, ela começa com um valor nulo e aplica a função passada (`+` , neste caso) ao primeiro item da lista, e coloca o resultado em uma variável interna que costumamos chamar de *acumulador*. Então, ela parte para os elementos seguintes, aplicando `+` entre o valor acumulado e o próximo item da coleção.

Se você conhece Ruby, já deve ter visto uma função similar a `reduce` , chamada `inject` . Ela funciona mais ou menos assim:

```
# código Ruby
valores = [33.0, 29.0]

valores.inject(:+)
# 62.0
```

Esta combinação `filter` , `map` , e `reduce` é muito popular e poderosíssima! Você certamente encontrará muitas destas funções por aí, em diversas linguagens, e por isso fiz questão de trazê-las o mais cedo possível no livro!

Funções anônimas

Até aqui, usamos duas coisas bem comuns para associar valores a nomes: `def` para dar nome a algum dado, e `defn` para dar nome a uma função que nós criamos. Acontece que nem

sempre precisamos dar um nome a alguma função, já que às vezes elas são utilizadas apenas uma vez. Nestes casos, podemos preferir que tais funções sejam anônimas. Por exemplo, no código JavaScript que salga uma senha, a função de salgar, quando anônima, ficaria assim:

```
gravarCliente("Gregório", "senhasegurademaís", function(senha) {  
    return "salgrosso" + senha;  
});
```

Vamos ver como ficaria em Clojure? Que tal pegarmos o exemplo para ver somente valores acima de 100? Começando com a função de filtragem declarada:

```
(defn valor-grande? [transacao]  
  (> (:valor transacao) 100))  
;; verifica se o valor de uma transação é maior que 100  
  
(filter valor-grande? transacoes)  
;; ({:valor 2700.0, :tipo "receita",  
;;   :comentario "Bico", :data "01/12/2016"})
```

Ora, o trecho de código na função que usamos para filtrar valores acima de 100 é bem pequeno e talvez seja utilizado em um só lugar. Parece ter um bom potencial para virar uma função anônima! Então, chegou a hora de entendermos melhor o que o `defn` faz.

Clojure é sintaticamente muito simples, com poucas palavras-chaves e notações nativas, sendo `def` um destes recursos nativos (*special form*, como consta na documentação da linguagem). Só que Clojure provê um recurso chamado *macros* (não são como as macros do Excel, ok?), que permite que a linguagem seja estendida. Às vezes, o mais simples é entender que algumas macros provêm atalhos. Este é o caso de `defn` : uma macro para definição de

funções.

Lembra-se de que funções são cidadãs de primeira classe em Clojure? Isso significa que funções podem ser associadas a nomes, assim como uma coleção, por exemplo. `def` é o que utilizamos para dar nome a algo, como nestes exemplos:

```
(def impares '(1 3 5 7 9))
(def ano-do-pentacampeonato-do-brasil 2002)
(def pi 3.14)
(def um-terco 1/3) ;; acredite, uma razão é um tipo válido
```

Existe outra forma especial (como `def`) para criar uma função: `fn` . Só que é uma função sem nome, que ninguém consegue chamar:

```
(fn [])
;; uma função que não recebe nenhum argumento e não faz nada

(fn [nome]
  (str "Olá, " nome "!"))
;; uma função que recebe um argumento, nome, e retorna uma
;; mensagem carinhosa.
```

Acontece que estas funções acabam ficando no limbo, porque não há como nós a referenciarmos. A não ser que já as executemos no momento da sua criação:

```
((fn [nome]
  (str "Olá, " nome "!"))
 "mundo novo")
;; "Olá, mundo novo!"
```

Perceba que há dois parênteses antes de `fn` : um significa que vamos chamar alguma função. O outro é o começo da construção da função sem nome. Logo após o parêntese que fecha a definição da função, passamos o único argumento que a função sem nome espera. Se tivéssemos usado `defn` , ficaria assim:

```
(defn ola [nome]
  (str "Olá, " nome "!"))

(ola "mundo novo")
```

O truque com `defn` é que ele nada mais é que uma macro que nos proporciona um atalho que, no fundo, nos serve como isso aqui:

```
(def ola (fn [nome]
  (str "Olá, " nome "!")))

(ola "mundo novo")
```

O que acontece aqui é que usamos `fn` para criar uma função, `def` para dar nome às coisas, e `defn` para facilitar nossa vida quando precisamos dar nomes a funções.

Agora que sabemos que como criar nossas funções anônimas, como faríamos para filtrar nossa coleção de transações sem aquela valor-grande? ? Assim:

```
(filter (fn [transacao]
  (> (:valor transacao) 100))
  transacoes)
;; ({:valor 2700.0, :tipo "receita",
;;   :comentario "Bico", :data "01/12/2016"})
```

O primeiro argumento para `filter` é uma função anônima, e o segundo é a coleção, `transações`.

Como funções anônimas são tão comuns nessa vida, existe uma forma curta de descrevê-las. Lembra que existem funções para criar listas e mapas, mas que existem formas curtas para criar estas estruturas de dados? O mesmo recurso é disponibilizado para funções anônimas. Assim:

```
(filter #(> (:valor %) 100)
```

transacoes)

Aqui utilizamos a forma `#(...)` para encurtar a declaração da função anônima. Perceba que agora não temos mais um argumento com o nome; antes tínhamos um argumento chamado `transacao`, e agora ele não é mais tão explícito. Mesmo assim, o argumento ainda existe. Lembre-se, `filter` vai passar por cada elemento da coleção e passá-lo para a função que citamos como primeiro argumento. Daí, cada elemento da coleção vira argumento da função anônima. Só que não precisamos dar um nome a este argumento. Tanto faz. Por isso, podemos referenciá-lo com `%`, como em `(:valor %)`.

E SE MINHA FUNÇÃO ANÔNIMA RECEBER MAIS DE UM ARGUMENTO?

Você poderá utilizar `%1`, `%2`, `%3`, daí em diante, para referenciar os argumentos na ordem em que eles são passados.

Tendo visto como lidar com funções anônimas, podemos utilizá-las para o caso de condensarmos todos os valores das despesas em um só valor. Só para lembrar, o código era assim:

```
(defn despesa? [transacao]
  (= (:tipo transacao) "despesa"))

(defn so-valor [transacao]
  (:valor transacao))

(reduce + (map so-valor (filter despesa? transacoes)))
;; 62.0
```

E pode ficar assim:

```
(reduce + (map #(:valor %)
              (filter #(= (:tipo %) "despesa")
                        transacoes)))
;; 62.0
```

Mas, e agora? Qual forma devo utilizar? Veja, atingimos o mesmo resultado de ambas as formas. Fica a seu critério qual forma utilizar quando ambas estiverem disponíveis.

Lendo código Clojure de uma outra forma

O código do exemplo anterior nos ajuda muito a contar como código em Clojure é lido diferente de outras linguagens. Primeiro procuramos saber o que acontece com `filter`, depois com `map` e, por fim, `reduce`. Mesmo que nossos olhos enxerguem primeiro `reduce`, depois `map` e `filter`. É bem diferente de muitas outras linguagens, não?

Para ajudar um pouco com a legibilidade, às vezes até vemos códigos organizados da seguinte forma, e então tentamos ler de dentro para fora:

```
(reduce +
  (map so-valor
    (filter despesa?
      transacoes)))
```

Mas isso ainda pode não ser tão legível em casos complexos, ou nem ser a preferência de algumas pessoas. Eis que surge uma macro interessante: `->`. Ela nos ajuda a escrevermos código deixando clara a ordem de execução. Imagine um caso simples onde queremos pegar só o valor da primeira transação de uma coleção:

```
(so-valor (first transacoes))
;; 33.0
```

Lemos de dentro para fora: primeiro pegamos o primeiro elemento com `first` e depois pegamos só o valor através de `so-valor`. Com a macro `->`, este código ficaria da seguinte forma:

```
(-> (first transacoes)
    (so-valor))
;; 33.0
```

Fica bem mais claro qual a ordem de execução, certo? Esta macro se chama *thread-first*. Apesar de ter *thread* no nome, não há nada de concorrência aqui. Mas vale pensar em *thread* como um fluxo, e o *first* (de *thread first*) é importante porque significa que o resultado de uma linha é usado como o primeiro argumento da função seguinte. É por isso que `so-valor` aparenta não receber nenhum argumento, quando, de fato, recebe o resultado da função anterior como seu primeiro argumento.

Existe uma outra macro parecida: `->>`, chamada de *thread-last*. Ela é útil nos casos em que precisamos passar o resultado da aplicação de uma função como o último argumento da função seguinte. Por exemplo:

```
(->> (filter despesa? transacoes)
      (map so-valor)
      (reduce +))
;; 62.0
```

Tanto `map` quanto `reduce` recebem uma função como primeiro argumento, e por isso a macro `->`, *thread-first*, não nos serve. Então, o resultado de `(filter despesa? transacoes)` é passado como último argumento para `(map so-valor)`, e seu resultado é passado como último argumento para `(reduce +)`.

É bem comum ver ambas as macros utilizadas por aí! Sinta-se à vontade para utilizá-las. Meu critério para usar uma destas macros

é quando há muitas operações em série em cima de um mesmo argumento inicial. Um caso muito comum é na escrita de *middlewares* para o Ring (<https://github.com/ring-clojure/ring>), uma biblioteca para aplicações Web. Um exemplo do uso do Ring é o seguinte:

```
(def app
  (-> (wrap-json-body app-routes {:keywords? true
                                   :bigdecimals? true})
      (wrap-json-response)
      (wrap-request-logger)
      (wrap-exception-handler)
      (wrap-response-logger)))
```

5.3 CONCLUSÃO

Um vasto capítulo, hein? Aqui vimos algo essencial para a prática da Programação Funcional: funções de grandeza superior e funções como cidadãos de primeira classe. Junto com alguns exemplos de uso de funções célebres (como `filter`, `map` e `reduce`), vimos como Clojure se diferencia no processo de aplicar funções a itens de uma coleção. De quebra, ainda tivemos alguns detalhes sobre Clojure com as macros `->` e `->>`. No próximo capítulo vamos um pouco além nesse mundo de funções, abordando os temas de composição e *currying*.

COMPOSIÇÃO DE FUNÇÕES E APLICAÇÃO PARCIAL DE FUNÇÕES

No capítulo anterior, vimos o poder de ter funções como cidadãs de primeira classe. Em linguagens que possuem este recurso, podemos usar funções como argumentos, assim como fazemos com outros tipos de dados. Isso permite que façamos algo um tanto rebuscado, como combinar várias funções em uma só, unindo vários comportamentos. É o que chamamos de composição de funções.

Existe um epigrama famoso do Alan Perlis, que diz: *"É melhor ter 100 funções para operar em uma estrutura de dados, do que 10 funções para operar em 10 estrutura de dados"*. Ele é muito referenciado em textos e apresentações sobre Programação Funcional, e Clojure representa muito bem este espírito, sendo a composição de funções uma das facilidades da linguagem.

No nosso domínio, que tal um exemplo em que geremos uma versão em texto do resumo de uma transação? Imagine uma função `texto-resumo` que faz com que uma transação vire o seguinte texto:

```
(texto-resumo uma-transacao-qualquer)
;; "01/12/2016 => R$ +2700.0"
```

É esperado que `texto-resumo` pegue a data, a moeda e o valor de uma despesa e os exiba com uma seta entre eles, com o valor precedido por um símbolo que indica se a transação é despesa ou receita. Vamos precisar da moeda na nossa coleção de transações, então segue uma nova definição de transações:

```
(def transacoes
  [{:valor 33.0 :tipo "despesa" :comentario "Almoço"
    :moeda "R$" :data "19/11/2016"}
   {:valor 2700.0 :tipo "receita" :comentario "Bico"
    :moeda "R$" :data "01/12/2016"}
   {:valor 29.0 :tipo "despesa" :comentario "Livro de Clojure"
    :moeda "R$" :data "03/12/2016"}])
```

SOBRE O ESTILO DO CÓDIGO

Você vai perceber que alguns blocos de código possuem quebra de linha que pode comprometer um pouco sua estética. Isso acontece porque há um limite de 67 caracteres por linha de código para a exibição no livro e preferi quebrar linhas a permitir que um elemento de um código ficasse com um pedaço em uma linha e outro pedaço em outra.

O capítulo 14 referencia um formatador de código (cljfmt, <https://github.com/weavejester/cljfmt>). Se preferir, pode começar a utilizá-lo. Os códigos da parte 3 deste livro podem ser encontrados em <http://gitlab.com/programacaofuncional/> e possuem a formatação que eu gostaria de ter utilizado aqui.

Agora precisamos de uma função que pegue o valor de uma transação e coloque + ou - antes do seu valor:

```
(defn valor-sinalizado [transacao]
  (if (= (:tipo transacao) "despesa")
    (str "-" (:valor transacao))
    (str "+" (:valor transacao))))

(valor-sinalizado (first transacoes))
;; "-33.0"

(valor-sinalizado (second transacoes))
;; "+2700.0"
```

Falta a moeda ainda, não é? Vamos atualizar valor-sinalizado :

```
(defn valor-sinalizado [transacao]
  (if (= (:tipo transacao) "despesa")
```

```

(str (:moeda transacao) " -" (:valor transacao))
(str (:moeda transacao) " +" (:valor transacao)))

(valor-sinalizado (first transacoes))
;; "R$ -33.0"

(valor-sinalizado (second transacoes))
;; "R$ +2700.0"

```

Há uma certa duplicação em `valor-sinalizado`. Estamos repetindo a forma de pegar a moeda e o valor de uma transação. Clojure possui uma outra forma especial que nos ajuda a associar valores a nomes, que é a forma `let`:

```

(defn valor-sinalizado [transacao]
  (let [moeda (:moeda transacao)
        valor (:valor transacao)]
    (if (= (:tipo transacao) "despesa")
      (str moeda " -" valor)
      (str moeda " +" valor))))

(valor-sinalizado (first transacoes))
;; "R$ -33.0"

(valor-sinalizado (second transacoes))
;; "R$ +2700.0"

```

Neste exemplo, `moeda` e `valor` podem ser entendidas como constantes locais. `let` permite que usemos `moeda` e `valor` dentro de todo o escopo que foi criado com `(let , até que seu )` apareça para encerrá-lo. Ela recebe um array com pares de elementos, nos quais os pares correspondem ao nome que queremos, e cada nome é seguido do valor. Neste caso, `moeda` é o nome e seu par, `(:moeda transacao)`, vira seu valor. O mesmo acontece com `valor` e `(:valor transacao)`.

Vamos aproveitar que reduzimos a redundância para expandir um pouco mais o conhecimento de como manipular mapas em

Clojure? Imagine que uma transação não tenha uma moeda definida, e, em um caso como este, queremos que "R\$" seja retornado por padrão. Sem uma moeda definida, como na seguinte transação aleatória, o resultado de `valor-sinalizado` fica assim:

```
(def transacao-aleatoria {:valor 9.0})

(valor-sinalizado transacao-aleatoria)
;; " +9.0"
```

Perceba que não veio nada como moeda. Se quisermos ter sempre "R\$" como valor padrão sempre que não há uma moeda definida, podemos dizer isso à função que pega o valor de uma chave em um mapa:

```
(:moeda transacao-aleatoria)
;; nil

(:moeda transacao-aleatoria "R$")
;; "R$"
```

E agora podemos atualizar nossa função `valor-sinalizado` para lidar com transações sem moeda definida:

```
(defn valor-sinalizado [transacao]
  (let [moeda (:moeda transacao "R$")
        valor (:valor transacao)]
    (if (= (:tipo transacao) "despesa")
      (str moeda " -" valor)
      (str moeda " +" valor))))

(valor-sinalizado (first transacoes))
;; "R$ -33.0"

(valor-sinalizado (second transacoes))
;; "R$ +2700.0"

(valor-sinalizado transacao-aleatoria)
;; "R$ +9.0"
```

Agora que conseguimos transformar uma transação no texto que queremos, precisamos de uma função que exiba uma transação com a data, na formatação que mencionamos antes: `data => R$ +-valor` .

```
(defn data-valor [transacao]
  (str (:data transacao) " => " (valor-sinalizado transacao)))

(data-valor (first transacoes))
;; "19/11/2016 => R$ -33.0"
```

E quisermos converter os valores para uma moeda diferente? Digamos que queremos converter o valor de uma transação para a moeda chinesa, o ¥ (leia-se *yuan*). Vamos assumir que R\$ 1,00 se equivale a ¥ 2,15. O que precisamos fazer é ter uma nova transação cujo valor é convertido para a moeda em questão, e o símbolo desta moeda é colocado na transação:

```
(defn transacao-em-yuan [transacao]
  (assoc transacao :valor (* 2.15 (:valor transacao))
         :moeda "¥"))

(transacao-em-yuan (first transacoes))
;; {:valor 70.95, :tipo "despesa", :comentario "Almoço",
;;  :moeda "¥", :data "19/11/2016"}
```

`assoc` , que vem do inglês *associate*, pega o mapa passado (`transacao`), e associa a este mapa os valores que seguem como argumento. Neste exemplo, `transacao` terá o valor das chaves `:valor` e `:moeda` atualizados, respectivamente, para 70.95 (`(* 2.15 33.0)`) e "¥" . Vamos dar uma pequena melhoria no exemplo removendo os valores de conversão de yuan de dentro da função `transacao-em-yuan` :

```
(def cotacoes
  {:yuan {:cotacao 2.15 :simbolo "¥"}})
```

```
(defn transacao-em-yuan [transacao]
  (assoc transacao :valor (* (:cotacao (:yuan cotacoes))
                             (:valor transacao))
          :moeda (:simbolo (:yuan cotacoes))))

(transacao-em-yuan (first transacoes))
;; { :valor 70.95, :tipo "despesa", :comentario "Almoço",
;;   :moeda "¥", :data "19/11/2016" }
```

Resolvemos um problema, mas criamos outro: removemos os valores de dentro de `transacao-em-yuan`, mas pegar os valores da cotação e o símbolo da moeda ficou mais difícil. Da forma como está, é preciso aninhar o processo de pegar o valor de uma chave, sendo este valor um outro mapa, com pares de chaves e valores.

Para facilitar, temos a função `get-in` que permite simplificar casos assim. Aqui vemos como buscar valores aninhados em um mapa de forma mais simples:

```
(defn transacao-em-yuan [transacao]
  (assoc transacao :valor (* (get-in cotacoes [:yuan :cotacao])
                             (:valor transacao))
          :moeda (get-in cotacoes [:yuan :simbolo])))

(transacao-em-yuan (first transacoes))
;; { :valor 70.95, :tipo "despesa", :comentario "Almoço",
;;   :moeda "¥", :data "19/11/2016" }
```

`get-in` recebe um mapa (`transacao`), que é de onde vamos pegar o valor desejado, e um array, que descreve o caminho até o valor, composto pelas chaves (`[:yuan :cotacao]` ou `[:yuan :simbolo]`). Se nosso mapa tivesse outras moedas, poderíamos fazer a mesma pesquisa, mas com `[:euro :cotacao]` e `[:euro :simbolo]` .

Ainda assim está um pouco mais complicado do que precisa

ser. `let` pode nos ajudar mais uma vez:

```
(defn transacao-em-yuan [transacao]
  (let [yuan (:yuan cotacoes)]
    (assoc transacao :valor (* (:cotacao yuan) (:valor transacao))
           :moeda (:simbolo yuan))))

(transacao-em-yuan (first transacoes))
;; {:valor 70.95, :tipo "despesa", :comentario "Almoço",
;;  :moeda "¥", :data "19/11/2016"}
```

Ora, agora podemos praticar a formatação com `data-valor` com uma transação tanto em real quanto em yuan:

```
(data-valor (first transacoes))
;; "19/11/2016 => R$ -33.0"

(data-valor (transacao-em-yuan (first transacoes)))
;; "19/11/2016 => ¥ -70.95"

(defn texto-resumo-em-yuan [transacao]
  (data-valor (transacao-em-yuan transacao)))

(map texto-resumo-em-yuan transacoes)
;; ("19/11/2016 => ¥ -70.95" ...
;;  "01/12/2016 => ¥ +5805.0" ...
;;  "03/12/2016 => ¥ -62.349999999999994")
```

Ops! 62.349999999999994 !? Bem, devido à imprecisão em operações aritméticas com números de pontos flutuantes (a cotação da moeda, 2,15, e o próprio valor da transação, 29,0), temos este número estranho. Em Clojure, pontos flutuantes e inteiros do tipo `BigInt` são contagiantes, o que significa que qualquer operação que envolva um `double` retornará um `double`. O mesmo acontece com `BigInt`.

Para termos uma melhor precisão em operações aritméticas, precisamos fazer uso do tipo `BigDecimal`:

```

(class 3.1)
;; java.lang.Double

(* 3.1 3.1)
;; 9.6100000000000001

(class 3.1M)
;; java.math.BigDecimal

(* 3.1M 3.1)
;; 9.6100000000000001

(* 3.1M 3.1M)
;; 9.61M

(class (* 3.1M 3.1M))
;; java.math.BigDecimal

```

Perceba que `(* 3.1 3.1)` também dá um resultado estranho. Já `(* 3.1M 3.1M)` nos dá o resultado desejado. Para usar `BigDecimal` s, basta anexar `M` ao número, como em `3.1M` . Assim, vamos corrigir tanto nosso conjunto de transações, quanto a cotação do yuan:

```

(def cotacoes
  {:yuan {:cotacao 2.15M :simbolo "¥"}})

(def transacoes
  [{:valor 33.0M :tipo "despesa" :comentario "Almoço"
    :moeda "R$" :data "19/11/2016"}
   {:valor 2700.0M :tipo "receita" :comentario "Bico"
    :moeda "R$" :data "01/12/2016"}
   {:valor 29.0M :tipo "despesa" :comentario "Livro de Clojure"
    :moeda "R$" :data "03/12/2016"}])

```

E agora podemos aplicar `texto-resumo-em-yuan` a todas as transações em paz:

```

(map texto-resumo-em-yuan transacoes)
;; ("19/11/2016 => ¥ -70.950" ...)
;; ("01/12/2016 => ¥ +5805.000" ...)
;; ("03/12/2016 => ¥ -62.350")

```


Joia! Agora vamos voltar à definição de `texto-resumo-em-yuan` :

```
(defn texto-resumo-em-yuan [transacao]
  (data-valor (transacao-em-yuan transacao)))
```

Esta função é um bom caso para usarmos a macro `thread-first` (`->`), já que a função nada mais faz que aplicar `data-valor` aos dados retornados por `transacao-em-yuan` :

```
(defn texto-resumo-em-yuan [transacao]
  (-> (transacao-em-yuan transacao)
      (data-valor)))
```

Mas também é um bom exemplo para a introdução do conceito de composição de funções!

6.1 COMPOSIÇÃO DE FUNÇÕES

Se você se acostumou com a ideia de composição no mundo da Orientação a Objetos, aqui é parecido: combinamos funções em uma só, de modo que a aplicação de uma só função corresponda à aplicação de várias. Aqui, quando o propósito de uma função é apenas combinar outras funções, passando para uma o resultado da outra, há o forte indício de que podemos fazer uso da composição de funções. Existe um recurso em Clojure que nos permite construir a composição de duas ou mais funções: a função `comp`. Usando `comp`, quer dizer, fazendo composição de `data-valor` e `transacao-em-yuan`, `texto-resumo-em-yuan` fica assim:

```
(def texto-resumo-em-yuan (comp data-valor transacao-em-yuan))

(map texto-resumo-em-yuan transacoes)
;; ("19/11/2016 => ¥ -70.950" ...
```

```
;; "01/12/2016 => ¥ +5805.000" ...
;; "03/12/2016 => ¥ -62.350")
```

Perceba que usamos `def`, e não `defn`. Lembrando, `defn` é a forma curta para `(def nome-da-funcao (fn [argumento] (<alguma operação aqui>)))`, abstraindo a necessidade de declarar `fn`. `comp` funciona como `fn`, pois cria uma função para nós, cujo nome é `texto-resumo-em-yuan`. E o corpo desta função é a composição de `data-valor` e `transacao-em-yuan`, sendo que esta última é aplicada ao argumento passado, e o resultado é utilizado como argumento para `data-valor`. Assim, as 3 definições a seguir de `texto-resumo-em-yuan` se comportam da mesma forma:

```
(defn texto-resumo-em-yuan [transacao]
  (data-valor (transacao-em-yuan transacao)))

(map texto-resumo-em-yuan transacoes)
;; ("19/11/2016 => ¥ -70.950" ...
;;  "01/12/2016 => ¥ +5805.000" ...
;;  "03/12/2016 => ¥ -62.350")

(defn texto-resumo-em-yuan [transacao]
  (-> (transacao-em-yuan transacao)
      (data-valor)))

(map texto-resumo-em-yuan transacoes)
;; ("19/11/2016 => ¥ -70.950" ...
;;  "01/12/2016 => ¥ +5805.000" ...
;;  "03/12/2016 => ¥ -62.350")

(def texto-resumo-em-yuan (comp data-valor transacao-em-yuan))

(map texto-resumo-em-yuan transacoes)
;; ("19/11/2016 => ¥ -70.950" ...
;;  "01/12/2016 => ¥ +5805.000" ...
;;  "03/12/2016 => ¥ -62.350")
```

Composição de funções é bastante útil quando há a repetição

na aplicação de função que costumam acontecer juntas. É uma importante forma de reúso. Uma outra forma é a aplicação parcial de uma função, conhecida em inglês como *partial application*.

6.2 APLICAÇÃO PARCIAL

Vimos uma função que converte um valor de real pra yuan:

```
(defn transacao-em-yuan [transacao]
  (let [yuan (:yuan cotacoes)]
    (assoc transacao :valor (* (:cotacao yuan) (:valor transacao))

           :moeda (:simbolo yuan))))

(transacao-em-yuan (first transacoes))
;; {:valor 70.950M, :tipo "despesa", :comentario "Almoço",
;;  :moeda "¥", :data "19/11/2016"}
```

Existe um recurso em Clojure chamado de desestruturação, do inglês *destructuring*, que permite aprimorarmos o uso de `let` quando precisamos extrair valores de um argumento que é um mapa. A ideia é associarmos nomes a valores de chaves, da mesma forma que `let` faz, mas com uma sintaxe que vai simplificar o conteúdo dentro da função:

```
(defn transacao-em-yuan [transacao]
  (let [{:yuan :yuan} cotacoes]
    (assoc transacao :valor (* (:cotacao yuan) (:valor transacao))

           :moeda (:simbolo yuan))))

(transacao-em-yuan (first transacoes))
;; {:valor 70.950M, :tipo "despesa", :comentario "Almoço",
;;  :moeda "¥", :data "19/11/2016"}
```

`cotacoes` é um mapa e um de seus elementos possui `:yuan` como chave. No exemplo anterior, o `let` faz uso da desestruturação para extrair de `cotacoes` o valor de `:yuan` e o

associa a `yuan` . Não é necessário que o mesmo nome seja repetido!

No exemplo a seguir, aprofundamos o uso de desestruturação para fazer a associação de valores um nível adentro de `cotacoes` :

```
(defn transacao-em-yuan [transacao]
  (let [{:cotacao :cotacao simbolo :simbolo} :yuan} cotacoes]
    (assoc transacao :valor (* cotacao (:valor transacao))
           :moeda simbolo)))

(transacao-em-yuan (first transacoes))
;; {:valor 70.950M, :tipo "despesa", :comentario "Almoço",
;;  :moeda "¥", :data "19/11/2016"}
```

O que acontece no `let` é que estamos separando o mapa dentro de `:yuan` da `cotacoes` , e, deste mapa, associamos os valores de `:cotacao` e `:simbolo` aos nomes `cotacao` e `simbolo` .

Visando a possibilidade de reutilizar a função também para outras moedas, vamos extrair a moeda e utilizá-la como parâmetro. E já podemos incluir mais cotações na nossa tabela:

```
(def cotacoes
  {:yuan {:cotacao 2.15M :simbolo "¥"}
   :euro {:cotacao 0.28M :simbolo "€"}})

(defn transacao-em-outra-moeda [moeda transacao]
  (let [{:cotacao :cotacao simbolo :simbolo} moeda} cotacoes]
    (assoc transacao :valor (* cotacao (:valor transacao))
           :moeda simbolo)))

(transacao-em-outra-moeda :euro (first transacoes))
;; {:valor 9.240M, :tipo "despesa", :comentario "Almoço",
;;  :moeda "€", :data "19/11/2016"}

(transacao-em-outra-moeda :euro (last transacoes))
;; {:valor 8.120M, :tipo "despesa", :comentario "Livro de Clojure",
;;  :moeda "€", :data "19/11/2016"}
```

```
;; :moeda "€", :data "03/12/2016"}

(transacao-em-outra-moeda :yuan (first transacoes))
;; {:valor 70.950M, :tipo "despesa", :comentario "Almoço",
;; :moeda "¥", :data "19/11/2016"}

(transacao-em-outra-moeda :yuan (last transacoes))
;; {:valor 62.350M, :tipo "despesa",
;; :comentario "Livro de Clojure", :moeda "¥",
;; :data "03/12/2016"}
```

Perceba que a única coisa que mudou entre as aplicações de funções com euro ou yuan foi a moeda (:euro ou :yuan). Podemos partir para o conceito do momento, aplicação parcial de funções, para comprimir um pouco o uso de transacao-em-outra-moeda . A aplicação parcial de uma função significa que pegamos uma função que tem vários argumentos e criamos uma nova, baseada nela, mas que recebe menos argumentos. Por exemplo:

```
(def transacao-em-euro (partial transacao-em-outra-moeda :euro))

(def transacao-em-yuan (partial transacao-em-outra-moeda :yuan))

(transacao-em-euro (first transacoes))
;; {:valor 9.240M, :tipo "despesa", :comentario "Almoço",
;; :moeda "€", :data "19/11/2016"}

(transacao-em-yuan (first transacoes))
;; {:valor 70.950M, :tipo "despesa", :comentario "Almoço",
;; :moeda "¥", :data "19/11/2016"}
```

O que aconteceu nas primeiras linhas do exemplo anterior foi que demos um nome à aplicação parcial de uma função. No primeiro caso, o nome é transacao-em-euro e corresponde à aplicação parcial de transacao-em-outra-moeda já recebendo um dos dois argumentos, :euro para o argumento moeda . A mesma coisa acontece com transacao-em-yuan , mas com

:yuan para moeda . E é uma aplicação parcial porque declaramos parte dos argumentos, não todos. A outra parte do argumento virá em algum outro momento, quando a função for, de fato, aplicada.

Quando aplicamos `transacao-em-euro` a `(first transacoes)` , a função parcial `transacao-em-euro` é convertida na aplicação de `transacao-em-outra-moeda` , que recebe dois argumentos, mas um deles já está definido, `:euro` . O segundo é uma transação, que é passada como a primeira da coleção de transações.

Da forma como está, estas funções podem ser utilizadas como parte de aplicações de `map` :

```
(map transacao-em-yuan transacoes)
;; ({:valor 70.950M, :tipo "despesa", :comentario "Almoço",
;;   :moeda "¥", :data "19/11/2016"}
;;  {:valor 5805.000M, :tipo "receita", :comentario "Bico",
;;   :moeda "¥", :data "01/12/2016"}
;;  {:valor 62.350M, :tipo "despesa",
;;   :comentario "Livro de Clojure", :moeda "¥",
;;   :data "03/12/2016"})
```

```
(map transacao-em-euro transacoes)
;; ({:valor 9.240M, :tipo "despesa", :comentario "Almoço",
;;   :moeda "€", :data "19/11/2016"}
;;  {:valor 756.000M, :tipo "receita", :comentario "Bico",
;;   :moeda "€", :data "01/12/2016"}
;;  {:valor 8.120M, :tipo "despesa",
;;   :comentario "Livro de Clojure", :moeda "€",
;;   :data "03/12/2016"})
```

Funciona bem porque, lembrando, `map` pega cada elemento de `transacoes` e a ele aplica `transacao-em-yuan` ou `transacao-em-euro` , e estas funções só precisam de um argumento.

Às vezes, podemos utilizar aplicação parcial de funções para

simplificar o entendimento do que uma função faz. Por exemplo, para juntar elementos de um array em uma string, com os elementos separados por vírgula, fazemos o seguinte:

```
(clojure.string/join ", " (map texto-resumo-em-yuan transacoes))  
;; "19/11/2016 => ¥ -70.950, 01/12/2016 => ¥ +5805.000, ...  
;; 03/12/2016 => ¥ -62.350"
```

`clojure.string/join` significa que vamos aplicar a função `join` que se localiza no namespace `clojure.string`. Esta função recebe um argumento que diz qual o delimitador entre os elementos e um array. Mas usar `clojure.string/join ", "` com muita frequência é bem chato. Ainda bem que podemos facilitar um pouco as coisas:

```
;; uma função parcial que juntará elementos em uma string,  
;; separando-os por vírgula  
(def juntar-tudo (partial clojure.string/join ", "))  
  
(juntar-tudo (map texto-resumo-em-yuan transacoes))  
;; "19/11/2016 => ¥ -70.950, 01/12/2016 => ¥ +5805.000, ...  
;; 03/12/2016 => ¥ -62.350"
```

Neste exemplo, `juntar-tudo` é a aplicação parcial de `clojure.string/join`, já definindo o primeiro argumento. Os demais argumentos vêm depois, quando a função parcial `juntar-tudo` é aplicada ao resultado de `(map texto-resumo-em-yuan transacoes)`.

6.3 CONCLUSÃO

A composição de funções é um recurso de alto potencial em Programação Funcional, com baixo custo de implementação, tendo em vista que funções são cidadãos de primeira classe. É muito útil quando temos transformações em série que costumam vir

juntas. A aplicação parcial de funções, apesar ocorrer com menos frequência do que a composição, pode simplificar bastante as partes de um programa. De novo, o custo de implementação é bem baixo pelo mesmo motivo do custo da composição. Com estes recursos em mão, temos mais ferramentas para reúso de código e escrita de programas mais enxutos.

No próximo capítulo, vamos falar sobre pureza e imutabilidade, que são características essenciais de funções que permitem que todas estas coisas legais sejam possíveis nesse mundo!

PUREZA E IMUTABILIDADE

Até aqui, vimos o potencial de trabalharmos com funções como cidadãos de primeira classe. Vimos também como construir funções mais ricas a partir de funções mais simples, através da composição. Ambas as ideias se tornam ainda mais poderosas por conta de duas características bastante importantes: pureza e imutabilidade.

7.1 PUREZA

Pureza é lidar com funções que não têm efeito colateral quando são aplicadas. Funções assim são chamadas de puras. Uma função que não tem efeitos colaterais permite que sua aplicação, desde que com um mesmo argumento, seja sempre previsível. Isto é conhecido como *transparência referencial*. A função `escrita-hacker` a seguir, por exemplo, é pura:

```
(def de-para [{:de "a" :para "4"}
              {:de "e" :para "3"}
              {:de "i" :para "1"}
              {:de "o" :para "0"}])

(defn escrita-hacker [texto dicionario]
  (if (empty? dicionario)
```

```

texto
(let [conversao (first dicionario)]
  (escrita-hacker (clojure.string/replace texto
                                           (:de conversao)
                                           (:para conversao))
                  (rest dicionario))))

(escrita-hacker "alameda" de-para)
;; "4l4m3d4"

```

Sempre que aplicarmos `escrita-hacker` a `"alameda"`, sempre teremos o mesmo resultado: `"4l4m3d4"`. Perceba que `escrita-hacker` é uma função recursiva, já que chama a si própria. Sei que está rolando uma enorme ansiedade, mas já já falaremos de recursão!

Uma função pura não faz operações que mudam o estado de alguma coisa. Exemplos de mudança de estado são:

- Alterar valores dentro de um mapa;
- Alterar uma String;
- Escrever em um arquivo;
- Imprimir algo na tela; e
- Inserir um registro no banco de dados.

Toda mudança de estado é um efeito colateral. E isso não é bem-vindo em nossos programas porque reduz a previsibilidade dos resultados esperados, o que acaba impactando negativamente na robustez dos nossos programas.

E também são consideradas impuras as funções que, para gerar algum resultado, executam processos de ler um arquivo, consultar um banco de dados ou obter um recurso da internet. Mesmo sendo processos de leitura, por saírem de dentro do domínio do programa, não geram resultados previsíveis, pois o arquivo, banco

de dados ou API que você vai ler, pode ter mudado entre uma leitura e outra.

Mas e qual a utilidade de um programa que não grava nada em um banco de dados ou lê informações de uma API? Verdade. Não que em programas em linguagens funcionais nunca haja efeitos colaterais. O que acontece é que eles são confinados. Estas linguagens dificultam efeitos colaterais, mas não os tornam impossíveis de acontecer.

Desfazendo uma impureza

No capítulo anterior vimos o seguinte código:

```
(def cotacoes
  {:yuan {:cotacao 2.15M :simbolo "¥"}
   :euro {:cotacao 0.28M :simbolo "€"}})

(defn transacao-em-outra-moeda [moeda transacao]
  (let [{:cotacao :cotacao simbolo :simbolo} moeda] cotacoes)
  ;; ^^^^^^^^^
  (assoc transacao :valor (* cotacao (:valor transacao))
          :moeda simbolo)))
```

Perceba que a função `transacao-em-outra-moeda` faz uso de `cotacoes`. Parece um código bem inofensivo, mas acontece que a definição de `cotacoes` acontece fora da função que a utiliza, e podemos considerar que isso torna impura a função `transacao-em-outra-moeda`. O correto é que a tabela de conversão seja passada como argumento para a função, assim:

```
(def cotacoes
  {:yuan {:cotacao 2.15M :simbolo "¥"}
   :euro {:cotacao 0.28M :simbolo "€"}})

(defn transacao-em-outra-moeda [cotacoes moeda transacao]
  ;; ^^^^^^^^^ o novo argumento
```

```
(let [{cotacao :cotacao simbolo :simbolo} moeda] cotacoes]
  (assoc transacao :valor (* cotacao (:valor transacao))
    :moeda simbolo)))
```

Mas acabamos quebrando os códigos que aplicavam a função, como `(transacao-em-outra-moeda :euro (first transacoes))`, porque só passam dois argumentos. Para resolver isso, podemos fazer com que `transacao-em-outra-moeda` seja a aplicação parcial de uma outra função que recebe a tabela de conversão padrão como argumento:

```
(defn transacao-convertida [cotacoes moeda transacao]
  ;; aqui no `let` tem uma desestruturação,
  ;; que falamos no capítulo anterior
  (let [{cotacao :cotacao simbolo :simbolo} moeda] cotacoes]
    (assoc transacao :valor (* cotacao (:valor transacao))
      :moeda simbolo)))

(def transacao-em-outra-moeda
  (partial transacao-convertida cotacoes))
```

Uma outra forma de lidarmos com este problema é fazer uso de um recurso de Clojure chamado *aridade múltipla*, que é quando temos uma função que se comporta de formas diferentes de acordo com a quantidade de argumentos que recebe. Assim, podemos fazer com que a função `transacao-em-outra-moeda`, quando chamada com 2 argumentos, chame a si mesma com a tabela de conversão padrão. Assim:

```
(defn transacao-em-outra-moeda
  ([cotacoes moeda transacao]
    (let [{cotacao :cotacao simbolo :simbolo} moeda] cotacoes]
      (assoc transacao :valor (* cotacao (:valor transacao))
        :moeda simbolo)))
  ([moeda transacao]
    (transacao-em-outra-moeda cotacoes moeda transacao)))
```

Ainda temos a tabela padrão sendo utilizada, mas ela não está

dentro do corpo da função, e agora temos uma versão pura de `transacao-em-outra-moeda` .

7.2 IMUTABILIDADE

Mudanças de estado são bem raras em Clojure. Isso porque a linguagem força o uso de estruturas de dados que não são modificáveis. Um mapa em Clojure não é editável. Uma String não é concatenável. Isso faz com que sejam imutáveis. Só para deixar claro: imutável é algo que não muda, de jeito nenhum, nem com reza braba! Nada muda em um dado depois que ele é criado. Estas estruturas de dados imutáveis são conhecidas como estruturas de dados permanentes.

Quando vemos um novo par de chave e valor sendo incluído em um mapa, o que acontece é que o mapa existente é reaproveitado como base para o novo mapa, com os novos valores. Ou quando incluímos um novo elemento em uma lista: na realidade, não incluímos um elemento, criamos uma nova lista que inclui os elementos anteriores e o novo elemento.

Mas quais são as vantagens da imutabilidade? No livro *The Joy of Clojure*, Michael Fogus e Chris Houser levantam algumas vantagens:

- Dada a invariância dos dados, nós só precisamos nos preocupar com a validação das informações no momento em que eles são criados. Quando há mutabilidade, é preciso aplicar as validações requeridas a cada alteração;
- Quando há a possibilidade de mutação, dois dados que são iguais agora podem não ser iguais no futuro. E aí, em um

cenário onde há processamento concorrente, é muito difícil acompanhar se houve mudança em um dado e qual processo realizou a mudança;

- Compartilhar dados que não mudam é fácil, ao ponto de que podemos aproveitar dados antigos em estruturas que representam novos dados.

Esta última característica é bem interessante de ver com exemplos. Vejamos a lista de países membros do Mercosul:

```
(def membros-fundadores
  (list "Argentina" "Brasil" "Paraguai" "Uruguai"))

membros-fundadores
;; ("Argentina" "Brasil" "Paraguai" "Uruguai")
```

A Venezuela passou a fazer parte do grupo em julho de 2012:

```
(def membros-pletos (cons "Venezuela" membros-fundadores))

membros-pletos
;; ("Venezuela" "Argentina" "Brasil" "Paraguai" "Uruguai")
```

`cons` é uma função que permite que adicionemos um elemento a uma coleção. Neste caso, "Venezuela" é incluída na primeira posição da lista. Daí, vemos que `membros-pletos` é uma nova lista, com 5 elementos. Mas a diferença é só o primeiro elemento. Será que, se tirarmos "Venezuela" da lista `membros-pletos`, o resultado é igual a `membros-fundadores`?

```
(rest membros-pletos)
;; ("Argentina" "Brasil" "Paraguai" "Uruguai")
```

`rest` é uma função que pega uma coleção, ignora o primeiro elemento, e retorna uma nova coleção contendo o resto. Neste caso, tirou "Venezuela" e retornou o resto.

Agora nos resta comparar as coleções:

```
(identical? (rest membros-pletos)
            membros-fundadores)
;; true
```

`identical?` é uma função que compara 2 elementos para saber se são os mesmos objetos. E nos ajuda a compreender que, aqui, a estrutura de dados inicial foi aproveitada, graças à imutabilidade. Como Clojure sabe que o dado não vai mudar, ela se aproveita disso para a construção de novos dados. E este reaproveitamento dos dados permite menor consumo de memória.

Pureza e imutabilidade na Programação Orientada a Objetos

Nem pureza, nem imutabilidade são exclusividades de linguagens de Programação Funcionais. Na verdade, são princípios que você pode levar para seus programas em linguagens orientadas a objeto. POO possui como uma das suas características prover ferramentas para a modelagem de conceitos e o estado de objetos, e o gerenciamento deste estado normalmente acontece de uma forma imperativa. Quer dizer, sempre descrevemos um passo a passo de como mudar o estado das coisas. Em POO, não precisamos tratar estado sempre de forma imperativa, e assim podemos muito bem trazer pureza e imutabilidade para linguagens como Java, C++, Python, Ruby.

Átomos e mutações

Quando convertemos uma transação para uma outra moeda, por exemplo, yuan, criamos um novo mapa, em vez de modificar o atual. Por exemplo:

```
(defn transacao-em-yuan [transacao]
  (let [{cotacao :cotacao simbolo :simbolo} :yuan} cotacoes]
    (assoc transacao :valor (* cotacao (:valor transacao))
              :moeda simbolo)))

(transacao-em-yuan (first transacoes))
;; {:valor 70.950M, :tipo "despesa", :comentario "Almoço",
;;  :moeda "¥", :data "19/11/2016"}
```

`assoc` gera um novo mapa com os novos valores, e a transação original permanece intocada.

Outra consequência da imutabilidade é que precisamos refazer referências a valores que temos. Por exemplo, vejamos a lista de transações que temos:

```
(def transacoes
  [{:valor 33M :tipo "despesa" :comentario "Almoço"
    :moeda "R$" :data "19/11/2016"}
   {:valor 2700M :tipo "receita" :comentario "Bico"
    :moeda "R$" :data "01/12/2016"}
   {:valor 29M :tipo "despesa" :comentario "Livro de Clojure"
    :moeda "R$" :data "03/12/2016"}])
```

`transacoes` é uma referência a uma lista de mapas. Se precisarmos colocar mais um elemento nesta lista, vamos precisar criar uma nova referência, com o novo valor e os anteriores:

```
(def transacoes
  [{:valor 33M :tipo "despesa" :comentario "Almoço"
    :moeda "R$" :data "19/11/2016"}
   {:valor 2700M :tipo "receita" :comentario "Bico"
    :moeda "R$" :data "01/12/2016"}
   {:valor 29M :tipo "despesa" :comentario "Livro de Clojure"
    :moeda "R$" :data "03/12/2016"}
   {:valor 45M :tipo "despesa" :comentario "Jogo no Steam"
    :moeda "R$" :data "26/12/2016"}])
```

Ou ainda:

```
(def transacoes
```



```

[{:valor 33M :tipo "despesa" :comentario "Almoço"
 :moeda "R$" :data "19/11/2016"}
 {:valor 2700M :tipo "receita" :comentario "Bico"
 :moeda "R$" :data "01/12/2016"}
 {:valor 29M :tipo "despesa" :comentario "Livro de Clojure"
 :moeda "R$" :data "03/12/2016"}]]

(def transacoes (cons {:valor 45M :tipo "despesa"
                       :comentario "Jogo no Steam" :moeda "R$"
                       :data "26/12/2016"}
                      transacoes))

```

O valor da coleção dentro de `transacoes` não muda, mas a referência `transacoes` agora se refere uma outra coleção.

Mas algumas vezes podemos precisar manter a referência e mudar o valor do seu conteúdo. Um contador, por exemplo, é um caso. *Caching*, outro. Clojure cuida para que as mutações ocorram de forma atômica. E podemos usar o mesmo recurso para manter uma base de dados de transações na memória, e faremos uso de `atom`, que é uma das formas que a linguagem provê para o gerenciamento de estado. Vejamos `atom` em uso:

```

(def registros (atom ()))
;; #'user/registros

```

Para mais detalhes sobre `atom`, os livros *Clojure Applied* e *Joy of Clojure* fazem uma discussão mais profunda sobre o assunto, e o livro *Programming Clojure* fala sobre os detalhes de como Clojure trata sobre a atomicidade de alterações de dados.

Acabamos de criar `registros`, que referencia um átomo que contém uma lista vazia. No REPL, se consultarmos apenas

registros , você verá algo diferente do que temos visto até agora com outras referências:

```
registros
;; #object[closure.lang.Atom 0x6422b10f {:status :ready, :val ()}
]
;; Não necessariamente 0x6422b10f, mas algo assim
```

Isto acontece porque o que temos é, de fato, o que compõe um átomo. Se quisermos obter o valor atual do estado do átomo, precisamos fazer uso de um novo elemento, o símbolo `@` :

```
@registros
;; ()
```

E assim obtemos a lista vazia que `registros` contém.

Para incluir novos elementos nesta lista, utilizamos a função `swap!` . Ela já chama a atenção pela exclamação no nome! Isso nos sugere que há uma mudança de estado por acontecer. `swap!` precisa de duas informações para fazer a mutação: o átomo alvo e uma função, junto com seus argumentos, que será aplicada aos valores que o átomo contém. Vejamos como inserir uns elementos neste átomo:

```
(swap! registros conj {:valor 29M :tipo "despesa"
                       :comentario "Livro de Clojure" :moeda "R$"
                       :data "03/12/2016"})
;; ({:valor 29M, :tipo "despesa", :comentario "Livro de Clojure",
;;   :moeda "R$", :data "03/12/2016"})
```

`swap!` recebe o átomo `registros` e a função `conj` , logo em seguida, com uma transação como argumento. O que `swap!` faz aqui é aplicar `conj` com seu argumento, a `registros` , que possui uma lista, até então, vazia. É o resultado do seguinte:

```
(conj {:valor 29M :tipo "despesa" :comentario "Livro de Clojure"
      :moeda "R$" :data "03/12/2016"} ())
```

```
;; percebe a lista vazia no final, ()
;; a transação passada para conj é incluída nesta lista
```

Daí utilizamos a mesma lógica para incluir os demais valores:

```
(swap! registros conj
  {:valor 2700M :tipo "receita" :comentario "Bico"
   :moeda "R$" :data "01/12/2016"})
;; ({:valor 2700M, :tipo "receita", :comentario "Bico",
;;   :moeda "R$", :data "01/12/2016"})
;; ({:valor 29M, :tipo "despesa", :comentario "Livro de Clojure",
;;   :moeda "R$", :data "03/12/2016"})
```

Para evitar duplicação, vamos criar uma função para inserção de transações neste banco de dados:

```
;; abstraindo inclusão de dados no átomo
(defn registrar [transacao]
  (swap! registros conj transacao))

;; fazendo uso desta abstração
(registrar {:valor 33M :tipo "despesa" :comentario "Almoço"
            :moeda "R$" :data "19/11/2016"})
;; ({:valor 33M, :tipo "despesa", :comentario "Almoço",
;;   :moeda "R$", :data "19/11/2016"})
;; ({:valor 2700M, :tipo "receita", :comentario "Bico",
;;   :moeda "R$", :data "01/12/2016"})
;; ({:valor 29M, :tipo "despesa", :comentario "Livro de Clojure",
;;   :moeda "R$", :data "03/12/2016"})

;; fazendo uso desta abstração de novo
(registrar {:valor 45M :tipo "despesa" :comentario "Jogo no Steam"
            :moeda "R$" :data "26/12/2016"})
;; ({:valor 45M, :tipo "despesa", :comentario "Jogo no Steam",
;;   :moeda "R$", :data "26/12/2016"})
;; ({:valor 33M, :tipo "despesa", :comentario "Almoço",
;;   :moeda "R$", :data "19/11/2016"})
;; ({:valor 2700M, :tipo "receita", :comentario "Bico",
;;   :moeda "R$", :data "01/12/2016"})
;; ({:valor 29M, :tipo "despesa", :comentario "Livro de Clojure",
;;   :moeda "R$", :data "03/12/2016"})

;; agora vamos abstrair a leitura de dados do átomo:
```

```
(def transacoes @registros)
transacoes
;; ({:valor 45M, :tipo "despesa", :comentario "Jogo no Steam",
;;   :moeda "R$", :data "26/12/2016"}
;;  {:valor 33M, :tipo "despesa", :comentario "Almoço",
;;   :moeda "R$", :data "19/11/2016"}
;;  {:valor 2700M, :tipo "receita", :comentario "Bico",
;;   :moeda "R$", :data "01/12/2016"}
;;  {:valor 29M, :tipo "despesa", :comentario "Livro de Clojure",
;;   :moeda "R$", :data "03/12/2016"})
```

Primeiro, criamos uma função para abstrair a inclusão de dados no átomo, `registrar`. Daí nós a utilizamos duas vezes para incluir duas transações, uma de cada vez, e elas são incluídas na cabeça da lista. Por fim, associamos o átomo a um nome, `transacoes`, sobrescrevendo ao que `transacoes` se referia antes de associarmos o átomo a este nome. Daí podemos reutilizá-lo nos exemplos seguintes.

7.3 IMUTABILIDADE NO NOSSO DOMÍNIO

Este cenário, onde há um estímulo muito forte para que não mudemos estado algum, faz com que repensemos como escrever nossos programas. Um exemplo de grande impacto é na manipulação de coleções, na extração de informações derivadas do processamento de seus elementos. Por exemplo, para ver o saldo da nossa conta. O que precisa ser feito é somar as receitas e subtrair as despesas. Em um processo imperativo tradicional, em Java antes da versão 8, seria algo assim:

```
public Double saldoDeFormaImperativa(List<Transacao> transacoes)
{
    Double saldo = 0.0;

    for (Transacao t : transacoes) {
        if (t. ehDespesa()) {
```

```

        saldo = saldo - t.getValor();
    } else {
        saldo = saldo + t.getValor();
    }
}

return saldo;
}

```

Aqui o valor de `saldo` é modificado a cada passagem pelos itens da coleção `transacoes`. Esta é a forma imperativa de realizar o processamento em coleções. Trabalhar com imutabilidade exige que façamos tal processamento de uma forma diferente, mas já muito conhecida, por meio da recursão.

7.4 RECURSÃO

O exemplo anterior pode até ser feito com bastante similaridade em Clojure, fazendo uso de átomos. Mas estamos aqui para aprender as coisas do jeito funcional, certo? E aqui a onda é aplicar recursão para um problema destes. Uma solução recursiva deve:

1. iniciar o saldo com zero;
2. encerrar o processo quando não existirem mais elementos sobre os quais iterar, e retornar o saldo;
3. para cada iteração, aumentar ou diminuir o saldo com o valor da transação; e
4. reiniciar o processo, desta vez com o saldo sendo o novo valor calculado na etapa anterior e as transações *sem* a transação utilizada na etapa anterior.

Até aqui usamos `(first transacoes)` com bastante frequência para pegar o primeiro elemento de uma coleção, e

vimos a função `rest` no exemplo sobre imutabilidade. Ambas serão bastante úteis em uma solução para este processo de calcular o saldo recursivamente:

```
(defn despesa? [transacao]
  (= (:tipo transacao) "despesa"))
;; vamos recuperar a função que checa se uma transação é despesa

(defn saldo-acumulado [acumulado transacoes]
  ;; if-let
  (if-let [transacao (first transacoes)]
    ;; se _transacao_ existir, continue calculando o saldo
    (saldo-acumulado (if (despesa? transacao)
                        (- acumulado (:valor transacao))
                        (+ acumulado (:valor transacao)))
                     (rest transacoes))
    ;; se não existir, a coleção de transacoes acabou e é hora de
    ;; retornar o resultado
    acumulado))
```

`if-let` é uma macro que funciona como `let`, só que o bloco dentro dela só é executado se o elemento entre colchetes existir (na verdade, se for qualquer coisa diferente de `nil` e `false`). Se houver o elemento, ele executa o primeiro bloco que é passado. No exemplo, é aplicar `saldo-acumulado` novamente. Quando acaba a lista de transações, que é quando `(first transacoes)` retornar `nil`, `if-let` vai executar o segundo bloco, que é retornar `acumulado`.

Ao aplicar `saldo-acumulado`, acabamos repetindo o mesmo processo, mas aqui é que entra o detalhe da recursão. A função é até a mesma, mas os argumentos são diferentes. Primeiro, decrementa-se ou incrementa-se o valor do saldo e repete-se o processo com uma nova lista de transações que não inclui a transação utilizada nesta iteração. Vamos rodar o exemplo:

```
(saldo-acumulado 0 transacoes)
```

```
;; 2593M

;; e qual seria o resultado com uma lista sem transações?
(saldo-acumulado 0 ())
;; 0

;; ou só os dois primeiros elementos?
(saldo-acumulado 0 (take 2 transacoes))
;; -78M
```

Podemos extrair o cálculo para uma função para simplificar um pouco a leitura de `saldo-acumulado` :

```
(defn calcular [acumulado transacao]
  (let [valor (:valor transacao)]
    (if (despesa? transacao)
      (- acumulado valor)
      (+ acumulado valor))))

(defn saldo-acumulado [acumulado transacoes]
  (if-let [transacao (first transacoes)]
    (saldo-acumulado (calcular acumulado transacao)
                     (rest transacoes))
    acumulado))
```

Vamos ver com detalhes o que acontece em cada etapa? Para isso utilizamos `prn` , que é uma função que recebe Strings e as imprime na tela. Assim, conseguimos ver um passo a passo do que acontece:

```
(defn saldo-acumulado [acumulado transacoes]
  (if-let [transacao (first transacoes)]
    (prn "Começou saldo-acumulado. Saldo até agora:" acumulado)
    (saldo-acumulado (calcular acumulado transacao)
                     (rest transacoes))
    acumulado))
;; CompilerException java.lang.IllegalArgumentException: if-let..
```

Ops! Deu erro. Isso quer dizer que `if-let` espera 1 ou 2 blocos, e acabamos passando 3: `(prn... , (saldo-`

acumulado... e acumulado . Precisamos escrever prn e saldo-acumulado de modo que elas sejam consideradas aplicadas dentro de um bloco só. Para isso, podemos contar com a função do . Assim:

```
(defn saldo-acumulado [acumulado transacoes]
  (if-let [transacao (first transacoes)]
    (do
      (prn "Começou saldo-acumulado. Saldo até agora:" acumulado)
      (saldo-acumulado (calcular acumulado transacao)
                       (rest transacoes)))
    acumulado))

(saldo-acumulado 0 transacoes)
;; "Começou saldo-acumulado. Saldo até agora:" 0
;; "Começou saldo-acumulado. Saldo até agora:" -45M
;; "Começou saldo-acumulado. Saldo até agora:" -78M
;; "Começou saldo-acumulado. Saldo até agora:" 2622M
;; 2593M
```

Também precisaremos usar do no segundo bloco do if-let :

```
(defn saldo-acumulado [acumulado transacoes]
  (if-let [transacao (first transacoes)]
    (do
      (prn "Começou saldo-acumulado. Saldo até agora:" acumulado)
      (saldo-acumulado (calcular acumulado transacao)
                       (rest transacoes)))
    (do
      (prn "Processo encerrado. Saldo final:" acumulado)
      acumulado)))

(saldo-acumulado 0 transacoes)
;; "Começou saldo-acumulado. Saldo até agora:" 0
;; "Começou saldo-acumulado. Saldo até agora:" -45M
;; "Começou saldo-acumulado. Saldo até agora:" -78M
;; "Começou saldo-acumulado. Saldo até agora:" 2622M
;; "Processo encerrado. Saldo final:" 2593M
;; 2593M
```

Agora que conseguimos colocar os prn , vamos ver uns outros

para exibir mais detalhes:

```
;; lembrando valor-sinalizado
(defn valor-sinalizado [transacao]
  (let [moeda (:moeda transacao "R$")
        valor (:valor transacao)]
    (if (despesa? transacao)
      (str moeda " -" valor)
      (str moeda " +" valor))))

(defn saldo-acumulado [acumulado transacoes]
  (prn "Começou saldo-acumulado. Saldo até agora:" acumulado)
  (if-let [transacao (first transacoes)]
    (do
      (prn "Valor da transação atual:"
            (valor-sinalizado transacao))
      (prn "Quantidade de transações restantes:"
            (count (rest transacoes)))
      (prn)
      (saldo-acumulado (calcular acumulado transacao)
                        (rest transacoes)))
    (do
      (prn "Processo encerrado. Saldo final:" acumulado)
      acumulado)))

(saldo-acumulado 0 transacoes)
;; "Começou saldo-acumulado. Saldo até agora:" 0
;; "Valor da transação atual:" "R$ -45"
;; "Quantidade de transações restantes:" 3
;;
;; "Começou saldo-acumulado. Saldo até agora:" -45M
;; "Valor da transação atual:" "R$ -33"
;; "Quantidade de transações restantes:" 2
;;
;; "Começou saldo-acumulado. Saldo até agora:" -78M
;; "Valor da transação atual:" "R$ +2700"
;; "Quantidade de transações restantes:" 1
;;
;; "Começou saldo-acumulado. Saldo até agora:" 2622M
;; "Valor da transação atual:" "R$ -29"
;; "Quantidade de transações restantes:" 0
;;
;; "Começou saldo-acumulado. Saldo até agora:" 2593M
;; "Processo encerrado. Saldo final:" 2593M
```

```
;; 2593M
```

Legal. Já dá para ter uma clareza do que acontece. Vamos remover as chamadas a `prn` e a `do` :

```
(defn saldo-acumulado [acumulado transacoes]
  (if-let [transacao (first transacoes)]
    (saldo-acumulado (calcular acumulado transacao)
                     (rest transacoes))
    acumulado))
```

É normal criarmos uma função que abstrai o início da iteração, para que não precisemos chamar `saldo-acumulado` com `0` :

```
(defn saldo [transacoes]
  (saldo-acumulado 0 transacoes))

(saldo transacoes)
;; 2593M
```

E agora podemos utilizar um outro recurso da linguagem: a sobrecarga de aridade. Algumas linguagens de programação, como Java e Ruby, permitem que funções sejam definidas com o mesmo nome, desde que com diferenças nos argumentos que recebe. Clojure, por sua vez, apenas permite que um nome seja utilizado por uma só função. Você pode até criar várias funções com um mesmo nome e seu programa vai rodar, mas sempre que uma nova definição de função surge, a referência para a definição anterior é perdida.

Vale lembrar que aridade corresponde ao número de argumentos que uma função recebe. Nos exemplos até aqui, `saldo-acumulado` tem aridade igual a dois, já que recebe dois argumentos: `acumulado` e `transacoes` . Com a sobrecarga de aridade, podemos ter uma só função `saldo` , que recebe uma lista de transações, ou um saldo e uma lista de transações:

```

(defn saldo

  ;; caso a função receba só um argumento
  ([transacoes]
   (saldo 0 transacoes))

  ;; caso a função receba dois argumentos
  ([acumulado transacoes]
   (if-let [transacao (first transacoes)]
     (saldo (calcular acumulado transacao) (rest transacoes))
     acumulado)))

(saldo transacoes)
;; 2593M

(saldo 0 transacoes)
;; 2593M

```

Legal, né? O importante aqui é ver como resolvemos problemas de forma recursiva, essencial no mundo da Programação Funcional. Mas temos uma revisão por fazer aqui. Esta função não funciona para uma lista muito grande de transações:

```

(defn como-transacao [valor]
  {:valor valor})

(def poucas-transacoes
  (map como-transacao (range 10)))

(def muitas-transacoes
  (map como-transacao (range 1000)))

(def incontaveis-transacoes
  (map como-transacao (range 100000)))

(saldo poucas-transacoes)
;; 45

(saldo muitas-transacoes)
;; 499500

```

```
(saldo incontaveis-transacoes)
;; StackOverflowError...
```

Isso acontece porque acabamos empilhando mais dados na pilha de execução do que o possível, dado o limite de memória disponibilizado para nosso programa. E, como ainda não utilizamos nenhum recurso de otimização que Clojure oferece, encontramos um erro de estouro de pilha.

Tail Call Optimization e Tail Recursion Elimination

Quando uma função é invocada, um conjunto de dados é alocado na memória do seu computador. Estes dados são empilhados, a cada chamada de função, no que é conhecido como pilha de execução. Nesta pilha, para cada função, são guardadas as variáveis locais à função, os parâmetros que lhe foram passados e o endereço de memória para o qual o programa deve retornar quando a função acabar. Quando a função encerra seu processamento, seus dados são removidos da pilha de execução. No exemplo anterior, ao aplicar `saldo`, a máquina virtual coloca na pilha os argumentos (uma coleção de transações e, talvez, o valor acumulado) e um endereço de retorno (abstraido para a gente).

O erro que acabamos de ver acontece quando aninhamos várias aplicações de funções sem que elas cheguem a seu fim. E assim acabamos empilhando muita coisa, a ponto de causar o estouro da pilha de execução. Existe um limite de memória que podemos utilizar e, de tanto empilhar coisas, estouramos o limite. Este limite é definido pela máquina virtual Java, que é configurável. Normalmente os programas são abortados quando um estouro de pilha acontece. Perceba que `saldo` aplica `saldo`,

e que cada vez que a aplicação acontece, novos dados são colocados na pilha. Com duas transações, as informações para a execução de `saldo` são empilhadas duas vezes, mas as informações da primeira execução de `saldo` precisa esperar a segunda execução terminar. Com duzentas transações, há o empilhamento duzentas vezes; quando a ducentésima aplicação de `saldo` acabar, os resultados vão sendo retornados e os dados desempilhados. Mas a partir de um alto número de transações, a pilha estoura. Como nada vai sendo desempilhado, só empilhado, uma hora atingimos o limite.

Tail Call Optimization (TCO) é uma técnica que busca otimizar a alocação de dados na pilha de execução quando a última instrução em uma função é a aplicação de uma outra função. Por estar na última posição de instruções de uma função, diz-se estar na cauda, traseira ou *rabeira*, termo utilizado lá na minha terra, que são traduções de *tail*. A última instrução é chamada de *tail call*. E a ideia da TCO é evitar empilhar dados de forma desnecessária. Em vez de colocar novos dados na pilha de execução, a estrutura já criada é reaproveitada.

Tail Recursion Elimination (TRE) é uma especialização de TCO. Ela é aplicada quando a última instrução é a aplicação da mesma função, ou seja, quando há recursão na última instrução de uma função. Da forma como `saldo` está, não há *tail recursion* porque a última instrução não é a aplicação de `saldo`. Vamos reescrever `saldo` para que haja *tail recursion*:

```
;; nova versão de saldo, com tail recursion
(defn saldo
  ([transacoes] (saldo 0 transacoes))
  ([acumulado transacoes]
   (if (empty? transacoes)
```

```

    acumulado
    (saldo (calcular acumulado (first transacoes))
      (rest transacoes))))

(saldo poucas-transacoes)
;; 45

(saldo muitas-transacoes)
;; 499500

(saldo incontaveis-transacoes)
;; StackOverflowError...

```

Como pode ser visto, o resultado continua sendo o mesmo: estouro de pilha. Só que agora podemos fazer uso de uma forma especial de Clojure que faz a *tail recursion elimination*: `recur` :

```

;; nova versão de saldo, desta vez com recur
(defn saldo
  ([transacoes] (saldo 0 transacoes))
  ([acumulado transacoes]
   (if (empty? transacoes)
       acumulado
       ;; vvvvvv aqui utilizamos recur em vez de saldo
       (recur (calcular acumulado (first transacoes))
              (rest transacoes)))))

(saldo muitas-transacoes)
;; 499500

(saldo incontaveis-transacoes)
;; 4999950000

```

`recur` é usada para indicar que um loop recursivo vai começar e o compilador pode realizar suas otimizações. Perceba que a última chamada em `saldo` é `recur`, e não uma nova chamada a `saldo`, como nos exemplos anteriores. `recur` primeiro avalia os argumentos (`(calcular acumulado (first transacoes))` e `(rest transacoes)`). Em paralelo, refaz as referências que são argumentos no loop (`acumulado` e

transacoes), com os novos valores avaliados na etapa anterior. E então o código volta a ser executado a partir do ponto logo após a entrada da função, antes do `if`. A aridade de `recur` tem que ser igual ao ponto de recursão (no nosso caso, dois). `recur` tem que estar na posição derradeira, ou então dá erro.

Tanto TCO quanto TRE são muito mais comuns no mundo da Programação Funcional do que no Imperativo ou da Orientação a Objetos. Isso se dá porque a maioria das soluções que vemos no mundo imperativo envolve loops com estruturas tipo `for` e recursões são pouco frequentes. No mundo funcional, recursão é algo muito comum. Algumas linguagens proveem formas diferentes de lidar com TRE. Scala, por exemplo, oferece uma anotação (`@tailrec`) quando você espera que a função seja otimizada. Como a máquina virtual Java não oferece TCO, linguagens implementadas para ela devem tratar TCO da sua forma. Lua, por outro lado, oferece TCO por padrão.

Uma pegada diferente para o mesmo problema

Lembra da definição de `reduce` no capítulo 5, na parte sobre funções de grandeza superior? Algumas recursões podem ser substituídas por `reduce`. Este cálculo de saldo também pode:

```
(reduce calcular 0 transacoes)
;; 2593M
```

E esse `0` ? `reduce` recebe uma função, uma coleção e, opcionalmente, um valor inicial entre ambas. O `0` é o primeiro valor, assim como passamos `0` para `saldo-acumulado`. E aí `calcular` é aplicada a cada elemento de `transacoes`. O resultado é definido como o acumulador e `reduce` parte para

repetir o mesmo procedimento com os demais itens de transações. As duas soluções são diferentes, apesar de gerarem o mesmo resultado. A ideia aqui é mostrar para você quais ferramentas estão disponíveis para a resolução de problemas.

7.5 CONCLUSÃO

Imutabilidade e pureza são essenciais para a escrita de programas funcionais. Junto com estes princípios, vem uma nova forma de pensar em soluções. E então recursão passa a ser mais comum no nosso dia a dia. No próximo capítulo, vamos falar de um recurso bem diferente, mas muito útil. Refiro-me à *preguiça*. Não aquela que lhe faz ver seriados em vez de seguir para o próximo capítulo deste livro, mas sim a capacidade de só fazer alguma coisa quando for realmente a hora de fazê-la.

Pensando bem, será que não são a mesma coisa?

PREGUIÇA

Para concluir esta parte do livro, um último tópico bem comum na Programação Funcional: preguiça, do inglês *laziness*. Preguiça não é exclusividade do mundo funcional, sendo algo comum em diversas linguagens de programação. Acontece que se torna ainda mais disponível e valiosa nas funcionais. Haskell, por exemplo, é uma linguagem onde preguiça é o padrão. Qualquer trecho de código só é executado quando realmente necessário. Clojure, por sua vez, não é completamente preguiçosa. Está um pouco mais para Java, sendo pouco preguiçosa.

Para entrar aos poucos no tema, vejamos preguiça em Java. Considere o seguinte trecho de código:

```
if (sedeDaCopaDoMundo != null &&  
    sedeDaCopaDoMundo.primeiraVezQueSedia()) {  
    // ...  
}
```

Se o valor de `sedeDaCopaDoMundo` for nulo, o trecho que chama o método `primeiraVezQueSedia()` não é executado. Isso é o que é chamado de avaliação preguiçosa, do inglês *lazy evaluation*. Quer dizer que uma parte do código só é executada quando for mesmo necessária. No exemplo anterior, a segunda parte do `if` só é executada (ou avaliada) quando `sedeDaCopaDoMundo` for diferente de nulo.

Por outro lado, ainda em Java, há casos em que é claro que um trecho de código não precisaria ser avaliado de imediato, mas é:

```
public class TesteDaPreguica {

    public static void main(String[] args) {
        String semUso = mensagem(); // uma variável sem uso
        String tambemSemUso = mensagem(); // outra sem uso

        // nada além disso e o programa se encerra
    }

    private static String mensagem() {
        System.out.println("Gravando, 1, 2, 3.");

        return "mensagem";
    }
}

// Gravando, 1, 2, 3.
// Gravando, 1, 2, 3.
```

As variáveis `semUso` e `tambemSemUso` são declaradas e inicializadas, chamando o método `mensagem()` para verificar qual o valor a ser associado a elas. Mas o programa se encerra logo que as associações são feitas e nada mais acontece. Mesmo não sendo necessário, Java vai lá e executa `mensagem()`, e por isso vemos `Gravando, 1, 2, 3` duas vezes. Isto é o que é chamado de *avaliação ansiosa*, ou estrita. Java é, então, parcialmente preguiçosa, já que há preguiça em poucos casos, não na linguagem inteira.

Agora vejamos como Clojure lida com a preguiça:

```
;; imprime uma mensagem
;; mas se o processamento for preguiçoso, nada deveria aparecer
;; e retorna uma string
(defn teste-da-preguica []
  (prn "Não deveria aparecer nada aqui"))
```

```

"nada")

;; recebe dois argumentos
;; mas não faz nada com eles
;; retornando uma string
(defn um-oi [a b]
  "oi")

(um-oi (teste-da-preguica) (teste-da-preguica))
;; "Não deveria aparecer nada aqui"
;; "Não deveria aparecer nada aqui"
;; "oi"

```

Como `um-oi` não faz nada com os argumentos, o trecho `(teste-da-preguica)` não precisa ser avaliado. No entanto, como vemos na saída do console, "Não deveria aparecer nada aqui" acaba aparecendo duas vezes. Aqui fizemos o uso de `prn` para ver o texto no terminal. Ela é uma função que imprime seus argumentos para a saída de dados, que, no REPL, é o terminal. Funciona da mesma forma que `pr`, a única diferença é que `prn` pula uma linha depois de imprimir seus argumentos, enquanto `pr` não faz isso.

No entanto, mesmo não sendo completamente preguiçosa, Clojure traz a capacidade de trabalharmos com sequências de forma que sua avaliação possa ser tardia. Aliás, já vimos diversos casos em que nossas listas foram tratadas assim. Várias operações em sequências que utilizamos até aqui são assim, como `map`, `filter` e `reduce`.

LISTAS, SEQUÊNCIAS E COLEÇÕES EM CLOJURE

Para evitar confusão, é importante explicar o que é uma lista, o que é uma sequência e o que é uma coleção, já que não significam a mesma coisa. Uma lista é uma estrutura de dados que guarda diversos valores, podendo ser repetidos e com tipos distintos. Uma coleção agrega vários valores (listas, mapas, conjuntos). Uma sequência é um tipo de coleção na qual que seus dados são acessados de forma linear. Tanto uma coleção, quanto uma sequência, são abstrações. Listas, mapas e conjuntos podem ser tratados como sequências, sendo implementações das abstrações.

Vejamos um exemplo de preguiça sendo aplicado em um dos códigos que já vimos, no uso de `filter`, por partes:

```
;; uma lista de transações
(def transacoes
  [{:valor 33.0 :tipo "despesa" :comentario "Almoço"
    :moeda "R$" :data "19/11/2016"}
   {:valor 2700.0 :tipo "receita" :comentario "Bico"
    :moeda "R$" :data "01/12/2016"}
   {:valor 29.0 :tipo "despesa" :comentario "Livro de Clojure"
    :moeda "R$" :data "03/12/2016"}
   {:valor 45M :tipo "despesa" :comentario "Jogo no Steam"
    :moeda "R$" :data "26/12/2016"}])

;; uma função que nos diz se uma transação é uma despesa
(defn despesa? [transacao]
  (= (:tipo transacao) "despesa"))

;; vamos pegar uma sequência que só tenha despesas
(filter despesa? transacoes)
;; ({:valor 33.0, :tipo "despesa", :comentario "Almoço",
;;   :moeda "R$", :data "19/11/2016"})
```

```
;; {:valor 29.0, :tipo "despesa", :comentario "Livro de Clojure"}

;; :moeda "R$", :data "03/12/2016"}
;; {:valor 45M, :tipo "despesa", :comentario "Jogo no Steam",
;; :moeda "R$", :data "26/12/2016"}}
```

Não parece haver nada diferente, certo? Se `filter` avalia listas tardiamente, o que deveríamos ter visto? Só para provar que estou falando a verdade, olhe qual é a classe que `filter` retorna:

```
(class (filter despesa? transacoes))
;; clojure.lang.LazySeq
```

Então, é mesmo uma sequência preguiçosa. Mas onde está a diferença? Olhe ela aqui:

```
;; vamos associar a sequência de despesas a um nome
(def despesas (filter despesa? transacoes))
;; #'user/despesas

;; `filter` só é aplicada quando `despesas` é avaliada
despesas
;; ({:valor 33.0, :tipo "despesa", :comentario "Almoço",
;; :moeda "R$", :data "19/11/2016"}
;; {:valor 29.0, :tipo "despesa", :comentario "Livro de Clojure"
;; :moeda "R$", :data "03/12/2016"}
;; {:valor 45M, :tipo "despesa", :comentario "Jogo no Steam",
;; :moeda "R$", :data "26/12/2016"}})
```

Repare que, quando declaramos `despesas`, (`filter despesa? transacoes`) não é avaliada. Acontece apenas quando queremos ver o valor de `despesas`. E, ainda assim, só acontece porque estamos no REPL e, ao avaliar `despesas`, o REPL imprime o resultado na tela, o que faz com que a sequência seja consumida.

Em um outro exemplo, com `map`, o mesmo acontece:

```
;; vamos pegar o valor e a moeda de uma transação
```

```
;; e informar que estamos fazendo isso
(defn valor-sinalizado [transacao]
  (prn "Pegando o valor e a moeda da transação:" transacao)
  (let [moeda (:moeda transacao "R$")
        valor (:valor transacao)]
    (if (= (:tipo transacao) "despesa")
      (str moeda " -" valor)
      (str moeda " +" valor)))))

(def transacao-aleatoria {:valor 9.0})

(valor-sinalizado transacao-aleatoria)
;; "Pegando o valor e a moeda da transação:" {:valor 9.0}
;; "R$ +9.0"

(def valores (map valor-sinalizado transacoes))
;; #'user/valores

valores
;; "Pegando o valor e a moeda da transação:" {:valor 33.0, ...
;; "Pegando o valor e a moeda da transação:" {:valor 2700.0, ...
;; "Pegando o valor e a moeda da transação:" {:valor 29.0, ...
;; "Pegando o valor e a moeda da transação:" {:valor 45M, ...
;; ("R$ -33.0" "R$ +2700.0" "R$ -29.0" "R$ -45")
```

Aqui aplicamos `prn` para ver quando, de fato, a função for aplicada. Daí, com `(valor-sinalizado transacao-aleatoria)`, a função é aplicada a só uma transação, e a mensagem de texto é exibida. Sendo assim, não há sequências sendo geradas ou processadas. Já com `map`, não vemos nada sendo impresso. As mensagens de texto só aparecem quando queremos imprimir o valor de `valores`. Isto acontece porque `map` produz sequências preguiçosas, que faz com que seus elementos só sejam processados quando a sequência for, de fato, consumida.

8.1 SEQUÊNCIAS PREGUIÇOSAS E LISTAS INFINITAS DE TRANSAÇÕES

Uma sequência preguiçosa permite que seus valores sejam computados quando necessário, criando um cenário onde a sequência existe, mas os dados em si ainda não existem em memória. Uma sequência preguiçosa não é necessariamente uma estrutura de dados, mas sim um objeto que entende como sequências funcionam, sendo capaz de computar um próximo elemento aplicando alguma função. E, sendo assim, este tipo de sequência permite que tenhamos sequências infinitas. Sim, infinitas. Isso quer dizer que não sabemos quando, nem se, a sequência terá um fim. E seus dados são consumidos até que não precisemos mais deles.

Um exemplo de uso de uma sequência infinita é a leitura de arquivos de texto com a função `line-seq`. `line-seq` permite que processemos uma linha de cada vez, sem que o arquivo inteiro seja todo ele carregado em memória. Ela é uma função que retorna uma sequência preguiçosa de Strings.

No nosso domínio, podemos usar uma sequência infinita para gerar quantas transações forem necessárias para casos de teste, por exemplo. Ou fazer a paginação de resultados como um *feed* como o Twitter ou o Facebook fazem: parece que os dados são infinitos.

Digamos que, para propósito de testes de relatórios, queremos várias transações. Muitas, umas 50. Nem o valor nem o tipo importam. Primeiro, para gerar valores aleatórios, vejamos o que podemos fazer:

```
;; rand-nth retorna um valor aleatório dentro de uma coleção
(rand-nth ["despesa" "receita"])
;; "despesa"
;; seu resultado pode ser diferente

;; rand-int retorna um número inteiro aleatório entre 0 e
```

```
;; o argumento, sem incluir o argumento nas possibilidades
;; daí multiplicamos por 0.01M para ter um número real com
;; duas casas decimais
(* (rand-int 100001) 0.01M)
;; 243.85M
;; seu resultado pode ser diferente
```

Agora que conseguimos gerar valores aleatórios e deixar o acaso escolher se a transação será uma receita ou uma despesa, vejamos como gerar uma transação aleatória:

```
(defn transacao-aleatoria []
  {:valor (* (rand-int 100001) 0.01M)
   :tipo (rand-nth ["despesa" "receita"])}))

(transacao-aleatoria)
;; {:valor 156.47M, :tipo "receita"}
;; seu resultado pode ser diferente
```

Legal. Agora podemos fazer uso da ideia de estruturas de dados infinitas, sendo aqui uma sequência. Em uma sequência infinita, não sabemos seu tamanho, nem temos certeza de onde ela acaba, afinal, ela é (surpresa!) infinita. Mas não significa que ocuparemos uma quantidade absurda de memória nos nossos computadores. Vamos pegar algumas transações aleatórias para começar a brincadeira:

```
;; repeatedly produz uma sequência preguiçosa, cujos elementos são
;; chamadas à função que lhe é passada como argumento
(repeatedly 3 transacao-aleatoria)
;; ... uma lista com 3 transações aleatórias

(class (repeatedly 3 transacao-aleatoria))
;; clojure.lang.LazySeq

;; quando não dizemos quantos elementos queremos, repeatedly cria
;; uma sequência infinita
(def transacoes-aleatorias (repeatedly transacao-aleatoria))
;; #'user/transacoes-aleatorias
```



```

;; uma transação aleatória
(take 1 transacoes-aleatorias)
;; ({:valor 8.53M, :tipo "despesa"})
;; seu resultado pode ser diferente

;; duas transações aleatórias
;; perceba que a primeira transação é exatamente igual à que foi
;; retornada no caso anterior
(take 2 transacoes-aleatorias)
;; ({:valor 8.53M, :tipo "despesa"})
;; ({:valor 154.85M, :tipo "despesa"})
;; seu resultado pode ser diferente

;; e aqui as 2 primeiras também são iguais às anteriores
(take 5 transacoes-aleatorias)
;; ({:valor 8.53M, :tipo "despesa"})
;; ({:valor 154.85M, :tipo "despesa"})
;; ({:valor 171.96M, :tipo "receita"})
;; ({:valor 460.30M, :tipo "despesa"})
;; ({:valor 942.23M, :tipo "receita"})
;; seu resultado pode ser diferente

```

`repeatedly` é uma função que constrói uma sequência preguiçosa, às vezes infinita. Se você não disser quantas repetições haverá, ela será infinita. Os elementos da sequência são chamadas a função que é passada como argumento. Para que `repeatedly` funcione, a função que lhe é passada, neste caso `transacao-aleatoria`, não deve receber argumentos. `repeatedly`, esta sim, pode receber um argumento, opcional, que é a quantidade de elementos. No exemplo anterior, vimos primeiro `repeatedly` gerando uma sequência de 3 elementos. Logo em seguida, vimos que, sem uma quantidade definida, temos uma sequência infinita, cujos elementos são gerados quando utilizamos `take`.

Vejam os que acontece quando pegamos os elementos de `transacoes-aleatorias` com `take`: primeiro pegamos um só elemento da sequência (`take 1`), e é aí que ele será computado,

chamando `transacao-aleatoria` uma vez. Quando aplicamos `take` com `2`, já temos o primeiro elemento, e o segundo será computado, com mais uma chamada à função `transacao-aleatoria`. Não significa que o valor de `transacoes-aleatorias` está mudando a cada chamada. O valor permanece o mesmo, que é uma sequência, mas seus elementos são "revelados" à medida que são necessários. É mais ou menos assim que fica:

```
(take 1 transacoes-aleatorias)
;; [elemento-1, ...]
;; onde ... quer dizer que não sabemos o que tem mais
;; pode ser mais um elemento, pode não ser mais nada, nil

(take 2 transacoes-aleatorias)
;; [elemento-1, elemento-2, ...]

(take 5 transacoes-aleatorias)
;; [elemento-1, elemento-2, ..., elemento-5, ...]

(= (take 5 transacoes-aleatorias) (take 5 transacoes-aleatorias))
;; true
```

Como `transacoes-aleatorias` mantém em memória os registros que já foram gerados, `(= (take 5 transacoes-aleatorias) (take 5 transacoes-aleatorias))` é verdade.

Se quisermos ver como é a sequência toda, não vai dar, pois ela é infinita e nunca vai terminar de ser processada:

```
;; vamos ver o valor da sequência toda
transacoes-aleatorias
;; ops... o processamento aqui não acaba nunca
;; aperte ctrl + c para parar
```

O processo não se encerra porque não há fim nesta lista, e o REPL tenta pegar todos os elementos para imprimi-los na tela. Mas aí ela nunca chega ao fim. E o REPL nunca para, tadinho.

`repeatedly` normalmente é utilizado quando queremos indicar que a função a ser repetida tem efeito colateral. Além de `repeatedly`, há outras funções projetadas para a geração de sequências preguiçosas: `repeat`, `iterate` e `cycle`. E, além destas funções, existe uma outra forma de criarmos listas preguiçosas: fazendo uso da macro `lazy-seq`.

8.2 A MACRO LAZY-SEQ

Lembra do capítulo anterior, sobre recursão, onde repetíamos uma operação sobre uma lista? Criar uma longa lista de elementos também pode ser feito de forma recursiva. Vamos começar relembando o uso de `cons`:

```
;; lembrando o uso de cons para colocar um elemento como cabeça
;; de uma sequência
(cons (transacao-aleatoria) transacoes)
;; ({:valor 35.39M, :tipo "receita"}
;;  {:valor 33M, :tipo "despesa", :comentario "Almoço",
;;   :moeda "R$", :data "19/11/2016"}
;;  {:valor 2700M, :tipo "receita", :comentario "Bico",
;;   :moeda "R$", :data "01/12/2016"}
;;  {:valor 29M, :tipo "despesa", :comentario "Livro de Clojure",
;;   :moeda "R$", :data "03/12/2016"}
;;  {:valor 45M, :tipo "despesa", :comentario "Jogo no Steam",
;;   :moeda "R$", :data "26/12/2016"})
;; seu resultado pode ser diferente
```

`cons` pega o primeiro argumento e o coloca como a cabeça de uma sequência, e os demais elementos são os que estão na sequência passada como segundo argumento. E o motivo de relembrar o uso de `cons` é porque vamos usá-la para criar a cabeça de uma sequência preguiçosa mais para a frente.

O trecho de código a seguir demonstra como, com recursão, conseguimos criar uma sequência com `cons`:

```

(defn aleatorias
  ;; começando a sequência criando a primeira transação
  ([quantidade]
   (aleatorias quantidade 1 (list (transacao-aleatoria))))
  ;; incrementando a sequência até que a quantidade desejada
  ;; seja atendida
  ([quantidade quantas-ja-foram transacoes]
   (if (< quantas-ja-foram quantidade)
       (aleatorias quantidade (inc quantas-ja-foram)
                           (cons (transacao-aleatoria)
                                transacoes))
       transacoes)))

;; poucas transações
(aleatorias 4)
;; { :valor 429.01M, :tipo "despesa" }
;; { :valor 480.21M, :tipo "despesa" }
;; { :valor 57.75M, :tipo "receita" }
;; { :valor 869.30M, :tipo "despesa" }
;; seu resultado pode ser diferente

;; muitas transações
(aleatorias 900000)
;; StackOverflowError...

```

Com uma quantidade pequena, `aleatorias` funciona tranquilamente. Na penúltima linha da função, nós criamos uma nova transação, a qual é colocada na cabeça da sequência usando `cons`. O resto da sequência, o segundo argumento para `cons` é composto pelas transações que foram geradas até agora. Mas uma quantidade muito grande (900.000 é o suficiente no meu computador) causa estouro de pilha. É o caso de otimizarmos com `recur`:

```

(defn aleatorias
  ([quantidade]
   (aleatorias quantidade 1 (list (transacao-aleatoria))))
  ([quantidade quantas-ja-foram transacoes]
   (if (= quantas-ja-foram quantidade)
       transacoes
       ;; aplicando a otimização na cauda

```

```

(recur quantidade (inc quantas-ja-foram)
  (cons (transacao-aleatoria) transacoes))))

;; poucas transações
(aleatorias 4)
;; ({:valor 955.90M, :tipo "receita"}
;;  {:valor 589.52M, :tipo "despesa"}
;;  {:valor 808.02M, :tipo "despesa"}
;;  {:valor 582.80M, :tipo "despesa"})
;; seu resultado pode ser diferente

;; muitas transações
(aleatorias 900000)
;; ...uma sequência com 900000 transações...

(class (aleatorias 4))
;; clojure.lang.Cons

(class (aleatorias 900000))
;; clojure.lang.Cons

```

Funcionou. Mas ainda não criamos uma sequência preguiçosa. Perceba que `(class (aleatorias 4))` demora menos que `(class (aleatorias 900000))`. Vejamos como medir:

```

(time (class (aleatorias 4)))
;; "Elapsed time: 0.054824 msecs"
;; clojure.lang.Cons

(time (class (aleatorias 900000)))
;; "Elapsed time: 917.45872 msecs"
;; clojure.lang.Cons

```

Como não é uma sequência preguiçosa, todos os elementos são criados antes que saibamos o tipo do retorno de `aleatorias`. Vamos torná-la preguiçosa:

```

(defn aleatorias
  ([quantidade]
   (aleatorias quantidade 1 (list (transacao-aleatoria))))
  ([quantidade quantas-ja-foram transacoes]

```

```

;; vamos embalar a parte recursiva na macro lazy-seq
(lazy-seq
  (if (= quantas-ja-foram quantidade)
      transacoes
      (aleatorias quantidade (inc quantas-ja-foram)
                            (cons (transacao-aleatoria)
                                transacoes))))))

(time (class (aleatorias 4)))
;; "Elapsed time: 0.03271 msecs"
;; clojure.lang.LazySeq

(time (class (aleatorias 900000)))
;; "Elapsed time: 0.029649 msecs"
;; clojure.lang.LazySeq

```

O retorno de `class` já nos diz que agora temos uma sequência preguiçosa! E `time` mostra como é insignificante a diferença de tempo entre pegar a classe de uma sequência preguiçosa grande e de uma pequena. A novidade na definição anterior é a presença da macro `lazy-seq`, que pega o que tem dentro dela e embala numa função, que se responsabilizará por criar uma sequência preguiçosa para nós. Neste caso, `(if ...)`.

Mas a sequência ainda não é infinita, já que nós definimos uma quantidade de elementos quando aplicamos a função. E que tal criarmos uma infinita? Assim:

```

(defn aleatorias []
  (lazy-seq
    (cons (transacao-aleatoria) (aleatorias))))

(time (class (take 4 (aleatorias))))
;; "Elapsed time: 0.170615 msecs"
;; clojure.lang.LazySeq

(time (class (take 900000 (aleatorias))))
;; "Elapsed time: 0.046173 msecs"
;; clojure.lang.LazySeq

```

```

(take 4 (aleatorias))
;; ({:valor 230.55M, :tipo "despesa"}
;;  {:valor 224.57M, :tipo "receita"}
;;  {:valor 63.63M, :tipo "despesa"}
;;  {:valor 283.42M, :tipo "despesa"})
;; seu resultado pode ser diferente

(take 900000 (aleatorias))
;; ...Muitas transações...

```

Aqui tem uma versão muito mais reduzida que nos trechos de código anteriores. Perceba que há uma recursão no segundo argumento para `cons`, na última linha da função. No entanto, mesmo pedindo muitos elementos (900000), não há estouro de pilha, graças ao trabalho que a macro `lazy-seq` faz para nós. Ainda, como não definimos o tamanho da sequência na sua construção (não passamos nenhuma quantidade), temos uma sequência infinita!

Uma sequência preguiçosa não precisa estipular com clareza um fim para ela. Funções como `first` e `take`, que consomem (minha tradução para *realize*) elementos da lista, não sabem quais serão os próximos elementos. Enquanto a sequência prover elementos, `take` os vai consumindo e `rest` nos conta qual é o restante da sequência, mesmo sem seus elementos ainda em memória.

8.3 CONCLUSÃO

Com a apresentação da preguiça, encerramos esta parte sobre conceitos de Programação Funcional. Até aqui vimos a importância que funções têm neste mundo, derivações desta importância e algumas particularidades muito presentes em linguagens de programação funcionais, como imutabilidade e

preguiça.

Na parte seguinte, construiremos dois projetos: uma ferramenta, bem simples, de linha de comando para conversão de moedas; e um serviço, um tanto mais complexo, usando o domínio de finanças pessoais que temos utilizado até aqui. A ideia é que este último projeto nos permita manter e manipular transações, capaz de receber e responder requisições HTTP. Microserviços, alguém?

Programação Funcional na prática

CRIANDO UMA APLICAÇÃO COM LEININGEN

Até aqui, vimos diversos trechos de código que mostram como Clojure funciona e como a Programação Funcional é abordada pela linguagem. Mas todos os exemplos foram executados no REPL. Uma aplicação que vai parar em um ambiente de produção precisa de um pouco mais de atenção: estrutura de arquivos, versionamento, editor de texto caprichado etc. E este apreço ao desenvolvimento em Clojure é abordado nesta parte do livro.

Vamos criar duas aplicações: uma primeira, mais simples, para demonstrar o uso do Leiningen, ferramenta para construção de código e gerenciamento de dependências, além do REPL; e uma segunda, mais completa, onde aplicaremos o domínio adotado na parte 2 e que atenderá requisições HTTP. Usaremos o Leiningen por ser a ferramenta mais bem estabelecida, mas há o Boot, uma boa e moderna alternativa, mas que não abordaremos neste livro.

Ao criarmos novos projetos com o Leiningen, ele permite que usemos *templates*, os quais nos ajudam a criar esqueletos de projetos de facetas variadas. O template padrão é para bibliotecas,

que devem ser utilizadas por outros projetos. Alguns templates úteis são:

- `app` , que nos permite criar uma aplicação de linha de comando, usado neste capítulo;
- `compojure` , muito utilizada para a criação de APIs HTTP, usado no próximo capítulo; e
- `luminus` , para a criação de aplicações Web.

Neste capítulo, vamos criar uma aplicação de linha de comando que faça a conversão entre moedas, dada a cotação do momento, inspirado no serviço que o site <https://free.currencyconverterapi.com/> oferece.

O código-fonte deste capítulo pode ser encontrado aqui:
<https://gitlab.com/programacaofuncional/conversor>

No terminal, digite o seguinte comando:

```
lein new app conversor
```

E o retorno para você deve ser algo assim:

```
Generating a project called conversor based on the  
'app' template.
```

Aqui usamos `lein new` para criar um novo projeto, `app` como template e `conversor` como nome do projeto.

Você verá que há uma nova pasta, chamada `conversor` com alguns arquivos e diretórios. Os mais notáveis são:

- `project.clj` , que contém metadados do projeto, definição de dependências e algumas configurações;
- `src/conversor/core.clj` , que contém um esqueleto da aplicação; e
- `test/conversor/core_test.clj` , que contém um esqueleto de testes para o código da aplicação.

A aplicação já está pronta para ser executada, e para isso basta o seguinte comando dentro da pasta recém-criada:

```
lein run
```

e o resultado deve ser:

```
Hello, World!
```

O que está acontecendo por aqui?

Apesar de não parecer, há bastante coisa acontecendo quando rodamos o comando `lein run` . Primeiro, o Leiningen vai procurar qual o método de entrada da aplicação, algo como o tradicional `main` do Java. Este método de entrada é definido no arquivo `project.clj` , na linha que diz `:main ^:skip-aot conversor.core` . Lembra-se do que falei sobre namespaces? `conversor.core` é o namespace que o Leiningen criou para nós. O que essa linha nos diz é que o método de entrada da aplicação está dentro deste namespace.

Vamos abrir o arquivo `core.clj` para ver o que tem lá dentro e entender o que este método de entrada faz? Logo na primeira linha você vai ver algo novo:

```
(ns conversor.core  
  (:gen-class))
```

A primeira linha nos diz que, neste arquivo, deste ponto em diante, estamos lidando com o namespace `conversor.core`. Todas as funções declaradas em seguida pertencerão a este namespace. A parte `(:gen-class)` tem a ver com a possibilidade de que o Java use código gerado com Clojure. Como estamos criando uma aplicação de linha de comando, em que a JVM vai chamar a função `-main` do nosso programa, o `:gen-class` vai gerar as classes Java para a JVM utilizar.

Existe também a possibilidade de código Clojure utilizar código criado com Java, o que é chamado de interoperabilidade com Java. Aqui neste capítulo tem um exemplo de interoperabilidade com a classe `String` do Java, no qual utilizamos alguns métodos desta classe.

Logo depois da definição do namespace temos a definição de uma função chamada `main`. Está como aquele ponto de entrada que foi definido no arquivo `project.clj`:

```
(defn -main
  "I don't do a whole lot ... yet."
  [& args]
  (println "Hello, World!"))
```

Mas esta função parece um pouco diferente das que vimos nos capítulos anteriores, né? Primeiro porque há um hífen antes do nome da função. O que acontece é que este hífen faz com que esta função seja estática. Como todo método `main` em Java é `static`, este também precisa ser. E funções que começam com `-`, quando chamadas de um programa em Java, são consideradas métodos estáticos.

Logo depois há uma `String`, que na verdade é um recurso opcional. Quando existe uma `String` logo depois do nome de uma

função, ela é considerada documentação, e por isso não faz diferença no comportamento de um programa. E na parte dos argumentos há um `&` antes de `args`, o que quer dizer que há uma quantidade indefinida de argumentos. Por fim, temos a chamada a uma função que imprime um texto no terminal: `(println "Hello, world!")`.

Vamos mudar um pouco este programa para gerar a mensagem que quisermos? Vamos transformar esta função em algo ligeiramente diferente. Substitua a função `main` por esta:

```
(defn -main
  "Nosso próprio main"
  [& args]
  (println "Clojureando por aqui!"))
```

Salve o arquivo e rode novamente a aplicação com o comando:

```
lein run
```

Perceba que o texto, como esperado, mudou. Agora chegou a hora de sangrar os dedos no teclado e criar nosso primeiro programa!

9.1 O CONVERSOR DE MOEDAS

A ideia é pegarmos uma moeda e ver qual a cotação dela com relação a uma outra. A API <https://free.currencyconverterapi.com> usa uma moeda como base para as conversões e podemos refletir isso no nosso programa. Sendo assim, como estamos lidando com uma aplicação de linha de comando, vamos precisar lidar com dois argumentos: uma moeda de referência e a moeda que nos interessar.

Como vimos há pouco, temos acesso aos argumentos passados para o programa graças a `[& args]` . Vejamos como trabalhar com eles. Primeiro, vamos imprimi-los na tela modificando `core.clj` :

```
(defn -main
  [& args]
  (println "Temos" (count args) "argumento(s)")
  (println "Os argumentos são: " args))
```

Vamos ver como fica rodando no terminal:

```
lein run
# Temos 0 argumento(s)
# Os argumentos são: nil
```

```
lein run USD BRL
# Temos 2 argumento(s)
# Os argumentos são: (USD BRL)
```

```
lein run --de=USD --para=BRL
# Temos 2 argumento(s)
# Os argumentos são: (--de=USD --para=BRL)
```

9.2 INTEROPERABILIDADE COM JAVA

Sabemos quais são os argumentos, mas ainda não os interpretamos. Precisamos pegar os valores de cada um, que é o que vem depois de `=` . contei um pouco sobre interoperabilidade mais cedo, e chegou a hora de vê-la na prática, experimentando um pouco com o REPL, uma técnica comum de quem desenvolve em Clojure. Vamos abrir um outro terminal e entrar no REPL:

```
lein repl
```

E então vamos ver como pegamos uma substring. Em diversas linguagens, para `--de=USD` seria algo como `"--`

de=USD".substring(5) . Em Java é assim. E como Clojure roda em cima da JVM e pode fazer uso de bibliotecas e recursos nativos do Java, por que não usarmos aqui? O que precisamos é aplicar a *função* `substring` em `--de=USD` (a ênfase em *função* é porque, em Java, `substring` é chamado de método). Vamos ver uns exemplos de interoperabilidade primeiro, e depois entender como isto pode nos ajudar. No REPL, digite o seguinte:

```
;; o valor de pi
(java.lang.Math/PI)
;; 3.141592653589793

;; uma forma de criar uma string
(new String "Vamos converter moedas")
;; "Vamos converter moedas"

;; que pode ser feita assim também
(String. "Vamos converter moedas")
;; "Vamos converter moedas"
```

Até aqui, nada muito impressionante. Mas as coisas começam a mudar quando queremos utilizar métodos dentro dos objetos criados. Clojure provê uma forma de lidar com o assunto de modo que sempre se pense nas funções primeiro. Existe algo chamado *dot notation* que vai traduzir nossa intenção em Clojure para chamadas a recursos do Java. Por exemplo:

```
;; toUpperCase tem um . antes
;; que é a macro que vai converter o código para nós
(.toUpperCase (String. "Vamos converter moedas"))
;; "VAMOS CONVERTER MOEDAS"

;; mesmo com toLowerCase
(.toLowerCase "BRL")
;; "brl"
```

Nos exemplos anteriores, tratamos de usar os métodos `toUpperCase` e `toLowerCase` que estão disponíveis nas strings

em Java. `(.toLowerCase "BRL")` em Clojure é equivalente a `"BRL".toLowerCase()` em Java. Aplicando a interoperabilidade no domínio do nosso programa, temos os seguintes exemplos:

```
(.substring "--de=USD" 5)
;; "USD"

(.substring "--para=BRL" 7)
;; "BRL"
```

Aqui, `(.substring "--de=USD" 5)` equivale a `--de=USD.substring(5)` em Java.

Agora que sabemos como manipular as strings, vamos criar uma função no nosso programa para interpretar os argumentos. Vamos sair do REPL e voltar ao editor de texto. Lembra de `cond`, que usamos no capítulo 2? Vamos utilizá-la mais uma vez, desta vez verificando se as strings se iniciam com `--de=` e `--para=`, e pegando seus respectivos valores. E é aqui que entram `.startsWith` e `.substring`. Edite mais uma vez o arquivo `core.clj` para que ele fique assim:

```
(ns conversor.core
  (:gen-class))

;; funções declaradas como defn- são visíveis apenas
;; para o namespace onde elas foram definidas
(defn- valores-em [argumento]
  (cond
    ;; .startsWith e .substring são as novidades aqui!
    (.startsWith argumento "--de=")
    {:de (.substring argumento 5)}
    (.startsWith argumento "--para=")
    {:para (.substring argumento 7)}
    :else {}))

(defn -main
  [& args]
  ;; vamos usar a função map nos argumentos
```

```
;; e interpretá-los
(println "os argumentos são:" (map valores-em args)))
```

Agora nosso programa sabe interpretar os argumentos. Na última linha, há o uso de `map` para aplicar `valores-em` para cada elemento de `args`. A função `valores-em` verifica se o argumento corresponde à moeda usada como base de conversão da API que vamos utilizar ou se é a moeda para a qual nos interessa converter. Ainda, se não for nenhum dos dois casos, a função vai retornar um mapa sem chave nem valor. Para tal, `valores-em` usa os métodos de `string` que vimos há pouco. E o resultado será uma lista cujos elementos são mapas com a chave e o valor de cada argumento:

```
lein run --de=USD --para=BRL
# os argumentos são: ({:de USD} {:para BRL})
```

`.substring` foi usado bastante nesses exemplos como demonstração da interoperabilidade com Java. Nativamente, Clojure oferece a função `subs` que faz a mesma coisa: `(subs "Era uma vez..." 4 7)` retorna `"uma"`.

9.3 DEPENDÊNCIAS

Só que interpretar argumentos é um problema comum e bem resolvido pela comunidade de desenvolvimento. Em Clojure não é diferente. Com uma comunidade em crescimento e amadurecendo, existem soluções robustas que podemos utilizar em nossos programas. Assim não precisamos resolver o que já está resolvido, permitindo que foquemos em resolver problemas de

negócio. No programa que estamos desenvolvendo utilizaremos bibliotecas para a interpretação de argumentos, para realizar requisições HTTP e para manipular JSON. São elas:

- Para interpretar argumentos, `tools.cli`
(<https://github.com/clojure/tools.cli>)
- Para requisições HTTP, `clj-http`
(<https://github.com/dakrone/clj-http>)
- Para manipular JSON, `Cheshire`
(<https://github.com/dakrone/cheshire>)

Para incluir estas dependências precisaremos editar o arquivo `project.clj`, começando com a biblioteca de interpretação de argumentos. Neste arquivo há uma seção que diz `:dependencies`, que é onde vamos fazer a referência para biblioteca. Vamos mudar o `project.clj` para que ele fique da seguinte forma:

```
(defproject conversor "0.1.0-SNAPSHOT"
  :description "FIXME: write description"
  :url "http://example.com/FIXME"
  :license {:name "Eclipse Public License"
            :url "http://www.eclipse.org/legal/epl-v10.html"}
  ;; as dependências vão aqui
  :dependencies [[org.clojure/clojure "1.10.0"]
                 [org.clojure/tools.cli "0.4.1"]]
  ;; fim das dependências
  :main ^:skip-aot conversor.core
  :target-path "target/%s"
  :profiles {:uberjar {:aot :all}})
```

O valor de `:dependencies` é um vetor, e cada elemento deste vetor é um outro vetor de tamanho igual a dois. Neste exemplo, `[org.clojure/tools.cli "0.4.1"]` é um vetor que possui dois valores: o nome da dependência e uma string que diz qual é a versão que utilizaremos. E o valor de `:dependencies` é um vetor

que, agora, possui dois elementos: `clojure` e `tools.cli`.

Agora podemos interpretar as moedas que nosso programa receberá. Em `core.clj`, vamos alterar `main` para que ela importe a função de `tools.cli` que vai nos ajudar com os argumentos:

```
(ns conversor.core
  (:require [clojure.tools.cli :refer [parse-opts]])
  (:gen-class))

;; ...
```

O trecho `(:require [clojure.tools.cli :refer [parse-opts]])` demonstra a forma como importamos funções de outros namespaces. Neste caso, fazemos uso do namespace `clojure.tools.cli`. E, em vez de fazer referência a todas as funções que ele pode conter, vamos fazer a referência apenas à função `parse-opts`; por isso, o `:refer`.

`parse-opts` é, então, a função que vai nos ajudar a interpretar os argumentos. `tools.cli` chama os argumentos como `--de=` e `--para=` de opções e pede que configuremos o que cada opção dessa significa. Então teremos um vetor de opções:

```
(def opcoes-do-programa
  [["-d" "--de moeda base" "moeda base para conversão"
   :default "eur"]
   ["-p" "--para moeda destino"
    "moeda a qual queremos saber o valor"]])
```

`opcoes-do-programa` é a definição das opções que nosso programa vai aceitar. Perceba que ele é um vetor e seus elementos também são vetores, sendo um vetor para cada opção. `-d` ou `--de` para a moeda base, `-p` ou `--para` para a moeda que nos interessa. `tools.cli` só precisa disso para interpretar os

argumentos para nós. E `parse-opts` retornará um mapa com tudo o que interpretou, inclusive com uma chave `:options`, que é o que nos interessa. Sendo assim, vejamos uma nova versão de `core.clj` que imprime as opções interpretadas por `tools.cli`:

```
(ns conversor.core
  (:require [clojure.tools.cli :refer [parse-opts]])
  (:gen-class))

(def opcoes-do-programa
  [["-d" "--de moeda base" "moeda base para conversão"
   :default "eur"]
   ["-p" "--para moeda destino"
   "moeda a qual queremos saber o valor"]])

(defn -main
  [& args]
  (prn "as opções são:" (:options
                        (parse-opts args opcoes-do-programa))))
```

`parse-opts` é uma função que recebe dois argumentos: `args`, os argumentos passados para o nosso programa; e `opcoes-do-programa`, que nós criamos. Daí separamos só o que tem em `:options`. Vejamos o resultado quando rodamos esta nova versão do nosso programa no terminal:

```
lein run --de=USD --para=BRL
# "as opções são:" {:de "USD", :para "BRL"}
```

Agora temos o que precisamos para consultar a taxa de câmbio entre duas moedas!

9.4 CONSULTANDO O SERVIÇO

Para consultar o serviço, precisaremos fazer requisições `GET`. Novamente, em vez de tentar criar uma solução própria para tal, poderemos focar no nosso problema e usar uma biblioteca pronta

para isso. Vamos editar o `project.clj` para incluí-la:

```
(defproject conversor "0.1.0-SNAPSHOT"
  :description "FIXME: write description"
  :url "http://example.com/FIXME"
  :license {:name "Eclipse Public License"
            :url "http://www.eclipse.org/legal/epl-v10.html"}
  :dependencies [[org.clojure/clojure "1.10.0"]
                 [clj-http "3.9.1"]
                 [org.clojure/tools.cli "0.4.1"]]
  :main ^:skip-aot conversor.core
  :target-path "target/%s"
  :profiles {:uberjar {:aot :all}})
```

Antes de editarmos nosso `core.clj`, vejamos como podemos fazer chamadas HTTP para o serviço. No REPL, digite o seguinte:

```
(require '[clj-http.client :as http-client])
;; nil
```

Este `nil` significa que o procedimento foi bem-sucedido, já que a operação não tem nada para nos retornar. É importante que o REPL seja aberto na pasta raiz do projeto para que ele reconheça as dependências (na mesma pasta onde se encontra o `project.clj`). Daí podemos experimentar um `GET` com esta biblioteca. Vamos fazer uma requisição simples para uma API pública:

```
(http-client/get "https://pokeapi.co/api/v2/pokemon/pikachu/")
;; um JSON com várias informações sobre o Pokémon Pikachu
```

Agora que conseguimos fazer requisições, vamos começar a ver as cotações.

OBTENDO UMA CHAVE GRATUITA PARA USO DA API

O serviço que utilizaremos, ainda que gratuito, exige que criemos uma chave. Para tal, acesse <https://free.currencyconverterapi.com/free-api-key/> e crie a sua. Você receberá um email com a chave e precisará clicar no link para validação.

Construindo requisições mais complexas

Para ver as cotações, precisaremos incluir na requisição a moeda de referência, a desejada e a chave da API, pois sem ela a requisição é rejeitada:

```
(def chave "83827ccc17475c9931b6")
;; aqui vai a chave que você recebeu por email
;; essa daqui não é válida. é um exemplo de como se parece

(def api-url
  "https://free.currencyconverterapi.com/api/v6/convert")
;; o endereço da API

(http-client/get api-url
  {:query-params {"q" "USD_BRL"
                  "apiKey" chave}})
;; JSON com a conversão de dólar norte-americano para
;; real brasileiro

(http-client/get api-url
  {:query-params {"q" "EUR_BRL"
                  "apiKey" chave}})
;; mais um JSON com a cotação do Euro
```

No código anterior utilizamos a mesma função `http-client/get` que vimos da primeira vez, mas com uma diferença:

passamos como argumento um mapa que contém dois pares chave-valor. A função concatenará o conteúdo deste mapa com a URL informada em uma string assim:

```
https://free.currencyconverterapi.com/api/v6/convert?q=USD_BRL&compact=ultra&apiKey=83827ccc17475c9931b6
```

Daí aplicamos a função duas vezes, uma convertendo de dólar para real e de euro para real.

Agora que sabemos manipular a biblioteca, fazendo requisições GET , vamos fazer referência a ela no nosso programa. Em `core.clj` , podemos fazer a referência a todas as funções que o cliente HTTP pode oferecer:

```
(ns conversor.core
  (:require [clojure.tools.cli :refer [parse-opts]]
            [clj-http.client :as http-client])
  (:gen-class))

;; ...
```

E utilizamos as funções: `http-client/NOME_DA_FUNÇÃO` , que é como fizemos no REPL. Perceba que é diferente de como referenciamos `parse-opts` . Para fazer uso da função `get` , em `http-client` , fizemos referência ao namespace da função: `http-client/get` . No caso de `parse-opts` , colocamos uma referência a ela dentro do nosso namespace, e por isso podemos chamá-la sem `tools.cli/` como prefixo.

Agora que conseguimos tanto interpretar os argumentos quanto fazer uma chamada ao serviço, vejamos como pode ficar nosso programa:

```
(ns conversor.core
  (:require [clojure.tools.cli :refer [parse-opts]]
            [clj-http.client :as http-client])
```



```
(:gen-class))

(def opcoes-do-programa
  ["-d" "--de moeda base" "moeda base para conversão"
   :default "eur"]
  ["-p" "--para moeda destino"
   "moeda a qual queremos saber o valor"]])

(def chave "83827ccc17475c9931b6")

(def api-url
  "https://free.currencyconverterapi.com/api/v6/convert")

(defn parametrizar-moedas [de para]
  (str de "_" para))

(defn -main
  [& args]
  (let [{:keys [de para]} (:options
                           (parse-opts args opcoes-do-programa))

        ;; aqui acontece um _destructuring_, pegando os valores
        ;; em `:de` e `:para` oriundos das opções da linha de comando
        (prn "Cotação:" (http-client/get api-url
                                           {:query-params { "q" (parametrizar-moedas de para)
                                                            "apiKey" chave}})))]))
```

No terminal:

```
lein run --de=USD --para=BRL
# "Cotação:" ... \"results\":{\"USD_BRL\": ...
```

```
lein run --de=USD --para=GBP
# "Cotação:" ... \"results\":{\"USD_GBP\": ...
```

```
lein run --de=CNY --para=BRL
# "Cotação:" ... \"results\":{\"CNY_BRL\": ...
```

Legal! Agora conseguimos pegar as opções passadas ao nosso programa e chamar o serviço com elas.

USO DE CHAVES NO CÓDIGO-FONTE

Trate chaves de API como senhas de bancos de dados: nunca as mantenha no código. Utilize variáveis de ambiente ou alguma ferramenta de gestão de configuração. Veja como definir variáveis de ambiente no seu sistema operacional, porque daqui para a frente não colocaremos a chave no código-fonte. Utilizaremos a função `System/getenv` para ler o valor de uma variável de ambiente.

9.5 INTERPRETANDO JSON

O resultado, apesar de válido, não está em um formato pronto para exibição e precisamos tratá-lo. Como o serviço retorna um JSON, que é um formato comum, é fácil interpretá-lo. Com a biblioteca Cheshire fica mais fácil. O nosso `project.clj` fica assim:

```
(defproject conversor "0.1.0-SNAPSHOT"
  :description "FIXME: write description"
  :url "http://example.com/FIXME"
  :license {:name "Eclipse Public License"
            :url "http://www.eclipse.org/legal/epl-v10.html"}
  :dependencies [[org.clojure/clojure "1.10.0"]
                 [clj-http "3.9.1"]
                 [cheshire "5.8.1"]
                 [org.clojure/tools.cli "0.4.1"]]
  :main ^:skip-aot conversor.core
  :target-path "target/%s"
  :profiles {:uberjar {:aot :all}})
```

Vejamos como trabalhar com ele no REPL (de novo, na pasta onde `project.clj` se encontra). Defina a variável de ambiente

CHAVE_API com o valor original da chave (83827ccc17475c9931b6) e abra o REPL (que pode ser assim: CHAVE_API=83827ccc17475c9931b6 lein repl). Nele, vamos usar o seguinte código para fazer algumas requisições:

```
(require '[cheshire.core :refer :all])
(require '[clj-http.client :as http-client])

(def chave (System/getenv "CHAVE_API"))
;; agora pegamos a chave de uma variável de ambiente

(def api-url
  "https://free.currencyconverterapi.com/api/v6/convert")

;; vamos pegar só :body, que é o conteúdo da resposta
(def usd-brl
  (:body (http-client/get api-url
    {:query-params {"q" "USD_BRL"
      "apiKey" chave}})))
;; #'conversor.core/usd-brl

;; convertendo JSON para um mapa
(parse-string usd-brl)
;; {"query" {"count" 1}, "results" {"USD_BRL" {"id" "USD_BRL"...
```

Neste exemplo, começamos separando o resultado da chamada ao serviço e o referenciando como `usd-brl`. Do conteúdo que o serviço nos retorna, o que nos importa é o que está em `:body`, por isso, a definição de `usd-brl` busca o valor para a chave `:body`. Este valor é uma string, e a biblioteca Cheshire entra em ação convertendo-a em um mapa, facilitando a manipulação do resultado. `parse-string` é a função que faz esta conversão. E para obter o resultado da cotação, pegamos o valor da chave `"val"` dentro de `"USD_BRL"`; o `get-in`, que vimos no capítulo 6, pode nos ajudar. No REPL:

```
(> (parse-string usd-brl)
  (get-in ["results" "USD_BRL" "val"]))
```

```
;; 3.716103
```

Com a macro `thread-first` (o símbolo `->` que vimos exemplos nos capítulos 5 e 6), aqui usamos `parse-string` para interpretar o JSON dentro de `usd-brl`. O resultado disto é passado para a aplicação de `get-in`, que vai buscar o valor para a chave `"BRL"` em `"rates"`, por sua vez, em `"results"`. O primeiro argumento que `get-in` espera, um mapa, é passado como o resultado de `parse-string` e, graças a `->`, não precisamos explicitar o argumento.

Como já conseguimos fazer tudo o que precisamos, podemos expor com clareza a informação que desejamos:

```
(ns conversor.core
  (:require [clojure.tools.cli :refer [parse-opts]]
            ;; referenciando parse-string do Cheshire
            [cheshire.core :refer [parse-string]]
            [clj-http.client :as http-client])
  (:gen-class))

(def opcoes-do-programa
  [["-d" "--de moeda base" "moeda base para conversão"
   :default "eur"]
   ["-p" "--para moeda destino"
   "moeda a qual queremos saber o valor"]])

(def chave (System/getenv "CHAVE_API"))

(def api-url
  "https://free.currencyconverterapi.com/api/v6/convert")

(defn parametrizar-moedas [de para]
  (str de "_" para))

(defn obter-cotacao [de para]
  (let [moedas (parametrizar-moedas de para)]
    (-> (:body (http-client/get api-url
                                {:query-params { "q" moedas
                                                "apiKey" chave}})))
```

```

    (parse-string)
    (get-in ["results" moedas "val"])))

(defn formatar [cotacao de para]
  (str "1 " de " equivale a " cotacao " em " para))

(defn -main
  [& args]
  (let [{:keys [de para]} (:options
                           (parse-opts args opcoes-do-programa))

        (-> (obter-cotacao de para)
              (formatar de para)
              (prn)))]
    ))

```

Vejamos como o programa se comporta no terminal:

```

lein run --de=USD --para=BRL
# "1 USD equivale a 3.716103 em BRL"

```

```

lein run --de=BRL --para=CNF
# "1 BRL equivale a 1.822966 em CNF"

```

Boa! Assim como esperamos. Dá para refatorar um pouco e separar funções em arquivos diferentes, com foco na legibilidade e separação de responsabilidades, antes de partirmos para o próximo capítulo. Por enquanto, não temos uma suíte de testes para nos dar segurança na refatoração, mas o próximo capítulo já vai abordar desenvolvimento guiado por testes para nós.

Movendo algumas peças

Todo o nosso programa está em um só arquivo. Tudo bem que ele é pequeno, mas temos várias responsabilidades nele:

- Interpretar opções;
- Invocar API;
- Exibir resultado.

Vamos passar cada uma delas para seu próprio namespace, em arquivos separados. Mesmo que eles fiquem pequenos, teremos uma segregação apropriada, o que vai permitir uma manutenção mais fácil no futuro. Vamos começar separando a interpretação de opções em uma função à parte. Ainda em `core.clj` :

```
;; ... escondendo parte superior, que não mudou ..

(defn interpretar-opcoes [argumentos]
  (:options (parse-opts argumentos opcoes-do-programa)))

(defn -main
  [& args]
  ;; fazendo uso de interpretar-opcoes
  (let [{de :de para :para} (interpretar-opcoes args)]
    (-> (cotar de para)
        (formatar de para)
        (prn))))
```

Agora que temos uma função para cada uma das responsabilidades listadas há pouco, vamos construir um namespace para cada. No diretório `src/conversor` , crie os arquivos `formatador.clj` , `interpretador_de_opcoes.clj` e `cambista.clj` . Vamos mover, então, funções para cada um destes arquivos. `formatador.clj` vai ficar assim:

```
(ns conversor.formatador)

(defn formatar [cotacao de para]
  (str "1 " de " equivale a " cotacao " em " para))
```

O arquivo `cambista.clj` fica assim:

```
(ns conversor.cambista
  (:require [clj-http.client :as http-client]
            [cheshire.core :refer [parse-string]]))

(def chave (System/getenv "CHAVE_API"))

(def api-url
```

```

"https://free.currencyconverterapi.com/api/v6/convert")

(defn parametrizar-moedas [de para]
  (str de "_" para))

(defn obter-cotacao [de para]
  (let [moedas (parametrizar-moedas de para)]
    (-> (:body (http-client/get api-url
                                {:query-params { "q" moedas
                                                "apiKey" chave}})))
        (parse-string)
        (get-in ["results" moedas "val"]))))

```

E o `interpretador_de_opcoes.clj` :

```

(ns conversor.interpretador-de-opcoes
  (:require [clojure.tools.cli :refer [parse-opts]]))

(def opcoes-do-programa
  [["-d" "--de moeda base" "moeda base para conversão"
    :default "eur"]
   ["-p" "--para moeda destino"
    "moeda a qual queremos saber o valor"]])

(defn interpretar-opcoes [argumentos]
  (:options (parse-opts argumentos opcoes-do-programa)))

```

ATENÇÃO!

Perceba que o namespace `interpretador-de-opcoes` é composto por hifens (-) e o nome do arquivo com subtraços (_). Isso ocorre devido a uma particularidade da máquina virtual Java, que não permite nomes de arquivos e diretórios com hifens.

E o nosso `core.clj` fica mais simples, assim:

```
(ns conversor.core
  ;; importando outros namespaces do projeto
  (:require [conversor.formatador :refer [formatar]]
            [conversor.interpretador-de-opcoes :refer
              [interpretar-opcoes]]
            [conversor.cambista :refer [obter-cotacao]]))

(defn -main
  [& args]
  (let [{:keys [de para]} (interpretar-opcoes args)]
    (-> (obter-cotacao de para)
        (formatar de para)
        (prn))))
```

Perceba que cada arquivo possui apenas a referência à biblioteca que utiliza. E é isso de refatoração. Veja que o comportamento se mantém o mesmo no terminal:

```
CHAVE_API=83827ccc17475c9931b6 lein run --de=USD --para=BRL
# "1 USD equivale a 3.1695 BRL"
```

```
CHAVE_API=83827ccc17475c9931b6 lein run --de=EUR --para=BRL
# "1 EUR equivale a 3.6623 BRL"
```

9.6 CONCLUSÃO

Aqui concluímos nosso primeiro programa em Clojure. Começamos com o uso do Leiningen para criar e rodar projetos, passamos pelo uso de bibliotecas no nosso programa e encerramos com a criação de nossos próprios namespaces.

No próximo capítulo vamos criar um novo projeto, com mais funcionalidades, que vai ser o foco dos demais capítulos. Vamos começar a construir o nosso próprio serviço para atender requisições HTTP!

CONTROLE FINANCEIRO PESSOAL VIA HTTP

No capítulo anterior nós criamos nosso primeiro programa em Clojure. Vimos como lidar com dependências e assim manipular argumentos de linha de comando, fazer requisições HTTP e interpretar JSON. Neste capítulo vamos começar um projeto um pouco mais complexo utilizando o domínio que abordamos na parte 2. Começaremos o processo de dar vida àqueles trechos de código e colocá-los, aos poucos, em um serviço. Este mesmo projeto evoluirá ao longo desta parte do livro.

E, ao fim desta parte, você terá desenvolvido uma boa demonstração de um cenário onde as linguagens de Programação Funcional se destacam: processar requisições HTTP. Em resumo, o que acontece é que um programa recebe dados de uma requisição, realiza algum processamento com eles e retorna o resultado. Em um cenário onde vemos uma crescente busca por microsserviços (que se comunicam via HTTP ou não), Programação Funcional acaba ficando ainda mais atraente.

Vamos começar?

10.1 O ESQUELETO COM O LEININGEN

Vimos no capítulo anterior o uso do `template app`. Aqui veremos o uso de um outro template, baseado na biblioteca `Compojure` (<https://github.com/weavejester/compojure>). Esta biblioteca nos ajuda a lidar com o roteamento de requisições HTTP, permitindo a criação de pequenas aplicações Web. `Compojure` é uma abstração sobre o `Ring` (<https://github.com/ring-clojure/ring>), que nos protege de detalhes de lidar com HTTP. `Ring` se inspira no `Rack`, de Ruby, e no `WSGI`, do Python.

O código-fonte deste capítulo pode ser encontrado aqui:
<https://gitlab.com/programacaofuncional/financeiro/>

Para criar o projeto, vamos rodar o seguinte comando no terminal:

```
lein new compojure financeiro
```

Compojure no nosso código

Perceba que há uma nova pasta, `financeiro`, que o Leiningen criou para nós. Diferente do que foi criado com o `template app`, dentro da pasta `src/financeiro` tem um arquivo chamado `handler.clj`. Este arquivo contém o código que lida com a manipulação de rotas e já vem com algum conteúdo para nos ajudar no arranque. Vamos vê-lo por partes.

```
(ns financeiro.handler
  (:require [compojure.core :refer :all]
            [compojure.route :as route]
            [ring.middleware.defaults :refer [wrap-defaults]]
```

```
site-defaults]]))  
  
;; ...o resto...
```

Neste trecho, vemos:

- a importação de todas as funções e macros no namespace `compojure.core` ;
- depois é feita uma referência a `compojure.route` , sob o nome de `route` ; e
- por último, dois itens do Ring: `wrap-defaults` (uma função) e `site-defaults` (um mapa com configurações para API HTTP), que se encontram no namespace `ring.middleware.defaults` .

Rotas em ação

Em seguida, vemos o uso de algumas das funções e macros importadas:

```
;; ...antes daqui são as importações...  
  
(defroutes app-routes  
  (GET "/" [] "Hello World")  
  (route/not-found "Not Found"))  
  
;; ...
```

`defroutes` é uma macro que vem da importação de `compojure.core` e é onde dizemos o que o programa deve fazer quando uma determinada URL for acessada. As rotas que já são tratadas são:

- a raiz (/), via `GET` , retornando `200 OK` , com o texto `"Hello World"` ; e
- qualquer outra coisa que não seja a raiz, com qualquer

verbo HTTP, retornando um 404 "Not Found" , graças a route/not-found .

Vamos ver o que acontece quando visitamos estas rotas? No terminal, na pasta raiz do projeto, rode o seguinte comando:

```
lein ring server
# ...
# Started server on port 3000
```

Este comando faz com que um servidor HTTP, o Jetty, seja iniciado e possamos utilizar nossa API. Por padrão, o servidor sobe na porta 3000, mas podemos utilizar outra apenas anexando o número ao final do comando (por exemplo, lein ring server 4444).

Você deve ter percebido que uma nova janela do seu navegador padrão se abriu apontando para `http://localhost:3000/` , com o texto "Hello World" . Assim como esperado, a rota raiz, com GET , que é o verbo usado na requisição pelo navegador, retorna o texto que vemos na tela.

DICA: SUBINDO O SERVIDOR SEM INTERRUPÇÕES

É muito chato o fato de uma nova janela do navegador ser aberta sempre que o servidor subir. Pior ainda quando o navegador padrão não está nem aberto. Felizmente, podemos substituir o comando pelo seguinte: `lein ring server-headless` . Daí o servidor sobe normalmente e não nos interrompe com uma janela inesperada. E faz muito mais sentido, tendo em vista que estamos gerando uma API, sem a expectativa de termos conteúdo que exijam um navegador.

Agora vejamos aquele `route/not-found` em ação. Visite qualquer outro endereço, por exemplo `http://localhost:3000/saldo` , e veja que o resultado é "Not Found" , com código HTTP `404` de resposta. Mas eu falei que este era o resultado esperado para qualquer rota, com qualquer verbo, não foi? Sendo assim, é possível ver um `404` mesmo tendo a raiz como rota, basta ter um verbo diferente. Vejamos como pode acontecer. No terminal, vamos utilizar o comando `curl` para indicar qual verbo queremos:

```
curl -X OPTIONS http://localhost:3000/  
# Not Found
```

Mesmo sendo uma requisição para a raiz da API, como o verbo é `OPTIONS` , o resultado também é `404` .

cURL E CLIENTES REST

Os exemplos do livro fazem uso do cURL para as requisições ao serviço, mas há outras alternativas que podem ser utilizadas. Embora o cURL esteja presente (ou ao menos disponível) em quase todos os sistemas operacionais, outras ferramentas mais amigáveis existem. Insomnia (<https://insomnia.rest/>), Postman (<https://www.getpostman.com/>) e Paw (<https://paw.cloud/>, pago e só para Mac) são bons exemplos.

Voltemos à macro `defroutes`. Ela introduz uma forma conveniente de indicar as rotas disponíveis e o que fazer quando são chamadas, abstraindo a chamada à função `routes`, do Compojure. O conteúdo de `def-routes` é passado como argumento para `routes`. Então a macro, em ação, faz com que o trecho de código seja o mesmo que o seguinte:

```
(def app-routes
  (routes ;; função que se encontra em `compojure.core`
    (GET "/" [] "Hello World")
    (route/not-found "Not Found")))
```

O nome `app-routes` não é especial. Podemos usar qualquer um. Às vezes, vemos mais de um conjunto de rotas sendo definido em um mesmo arquivo, e em casos assim vemos `defroutes` sendo utilizada mais de uma vez, com nomes variados. `routes` é que é de fato importante aqui.

Rotas e wrappers

O trecho final em `handler.clj` é o seguinte:

```
;; ...já vimos o que tem aqui para cima

(def app
  (wrap-defaults app-routes site-defaults))
```

Este trecho parece bem inofensivo, mas é muito importante. Aqui entram em uso `wrap-defaults` e `site-defaults`, que foram importadas no começo do arquivo. `wrap-defaults` é uma função que pega a definição de rotas (`app-routes`) e define algumas configurações (`site-defaults`). `site-defaults` é um mapa que descreve configurações padrão, como a habilitação de *cookies*, proteção contra *cross-site scripting*, UTF-8 como codificação padrão. No próximo capítulo nós vamos mudar a configuração de `site-defaults` para `api-defaults`, já que estamos construindo uma API. Mas, por enquanto, deixemos como está.

Cadê o `main` ?

O projeto que fizemos no capítulo anterior tinha uma função `main` que era o ponto de entrada para o programa. Para programas feitos com Compojure (com Ring, na verdade), precisamos dizer qual o ponto de entrada no arquivo `project.clj`. Se você abrir este arquivo, vai perceber que ele tem algumas configurações e dependências a mais que o projeto do capítulo anterior.

E uma destas configurações é `:ring {:handler financeiro.handler/app}`. É aí onde definimos qual é o ponto

de entrada para o nosso programa. O Ring sabe ouvir as requisições HTTP, mas não sabe o que fazer com elas. Por isso, precisamos dizer para quem o Ring deve delegar a responsabilidade de interpretar a requisição.

E, do ponto de vista do código, isso é tudo o que precisamos para começarmos a ter sites e serviços no ar com o Compojure. Vamos falar agora sobre testes.

10.2 TESTES

Tanto este projeto quanto o projeto do capítulo anterior têm casos simples de teste que foram criados pelos templates no Leiningen. Convido você a voltar para a pasta raiz do projeto do capítulo anterior e rodar o seguinte comando:

```
# lembre-se de que estamos falando do projeto anterior!
# cd ../conversor
lein test
```

O resultado deve ser algo assim:

```
lein test conversor.core-test

lein test :only conversor.core-test/a-test

FAIL in (a-test) (core_test.clj:7)
  FIXME, I fail.
expected: (= 0 1)
  actual: (not (= 0 1))

Ran 1 tests containing 1 assertions.
1 failures, 0 errors.
Tests failed.
```

Isso quer dizer que há um caso de teste chamado `a-test` no arquivo `core_test.clj`. Ele espera que `0` seja igual a `1` e,

claro, o teste falha porque esta afirmação não é verdadeira. Não vamos consertá-lo, mas sim evoluir o projeto deste capítulo praticando TDD.

O template `compojure` também cria alguns cenários para nós. Na pasta raiz do projeto deste capítulo, rode o seguinte comando:

```
lein test
```

E o resultado deve ser o seguinte:

```
lein test financeiro.handler-test

Ran 1 tests containing 3 assertions.
0 failures, 0 errors.
```

Isso quer dizer que há 1 caso de teste com 3 asserções (minha tradução favorita para *assertion*), sendo que nenhuma delas falhou. Vejamos o que dizem estas asserções. No arquivo `handler_test.clj`, vemos o seguinte, por partes novamente:

```
(ns financeiro.handler-test
  (:require [clojure.test :refer :all]
            [ring.mock.request :as mock]
            [financeiro.handler :refer :all]))

;; ...mais coisas depois daqui...
```

Este arquivo de teste define um novo namespace, `financeiro.handler-test`, e importa algumas coisas:

- a biblioteca padrão de teste (`clojure.test`);
- uma biblioteca de *mocking* de requisições para o Ring (`ring.mock.request`), que vamos detalhar mais à frente; e
- e o namespace que vamos testar (`financeiro.handler`).

Daí surge o caso de teste com suas asserções:

```
(deftest test-app
  (testing "main route"
    (let [response (app (mock/request :get "/"))]
      (is (= (:status response) 200))
      (is (= (:body response) "Hello World"))))

  (testing "not-found route"
    (let [response (app (mock/request :get "/invalid"))]
      (is (= (:status response) 404)))))
```

Aqui temos dois cenários, um para cada rota que o programa tem:

- para a rota raiz, no cenário nomeado "main route" , que verifica que o código de resultado da requisição é 200 e que o corpo da resposta é "Hello World" ; e
- para uma rota inválida, em "not-found route" , que verifica que o código de resposta é 400 (se bem que poderia também verificar que o corpo da resposta é "Not Found").

Estes são testes unitários para as rotas. Com o uso de *mocks* para as requisições, o fluxo de execução passa apenas pelo conteúdo de `handler.clj` .

Apesar de Clojure oferecer uma boa ferramenta de testes como parte da plataforma, existe uma alternativa que prefiro: o Midje (<https://github.com/marick/Midje>). É a ferramenta que vamos utilizar daqui para a frente no livro. Vamos começar migrando os testes atuais.

Midje: uma ferramenta alternativa de testes

Midje traz uma proposta diferente para a descrição de casos de testes, que é baseada na forma como diversos livros de Clojure abordam exemplos do que é esperado que um trecho de código faça. Também oferece algumas ferramentas interessantes para utilizarmos no REPL.

Vamos começar a utilizar o Midje incluindo as dependências necessárias. Em `project.clj` há uma seção para dependências puramente para desenvolvimento, onde incluímos aquelas que não são necessárias para o nosso programa rodar em produção. Já temos `servlet-api` e `ring-mock`, e agora vamos incluir o Midje.

Em `project.clj`, vamos alterar o final do arquivo. Deixará de ficar assim:

```
{:dev {:dependencies [[javax.servlet/servlet-api "2.5"]
                     [ring/ring-mock "0.3.2"]]]}
```

Para ficar assim:

```
{:dev {:dependencies [[javax.servlet/servlet-api "2.5"]
                     [ring/ring-mock "0.3.2"]
                     [midje "1.9.6"]]
      :plugins [[lein-midje "3.2.1"]]]}
```

Aqui incluímos uma dependência de desenvolvimento, anexando `[midje "1.9.6"]` ao vetor de dependências que já existiam, e também colocamos um plugin, `lein-midje`, que nos permite rodar testes feitos com o Midje via Leiningen. Assim como rodamos os testes com `lein test`, vamos rodá-los com `lein midje`, por isso o plugin.

Vamos verificar se a inclusão das dependências deu certo. No terminal:

```
lein midje
```

O resultado esperado é:

```
>>> Midje summary:
No facts were checked. Is that what you wanted?

>>> Output from clojure.test tests:

Ran 1 tests containing 3 assertions.
0 failures, 0 errors.
```

Perceba que a segunda linha temos a indicação de que nenhum fato foi verificado. Midje propõe o uso do termo "fato" para um grupo mínimo de verificações que façam sentido. E também, na quarta linha, fala-se em `clojure.test`, o que é normal, já que ainda temos que migrar os testes para o formato do Midje.

Vamos lá migrar os testes que temos. Em `test/financeiro/handler_test.clj`, vamos importar o Midje e remover `clojure.test`. Na parte que define o que deve ser importado, no topo, vamos mudar para o seguinte:

```
(ns financeiro.handler-test
  (:require [midje.sweet :refer :all]
            [ring.mock.request :as mock]
            [financeiro.handler :refer :all]))

;; ...depois daqui são os fatos...
```

Removemos a importação das funções em `clojure.test` e importamos as do Midje, fazendo referência a todas as funções no namespace `midje.sweet`. As demais importações se mantêm.

Agora vamos reescrever os casos de teste. Cada asserção virará um fato, e os fatos relacionados podem ser agrupados em `facts`. No Midje, as asserções são escritas assim:

```
(fact "A sede da Copa do Mundo de Futebol de 2014 foi o Brasil"
  (sedes 2014) => "Brasil")
```

=> é uma macro que nos ajuda a pensar que quando a função à esquerda for aplicada, o resultado esperado é o valor à direita. Neste exemplo, supondo que `sedes` é um mapa cuja chave é o ano do evento e o valor o nome do país-sede, o valor esperado para a chave 2014 é "Brasil". Como a afirmação é verdadeira (ainda que muita gente queira repetir o jogo da semifinal contra a Alemanha), o teste passa.

Com esta migração de ferramenta, cada `testing`, que agrupa asserções, vira `facts`. E, para as asserções, temos duas opções: cada asserção pode virar um `fact` ou podemos agrupar asserções em um só `fact`. Vejamos como nosso código fica com um só `fact` para cada asserção:

```
(facts "Dá um 'Hello World' na rota raiz"
  (fact "o status da resposta é 200"
    (let [response (app (mock/request :get "/"))]
      (:status response) => 200))

  (fact "o texto do corpo é 'Hello World'"
    (let [response (app (mock/request :get "/"))]
      (:body response) => "Hello World"))))

(facts "Rota inválida não existe"
  (fact "o código de erro é 404"
    (let [response (app (mock/request :get "/invalid"))]
      (:status response) => 404)))
```

Vamos rodar os testes para ver o resultado? No terminal, invocamos o Leiningen e o plugin do Midje:

```
lein midje
```

```
# All checks (3) succeeded.
```

Boa! Os testes passaram. Significa que nossa migração foi bem-sucedida. Vamos ver o que acontece quando os testes falham? Vamos experimentar trocar a mensagem da rota raiz para algo em português, reescrevendo um pouco o primeiro grupo de fatos:

```
(facts "Dá um 'Olá, mundo!' na rota raiz"
  (fact "o status da resposta é 200"
    (let [response (app (mock/request :get "/"))]
      (:status response) => 200))

  (fact "o texto do corpo é 'Olá, mundo!'"
    (let [response (app (mock/request :get "/"))]
      (:body response) => "Olá, mundo!")))
```

Antes de rodarmos os testes, vamos ver um recurso bacana do Midje: a execução automática de testes. Para isso, basta adicionar o sufixo `:autotest` no comando do Leiningen:

```
lein midje :autotest
```

```
# FAIL "Dá um 'Olá, mundo!' na rota raiz - o texto do corpo é ..."
#   Expected: "Olá, mundo!"
#   Actual: "Hello World"
#   Diffs: strings have 1 difference (0% similarity)
#           expected: "(Olá, mundo!)"
#           actual:   "(Hello World)"
# FAILURE: 1 check failed. (But 2 succeeded.)
```

A resposta no terminal é, claro, de que os testes falharam porque o resultado esperado não foi encontrado. Façamos o teste passar, alterando a rota raiz para que ela retorne o texto esperado. Em `handler.clj`:

```
(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  (route/not-found "Not Found"))
```

Perceba que os testes rodaram automaticamente, dizendo-nos que o resultado foi um sucesso! O terminal deve mostrar o

seguinte:

```
Loading (financeiro.handler financeiro.handler-test)
All checks (3) succeeded.
[Completed at 18:57:18]
```

Já que mudamos o texto da rota raiz, vamos mudar também o texto da rota não encontrada, começando pelo teste:

```
(facts "Rota inválida não existe"
  (fact "o código de erro é 404"
    (let [response (app (mock/request :get "/invalid"))]
      (:status response) => 404)))

;; este fato é novo
(fact "o texto do corpo é 'Recurso não encontrado'"
  (let [response (app (mock/request :get "/invalid"))]
    (:body response) => "Recurso não encontrado"))
```

Em handler.clj :

```
(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  (route/not-found "Recurso não encontrado"))
```

Jóia! Os testes passam. Hora de refatorar os testes, afinal devemos prezar pela qualidade do código de teste também. Existe uma duplicação das requisições em cada `facts` . Os mocks das requisições são iguais em cada um dos fatos dentro dos agrupamentos. Podemos remover a duplicação assim:

```
(facts "Dá um 'Olá, mundo!' na rota raiz"
  (let [response (app (mock/request :get "/"))]
    (fact "o status da resposta é 200"
      (:status response) => 200)

    (fact "o texto do corpo é 'Olá, mundo!'"
      (:body response) => "Olá, mundo!")))

(facts "Rota inválida não existe"
  (let [response (app (mock/request :get "/invalid"))]
```

```
(fact "o código de erro é 404"
  (:status response) => 404)

(fact "o texto do corpo é 'Recurso não encontrado'"
  (:body response) => "Recurso não encontrado"))
```

Cobertura de testes

Que tal fechar o capítulo avaliando como está a cobertura dos nossos testes? Faremos isto incluindo o plugin Cloverage, que nos indica quanto do nosso código é coberto a partir dos nossos testes. Em `project.clj`, altere os plugins de desenvolvimento para o seguinte:

```
:plugins [[lein-midje "3.2.1"]
          [lein-cloverage "1.0.13"]]
```

E, no terminal, rode o seguinte comando:

```
lein cloverage --runner :midje
```

O Cloverage gerará relatórios na pasta `target/coverage` e também exibirá uma tabela no terminal. Contamos com 100% de cobertura! Ainda que tenhamos feito pouco código até aqui, devemos celebrar as pequenas conquistas também.

10.3 CONCLUSÃO

Vimos bastante coisa até aqui que nos ajudam com o fundamental para a construção de uma API em Clojure. Cobrimos a construção de um esqueleto com o Leiningen e o template do Compojure; como subir um servidor HTTP para ver nossa aplicação no ar; onde são definidas as rotas do nosso programa; como são feitos testes unitários com o Midje; e como ver um relatório de cobertura de testes.

Vamos começar o próximo capítulo construindo um teste de aceitação para a primeira funcionalidade da nossa API: obter o saldo. Vamos começar devagar, verificando que não há saldo. Depois incluiremos o processamento de JSON, a princípio apenas nas respostas da API. E encerraremos o capítulo publicando nossa API como um artefato, permitindo que disponibilizemos o serviço sem a necessidade do Leiningen.

EVOLUÇÃO DA API: SALDO INICIAL

No capítulo anterior nós começamos a construir nossa API para gerenciamento de finanças pessoais. Concluímos o esqueleto com Compojure, com direito a uma suíte de testes, e deixamos as funcionalidades para os capítulos seguintes.

Neste capítulo veremos como incluir uma nova rota, `/saldo`, começando com a construção de um teste de aceitação. Enquanto criamos e evoluímos a rota, alteraremos algumas configurações da API. Uma destas mudanças permitirá que lidemos com JSON como formato de troca de informação com clientes da API.

11.1 COMEÇANDO COM TESTES DE ACEITAÇÃO

A proposta aqui é que tenhamos uma rota `/saldo` que nos retorne o saldo que consta na base de transações. Como ainda não temos registros de transações, esta rota vai retornar `0`. Então poderemos rodar o seguinte comando:

```
curl http://localhost:3000/saldo
```

E o resultado esperado deverá ser `0`. No momento, claro, o

resultado é Recurso não encontrado .

Já vimos onde ficam as rotas e como criar testes unitários para elas, e agora nosso trabalho será criar uma nova rota, que também terá testes unitários. E ainda podemos aumentar nossa confiança de que o programa comporta-se como esperado incluindo testes de aceitação automatizados.

Então vamos lá criar nosso primeiro teste de aceitação. O teste tem que ser capaz de:

- Iniciar um servidor HTTP;
- Fazer uma requisição para /saldo ;
- Conferir que o resultado é 0 ; e
- Parar o servidor HTTP.

Iniciar o servidor HTTP

Em vez de criar o teste todo logo agora, vamos por parte, começando por ver como manipulamos o servidor. Sem um servidor, os testes não vão rodar. Portanto, precisamos incluir a dependência para manipular o servidor HTTP. Em `project.clj` , inclua, nas dependências de desenvolvimento, `ring-core` e `ring-jetty-adapter` :

```
{:dev {:dependencies [[javax.servlet/servlet-api "2.5"]
                     [ring/ring-mock "0.3.2"]
                     [midje "1.9.6"]
                     [ring/ring-core "1.7.1"]
                     [ring/ring-jetty-adapter "1.7.1"]]
```

`ring-jetty-adapter` é a biblioteca que nos ajudará a iniciar ou parar um servidor HTTP. `ring-core` é o núcleo do Ring, e é uma dependência do `ring-jetty-adapter` .

Se você tiver rodado os testes com `:autotest` e eles ainda estiverem rodando, será preciso encerrar o processo e iniciar novamente para que as novas dependências sejam copiadas da internet. Se o terminal onde o comando foi rodado não estiver esperando comandos, também é este o caso. Na dúvida, pode encerrar com `Control + C` e rodar novamente `lein midje :autotest`, que a próxima execução acontecerá com as novas bibliotecas.

Com as primeiras dependências resolvidas, vamos criar um novo arquivo para o teste de aceitação. Na pasta `test/financeiro`, crie um arquivo chamado `saldo_aceitacao_test.clj`. Neste arquivo, no topo, vamos definir o namespace e importar a função que nos permite subir o servidor HTTP Jetty:

```
(ns financeiro.saldo-aceitacao-test
  (:require [ring.adapter.jetty :refer [run-jetty]]))
```

Salve o arquivo e repare que o `:autotest` já detectou o novo teste, indicando que não há fatos a serem verificados nele. Vamos aproveitar que está tudo rodando e colocar o código que inicia e para o Jetty. Altere o arquivo de teste para que ele fique como o seguinte:

```
(ns financeiro.saldo-aceitacao-test
  (:require [financeiro.handler :refer [app]]
            ;; a referência acima é pra dizer pro Jetty quais
            ;; são as rotas disponíveis
            [ring.adapter.jetty :refer [run-jetty]]))

(def servidor (atom nil))

(defn iniciar-servidor [porta]
  (swap! servidor
    (fn [_] (run-jetty app {:port porta :join? false}))))
```

```
(defn parar-servidor []  
  (.stop @servidor))
```

Aqui incluímos a referência às rotas que nossa aplicação trata e definimos como manipular o servidor: primeiro, criamos um átomo para guardar uma referência a uma instância do Jetty, e depois temos funções para iniciar e parar o servidor. `iniciar-servidor` define um valor para o átomo `servidor`, que é o resultado de um processamento da função `run-jetty`. Perceba que há uma função anônima em `iniciar-servidor` porque `swap!` espera uma função como argumento! `run-jetty`, por sua vez, espera saber quais são as rotas disponíveis. A definição de que porta será utilizada é parametrizada.

Perceba que tem um `_` na definição da função anônima dentro de `iniciar-servidor`. Usar `_` é uma convenção que diz que não é preciso dar um nome ao argumento porque não vamos utilizá-lo. É uma convenção bastante comum também em outras linguagens de Programação Funcional.

E depois, na função `parar-servidor`, temos uma interoperação com Java: `(.stop @servidor)` quer dizer que vamos chamar o método `stop()` da instância referenciada por `@servidor`.

Vamos criar um fato só para experimentar o processo de iniciação e encerramento do servidor. Precisaremos incluir a referência ao Midje, claro! Nosso arquivo ficará assim:

```
(ns financeiro.saldo-aceitacao-test
```

```

(:require [midje.sweet :refer :all] ;; referência ao Midje
  [financeiro.handler :refer [app]]
  [ring.adapter.jetty :refer [run-jetty]])

(def servidor (atom nil))

(defn iniciar-servidor [porta]
  (swap! servidor
    (fn [_] (run-jetty app {:port porta :join? false}))))

(defn parar-servidor []
  (.stop @servidor))

;; fato para verificar a manipulação do servidor
(fact "Inicia e para o servidor"
  (iniciar-servidor 3001)
  (parar-servidor))

```

Perceba que há mensagens no terminal que indicam que o servidor foi iniciado e encerrado. O texto deve ser algo assim:

```

2017-09-19 21:41:06.700:INFO:oejs.servidor:pool-1-thread-1: jet .
..
2017-09-19 21:41:06.704:INFO:oejs.servidorConnector:pool-1-thre .
..
0:3001}
2017-09-19 21:41:06.704:INFO:oejs.servidor:pool-1-thread-1: Sta .
..
2017-09-19 21:41:06.705:INFO:oejs.servidorConnector:pool-1-thre .
..
0:3001}

```

Isso é bom. Conseguiremos preparar o servidor para os nossos testes. Podemos apagar este fato, já que ele não é um teste de verdade. Com isto pronto, resta-nos focar em como nosso teste fará uma requisição para este servidor. O resultado desta requisição é o que será verificado pelo teste. E precisaremos incluir uma dependência para fazer a requisição HTTP, que é a mesma que incluímos no projeto do capítulo 9. Em `project.clj`, inclua a `clj-http`, que nos habilita fazer as requisições HTTP:

```
:dependencies [[org.clojure/clojure "1.9.0"]
               [compojure "1.6.0"]
               [ring/ring-defaults "0.3.1"]
               [clj-http "3.9.1"]]
;; inclusão da biblioteca clj-http acima
```

E no teste, inclua a referência a `clj-http.client`, para que possamos chamar as funções da biblioteca `clj-http`:

```
(ns financeiro.saldo-aceitacao-test
  (:require [midje.sweet :refer :all]
            [financeiro.handler :refer [app]]
            [ring.adapter.jetty :refer [run-jetty]]
            [clj-http.client :as http]))
;; referência ao namespace client
;; da biblioteca clj-http
```

Agora temos o que precisamos para construir nosso caso de teste. Vamos fazer com que nosso teste faça uma requisição HTTP para `http://localhost:3001/saldo` e vamos nos assegurar de que o saldo é `0`. Sendo assim, quando este teste eventualmente passar, saberemos que a funcionalidade está pronta. Vejamos como nosso teste vai ficar:

```
(fact "O saldo inicial é 0"
  (iniciar-servidor 3001)

  ;; faz uma requisição HTTP e verifica que o retorno é 0
  (:body (http/get "http://localhost:3001/saldo")) => "0"

  (parar-servidor))
```

Ao rodar o teste, perceba que o resultado não é nem um pouco o esperado. Na verdade, a resposta que recebemos é que a rota não existe. Claro! Ainda não a implementamos. Pensando em passos pequenos, façamos o teste passar criando uma nova rota e retornando `"0"` de imediato. Em `handler_test.clj`, criemos novos testes unitários para verificar a rota:

```
(facts "Saldo inicial é 0"
  (let [response (app (mock/request :get "/saldo"))]
    (fact "o status da resposta é 200"
      (:status response) => 200)

    (fact "o texto do corpo é '0'"
      (:body response) => "0")))
```

Aqui criamos uma requisição falsa, com `mock/request`, para a rota `/saldo`. A resposta a essa requisição é referenciada por `response`, dentro do `let`. E, com esta requisição falsa feita, verificamos dois fatos: que o status da resposta é `200` e que o corpo da resposta é `"0"`. O teste falha, claro, porque ainda não acertamos o código de produção. Vamos resolver isto já!

Este teste em particular mudará no próximo capítulo, quando criarmos a lógica de manipulação de transações para um namespace específico. A verificação da existência da rota será mantida, mas precisaremos criar um *mock* para usar neste teste (veremos como criar nossos próprios mocks mais à frente ainda neste capítulo).

Para criar a rota, em `handler.clj`, substitua a definição das rotas pelo seguinte:

```
(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  (GET "/saldo" [] "0")
  (route/not-found "Recurso não encontrado"))
```

Todos os testes passam!

Filtrando testes

Só que agora os testes unitários e os de aceitação rodam juntos, ainda não há distinção entre eles além de seus nomes. Normalmente buscamos rodar os unitários primeiro e, em seguida, os de aceitação. Como os testes unitários são mais rápidos, já que o código coberto pelo teste é menor, temos um feedback mais imediato caso algo dê errado. Os testes de aceitação, por sua vez, envolvem o carregamento de dependências e cobrem mais trechos do código, fazendo com que eles sejam bons para validar que partes distintas, mas relacionadas, conseguem interagir entre si. Mas como cobrem mais trechos, demoramos um pouco mais para identificar a causa de uma falha no teste.

O Midje provê uma forma de agrupar testes com rótulos para que possamos filtrá-los depois. Basta que cada fato descreva qual rótulo se aplica a ele. Vamos rotular o teste de aceitação que temos:

```
;; o rótulo é logo depois do texto que descreve o teste
(fact "O saldo inicial é 0" :aceitacao ;; o rótulo que o Midje
                                   ;; vai enxergar

  (iniciar-servidor 3001)

  (:body (http/get "http://localhost:3001/saldo")) => "0"

  (parar-servidor))
```

O código que acabamos de ver é praticamente igual ao teste de aceitação que escrevemos há pouco, com uma ligeira diferença: temos o rótulo `:aceitacao` logo depois da descrição do teste. É assim que fazemos o uso dos rótulos do Midje. Para rodar só os testes com o rótulo `aceitacao`, fazemos assim:

```
lein midje :filters aceitacao
```

E para rodar quaisquer outros testes que não sejam de aceitação:

```
lein midje :filters -aceitacao
```

Rodar `lein midje`, sem filtros, faz com que todos os testes rodem.

Separação física dos testes

Já separamos logicamente os tipos de testes uns dos outros. Mas podemos ir além e melhorar nossa estrutura de arquivos: ter pastas que separam testes por seus tipos. Ainda assim precisaremos dos filtros, mas teremos mais clareza sobre onde está cada tipo de teste.

Vamos renomear a pasta de teste que temos para que ela indique que seu conteúdo é de testes unitários e, em seguida, criar uma pasta de testes de aceitação e mover para lá o teste de aceitação que temos. Se você está em Linux ou Mac, na pasta raiz do projeto, faça o seguinte no terminal:

```
mkdir -p test/{unitarios,aceitacao}/financeiro
mv test/financeiro/handler_test.clj test/unitarios/financeiro/
mv test/financeiro/saldo_aceitacao_test.clj \
  test/aceitacao/financeiro/
rm -rf test/financeiro
```

Neste ponto, se rodarmos os testes, veremos erros indicando que não foram encontrados os arquivos de teste. Como modificamos a estrutura padrão de arquivos, precisamos dizer para o Leiningen onde os encontrar, e isso acontece na configuração `:test-paths` no arquivo `project.clj`. Vamos ajustar esta configuração, colocando as pastas `test/unitarios` e `test/aceitacao` como os valores de `:test-paths`. Assim:

```
(defproject financeiro "0.1.0-SNAPSHOT"
  :description "FIXME: write description"
  :url "http://example.com/FIXME"
```

```

:min-lein-version "2.0.0"
:dependencies [[org.clojure/clojure "1.9.0"]
               [compojure "1.6.1"]
               [ring/ring-defaults "0.3.2"]
               [clj-http "3.9.1"]]
:plugins [[lein-ring "0.12.1"]]
:ring {:handler financeiro.handler/app}
:profiles {:dev {:dependencies [[javax.servlet/servlet-api "2.5
]
                                [ring/ring-mock "0.3.2"]
                                [midje "1.9.6"]
                                [ring/ring-core "1.7.1"]
                                [ring/ring-jetty-adapter "1.7.1"
]
                                :plugins [[lein-midje "3.2.1"]
                                           [lein-cloverage "1.0.13"]]]}}
;; a linha a seguir define onde ficam os testes
:test-paths ["test/unitarios" "test/aceitacao"])

```

Apenas a última linha é novidade. Perceba que agora temos sucesso ao rodar os testes. Se você estava rodando os testes no modo automático, precisará encerrar o processo e começar um novo.

Refatoração dos testes

No nosso teste de aceitação, o conteúdo do teste manipula o servidor, preparando o cenário para que o teste rode, já que depende de um servidor no ar, e limpando o cenário quando o teste acabar. Estes processos de preparar o cenário e limpá-lo são chamados, respectivamente, de *setup* e *teardown*. Setup e teardown não fazem parte do teste em si, mas são necessários. Quando eles estão dentro do corpo do teste, acabam nos atrapalhando na identificação de onde o teste de fato começa e termina.

Vamos refatorar nosso teste de aceitação para contar com a ajuda do Midje para separar o setup e o teardown do teste em si.

Primeiro, relembremos como o teste está:

```
(fact "O saldo inicial é 0" :aceitacao
  ;; configuração de dependências para o teste
  (iniciar-servidor 3001)

  (:body (http/get "http://localhost:3001/saldo")) => "0"

  (parar-servidor)) ;; encerramento das dependências do teste
```

Como o teste precisa ter um servidor rodando, este é iniciado antes do teste. O Midje chama esta configuração de *background* e permite que a gente diga o que deve ser executado antes ou depois dos testes.

Vamos ajustar nosso teste de aceitação para mover a iniciação do servidor para uma etapa de setup:

```
(against-background [(before :facts (iniciar-servidor 3001))]  
  (fact "O saldo inicial é 0" :aceitacao  
    (:body (http/get "http://localhost:3001/saldo")) => "0"  
    (parar-servidor)))
```

Perceba que nosso fato agora é encoberto por `against-background`. Ele é o recurso do Midje que justamente vai nos ajudar com setup e teardown. No nosso caso, o setup que estava dentro do fato foi movido para dentro de um array que é passado como argumento para `against-background`. O trecho `before :facts (iniciar-servidor...` indica ao Midje que ele deve aplicar a função `iniciar-servidor` antes de todos os fatos agrupados dentro deste `against-background`. Por enquanto só temos um, mas quando criarmos funcionalidades que nos permitam alterar o saldo, teremos mais fatos aqui que se aproveitarão deste setup.

Temos também o encerramento do servidor HTTP, que se

caracteriza como etapa de limpeza do cenário, ou *teardown*. Vamos movê-lo do fato em si para uma das configurações do Midje:

```
(against-background [(before :facts (iniciar-servidor 3001))
                      (after :facts (parar-servidor))])
  (fact "O saldo inicial é 0" :aceitacao
    (:body (http/get "http://localhost:3001/saldo")) => "0"))
```

Perceba que o fato agora só tem o que estamos efetivamente verificando. Fica muito mais fácil entender o que se passa e fazer qualquer manutenção no teste.

Mas ainda temos uma certa duplicação: o número da porta, 3001, se repete. Vamos separá-la:

```
(def porta-padrao 3001)

(against-background [(before :facts
                            (iniciar-servidor porta-padrao))
                    (after :facts (parar-servidor))])
  (fact "O saldo inicial é 0" :aceitacao
    (:body (http/get (str "http://localhost:"
                        porta-padrao "/saldo")) => "0"))
```

Criamos uma referência a 3001 como nome `porta-padrao` e passamos esta referência para a aplicação da função `iniciar-servidor`. A mesma referência também foi utilizada para a construção da URL para a requisição `GET`.

Com os testes ainda passando, vamos para uma próxima refatoração: melhorar a legibilidade dentro do fato. Confesso que não acho legível o seguinte trecho:

```
(:body (http/get (str "http://localhost:"
                    porta-padrao "/saldo")) => "0"
```

Podemos fazer melhor que isso. A ideia é deixar claro que

temos interesse no corpo da resposta à requisição da rota `/saldo`. Podemos melhorar este trecho fazendo o seguinte:

- criar uma função que ajude a montar a string que corresponde à URL da requisição para a rota que estamos testando; e
- abstrair o uso de `:body`, visando facilitar a leitura do conteúdo da resposta.

Vamos começar definindo uma função para montar o endereço para a rota:

```
(defn endereco-para [rota] (str "http://localhost:"  
                                porta-padrao rota))
```

O código anterior é bem simples, contendo apenas concatenação de strings. O próximo é um pouco mais complexo:

```
(def requisicao-para (comp http/get endereco-para))
```

Aqui criamos uma composição de funções que nos ajuda a abstrair o uso de `http/get` e a construção de todo o endereço para a rota. Como estas duas operações andam juntas, faz sentido uni-las. Agora, para focar apenas no conteúdo da resposta:

```
(defn conteudo [rota] (:body (requisicao-para rota)))
```

Aqui chamamos a composição que criamos, `requisicao-para`, passando a rota que desejamos, `"/saldo"`, no teste de aceitação que temos. E agora o fato fica mais autoexplicativo:

```
(fact "O saldo inicial é 0" :aceitacao  
      (conteudo "/saldo") => "0")
```

Porém, acabamos ficando com muitas funções auxiliares no arquivo de teste. Podemos organizar melhor a distribuição das

responsabilidades se movermos os procedimentos auxiliares para outro arquivo e referenciá-lo no arquivo de teste. Na pasta `test/aceitacao/financeiro`, crie um arquivo chamado `auxiliares.clj`. Vamos mover para ele tudo o que não for cenário de teste do arquivo `saldo_aceitacao_test.clj`. O conteúdo do novo arquivo ficará assim:

```
(ns financeiro.auxiliares
  (:require [ring.adapter.jetty :refer [run-jetty]]
            [financeiro.handler :refer [app]]
            [clj-http.client :as http]))

(def servidor (atom nil))

(defn iniciar-servidor [porta]
  (swap! servidor
    (fn [_] (run-jetty app {:port porta :join? false}))))

(defn parar-servidor []
  (.stop @servidor))

(def porta-padrao 3001)

(defn endereco-para [rota] (str "http://localhost:"
                                porta-padrao rota))

(def requisicao-para (comp http/get endereco-para))

(defn conteudo [rota] (:body (requisicao-para rota)))
```

E o arquivo do teste de aceitação ficará mais sucinto:

```
(ns financeiro.saldo-aceitacao-test
  (:require [midje.sweet :refer :all]
            [financeiro.auxiliares :refer :all]))

(against-background [(before :facts (iniciar-servidor
                                     porta-padrao))
                    (after :facts (parar-servidor))])

(fact "O saldo inicial é 0" :aceitacao
  (conteudo "/saldo") => "0"))
```

Certamente teremos um ganho em manutenibilidade.

11.2 JSON COMO FORMATO DE REPRESENTAÇÃO DE DADOS

A estrutura da nossa API tem evoluído bastante: saímos de um esqueleto mínimo e agora temos uma organização para a nossa suíte de testes. Mas o conteúdo da nossa API ainda é retornado como HTML, que é o padrão adotado pelo template do Compojure. Olha o que o cURL nos diz no terminal:

```
curl --head http://localhost:3000/saldo
```

```
#...
```

```
Content-Type: text/html; charset=UTF-8
```

```
#...
```

A opção `--head` indica ao cURL para que ele exiba apenas informações sobre a resposta, e não o conteúdo.

Ainda que HTML não seja um empecilho aqui, existem formatos mais apropriados para tal, como JSON e XML. Lidaremos com JSON em nossa API, que é um formato bem mais enxuto que XML e há diversas ferramentas maduras para visualização de dados em JSON.

Já vimos como manipular JSON no capítulo 9, no projeto de conversão de moedas. Naquele projeto, nós consumíamos dados utilizando JSON, e agora passaremos a produzir desta forma. Também consumiremos, mas nos testes, visando garantir a

integridade do que estamos produzindo.

O processo de introduzir JSON vai envolver os seguintes passos:

- Incluir a biblioteca Cheshire para a manipulação de JSON;
- Alterar nosso teste de aceitação para que ele interprete o dado retornado, que agora utilizará JSON;
- Idem, mas para os testes unitários; e
- Alterar o código de produção.

Começamos incluindo a dependência à biblioteca Cheshire, em `project.clj` :

```
:dependencies [[org.clojure/clojure "1.10.0"]
               [compojure "1.6.1"]
               [cheshire "5.8.1"]
               [ring/ring-defaults "0.3.2"]
               [clj-http "3.9.1"]]
```

Uma outra coisa que vimos no capítulo 9 foi a forma de interpretar uma string em JSON através da função `parse-string`. Nosso teste de aceitação fará uso desta função para verificar o resultado esperado. Vamos ver como nosso teste de aceitação fica com a introdução de JSON. O arquivo `saldo_aceitacao_test.clj` ficará assim:

```
(ns financeiro.saldo-aceitacao-test
  (:require [midje.sweet :refer :all]
            ;; incluímos a bibliote para manipulação de JSON
            [cheshire.core :as json]
            [financeiro.auxiliares :refer :all]))

(against-background [(before :facts (iniciar-servidor
                                     porta-padrao))
                    (after :facts (parar-servidor))])

(fact "O saldo inicial é 0" :aceitacao
  ;; (conteudo "/saldo") deve retornar um JSON
```

```
;; e json/parse-string vai interpretar seu conteúdo
(json/parse-string (conteudo "/saldo")) => "0"))
```

Apesar de "0" ser um JSON válido, podemos fazer melhor, indicando do que este número se trata. Como estamos usando os testes para guiar o comportamento da nossa aplicação, podemos dizer como a estrutura do JSON deve ser definida. Vamos, portanto, esperar que nossa aplicação retorne um mapa com "saldo" como chave, e seu valor, um número que corresponda ao saldo. Nosso teste ficará assim:

```
(fact "0 saldo inicial é 0" :aceitacao
  ;; agora o que é esperado é um mapa
  ;; onde "saldo" é a chave
  (json/parse-string (conteudo "/saldo")) => {"saldo" 0}))
```

Como é de se esperar, este teste falha. O código de produção não foi atualizado para retornar o resultado como um mapa. O Midje nos diz o seguinte:

```
FAIL 0 saldo inicial é 0 at (saldo_aceitacao_test.clj:10)
Expected:
{"saldo" 0}
Actual:
0
```

O código da aplicação retorna 0, enquanto o teste espera um mapa com chave igual a "saldo" e valor igual a 0. Na verdade, nosso caso de teste pode ser um pouco mais idiomático se esperar que a chave do mapa seja uma keyword em vez de uma string, isto é, :saldo em vez de "saldo". Precisamos indicar para o Cheshire que esta é a forma que esperamos. Logo, o fato fica assim:

```
(fact "0 saldo inicial é 0" :aceitacao
  ;; json/parse-string pode receber um boolean como terceiro
  ;; argumento e caso seja _true_, o Cheshire vai utilizar
  ;; keywords em vez de String como chaves no mapa
  (json/parse-string (conteudo "/saldo") true) => {:saldo 0}))
```

PAUSA PARA OPINIÃO

Quero aproveitar para compartilhar uma opinião. Acabamos de ver uma função que recebe um boolean para indicar um desvio no seu comportamento padrão: `parse-string` pode, ou não, receber um boolean. Quando este boolean existir e for verdadeiro, ele faz com que a função gere um resultado diferente. Bob Martin (ou Uncle Bob), no livro *Clean Code*, chama argumentos como este de *flag arguments*. Neste mesmo livro, ele recomenda uma alternativa a funções com *flag arguments*: criar duas funções, uma para cada um dos desvios no comportamento, e cujos nomes indiquem estes desvios.

E eu concordo bastante neste ponto. No caso de `parse-string`, poderíamos ter duas funções: `parse-string-producing-strings-as-keys` e `parse-string-producing-keywords-as-keys`. Ou quaisquer outros nomes. O importante é que o nome da função seja o mais descritivo possível neste nível de abstração. Não há documentação melhor do que código autoexplicativo!

O nosso teste de aceitação continua falhando. Precisamos alterar nosso código de produção para que ele gere um JSON para nós. Nosso arquivo `handler.clj` fica assim:

```
(ns financeiro.handler
  (:require [compojure.core :refer :all]
            [compojure.route :as route]
            ;; não podemos esquecer de incluir a dependência
            [cheshire.core :as json])
```

```

[ring.middleware.defaults :refer [wrap-defaults
                                   site-defaults]]))

(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  ;; `generate-string` é a função do Cheshire que cria um JSON
  ;; e aqui passamos pra ela um mapa, {:saldo 0}
  (GET "/saldo" [] (json/generate-string {:saldo 0}))
  (route/not-found "Recurso não encontrado"))

(def app
  (wrap-defaults app-routes site-defaults))

```

O Cheshire tanto interpreta quanto gera JSON. A função `generate-string` espera um mapa com os dados a serem codificados (de *encoding*) e, opcionalmente, um segundo mapa com configurações. Um exemplo legal de um tipo de configuração é gerar um JSON embelezado, com indentação e espaçamento apropriados.

E, com esta mudança, nosso teste de aceitação passa! Mas, opa, esquecemos do teste unitário. Agora é ele que falha. Claro, mudamos o comportamento do código de produção sem ajustar o teste.

Conforme o Midje, o que acontece é que o resultado que obtemos é `"{\\"saldo\\":0}"`, diferente de `"0"`, que é o que o teste espera. Uma forma de resolver este problema é simples: só mudar o texto esperado no teste. Outra forma é criar um mock da interação com o Cheshire e deixar que o conteúdo deste mock seja lido pelo teste. Nesse caso, a unidade sendo testada estaria completamente isolada. Eu não tenho nenhum apego a nenhuma das duas, mas aqui neste livro vamos partir para a segunda abordagem. Não se preocupe se você prefere a primeira forma, às vezes a solução mais simples é a mais apropriada.

Acabamos de ver isto aqui: `\"` . Esta é a forma de escapar (permitir um uso anormal) caracteres dentro de strings. Se você quiser colocar aspas dentro de uma string, precisa escapar. No JSON que acabamos de ver, `saldo` está entre aspas, e o conteúdo está dentro de uma string. Por isso vemos as aspas sendo escapadas.

Vejamos como nosso `handler_test.clj` fica, ao focar no fato que falha. Primeiro precisamos incluir a referência ao Cheshire:

```
(ns financeiro.handler-test
  (:require [midje.sweet :refer :all]
            [ring.mock.request :as mock]
            ;; inclusão da referência o Cheshire
            [cheshire.core :as json]
            [financeiro.handler :refer :all]))
```

Mesmo sabendo que vamos criar um mock para esta a integração com o Cheshire, ainda será preciso criar uma referência para ele no `:require` . E agora precisamos alterar o teste:

```
(facts "Saldo inicial é 0"
  ;; aqui a gente define o mock
  (against-background (json/generate-string {:saldo 0}))
    => "{\\\"saldo\\\":0}")

  (let [response (app (mock/request :get "/saldo"))]
    (fact "o status da resposta é 200"
      (:status response) => 200)

    ;; e aqui nós mudamos a descrição e o conteúdo do fato
    (fact
      "o texto do corpo é um JSON cuja chave é saldo e o valor é
0"
```

```
(:body response) => "{\\"saldo\":0}"))
```

Sucesso! O que fizemos aqui foi criar um pano de fundo, um contexto, para os fatos relacionados a saldo. O mesmo `against-background` que utilizamos no teste de aceitação pode ser utilizado aqui também, desta vez com o propósito de configurarmos que vamos chamar uma dependência, com um argumento já especificado, e que esperamos que um determinado resultado seja retornado. No nosso caso, descrevemos que o nosso teste vai chamar uma função `json/generate-string`, com o argumento `{:saldo 0}`, e definimos que a string `"{\\"saldo\":0}"` seja retornada. Sendo assim, nosso teste unitário não interage de verdade com o Cheshire.

Mais uma vez, poderíamos ter resolvido a falha do teste apenas alterando o resultado esperado. E eu não faço nenhum julgamento com relação a esta abordagem. Ok, o teste não seria mais tão unitário, isolado, mas é algo muito simplório para jogarmos esta solução no ostracismo. Entretanto, há um outro forte motivo para que nossa solução tenha adotado `mocks`: agora temos o conhecimento necessário para, mais para a frente, criar `mocks` que serão indispensáveis! Em breve, teremos a necessidade de interagir com bancos de dados, e esta experiência virá bem a calhar.

11.3 O TOQUE FINAL

Nosso programa agora retorna o saldo como JSON, como podemos ver no terminal:

```
curl http://localhost:3000/saldo  
# {"saldo":0}
```

Acontece que o dado está no formato JSON, mas não é só isso

que os navegadores enxergam para entender que o resultado é, de fato, um JSON. O cabeçalho da resposta HTTP também é importante. Vejamos o que nossa resposta diz. No terminal:

```
curl -I http://localhost:3000/saldo
# ...
# Content-Type: text/html; charset=UTF-8
# ...
```

Dentre outras coisas, nossa aplicação retorna `text/html` como formato da resposta. Precisamos mudar isso para `application/json` e podemos deixar o `charset=UTF-8`. Para fazer isso, precisamos mudar um pouco nosso `handler.clj` de modo que a resposta indique o valor correto pra `Content-Type`.

Começando com a criação de um fato para verificar o cabeçalho da resposta, nosso `handler_test.clj` ganha um novo teste:

```
(facts "Saldo inicial é 0"
  (against-background (json/generate-string {:saldo 0})
    => "{\"saldo\":0}"))

(let [response (app (mock/request :get "/saldo"))]
  ;; o novo fato que verifica o 'Content-Type' no cabeçalho
  (fact "o formato é 'application/json'"
    (get-in response [:headers "Content-Type"])
    => "application/json; charset=utf-8")

  (fact "o status da resposta é 200"
    (:status response) => 200)

  (fact
    "o texto do corpo é um JSON cuja chave é saldo e o valor é 0"
    (:body response) => "{\"saldo\":0}"))
```

Este novo fato verifica o valor para a chave `Content-Type` dentro do cabeçalho. Para o código de produção, nosso ajuste é o

seguinte:

```
(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  (GET "/saldo" [] {:headers {"Content-Type"
                              "application/json; charset=utf-8"}
                    :body (json/generate-string {:saldo 0})})
  (route/not-found "Recurso não encontrado"))
```

Aqui fizemos com que o retorno de uma requisição a `/saldo` seja um mapa, cujas chaves são `:headers` e `:body`. Até então não foi preciso criar nada disso, e é porque o Ring nos dá uma ajuda aqui: para qualquer conteúdo diferente de um mapa, é criado um mapa com a chave `:body` e este conteúdo vira o valor para esta chave.

Agora que precisamos definir um valor no cabeçalho da resposta HTTP, consequentemente criando um mapa como resposta, precisamos explicitar a existência do `:body`. Perceba que o conteúdo para a chave `:headers` também é um mapa, mas suas chaves são strings, e não keywords. Isto é relevante se você quiser definir mais cabeçalhos.

Agora vamos conferir se temos o cabeçalho correto na resposta. No terminal:

```
curl -I http://localhost:3000/saldo
# ...
# Content-Type: application/json; charset=utf-8
# ...
```

Legal! Conseguimos indicar para clientes do nosso serviço que o tipo do conteúdo é mesmo um JSON. Agora podemos limpar um pouco o conteúdo dentro de `defroutes` extraindo uma função para o conteúdo da rota `GET /saldo`. Com o capítulo chegando ao fim, recapitulemos como nosso `handler.clj` fica:


```

(ns financeiro.handler
  (:require [compojure.core :refer :all]
             [compojure.route :as route]
             [cheshire.core :as json]
             [ring.middleware.defaults :refer [wrap-defaults
                                              site-defaults]]))

(defn saldo-como-json []
  {:headers {"Content-Type" "application/json; charset=utf-8"}
   :body (json/generate-string {:saldo 0})})

(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  (GET "/saldo" [] (saldo-como-json))
  (route/not-found "Recurso não encontrado"))

(def app
  (wrap-defaults app-routes site-defaults))

```

11.4 CONCLUSÃO

Aqui concluímos mais uma etapa na construção do nosso serviço. Este capítulo nos introduziu aos testes de aceitação com o Midje, com o qual aprendemos mais um pouco sobre como configurar projetos em Clojure separando tipos diferentes de testes em pastas diferentes. Também vimos como gerar dados no formato JSON, inclusive indicando no cabeçalho da resposta HTTP que o tipo do dado é mesmo JSON.

No próximo capítulo nós vamos colocar um pouco mais de funcionalidade neste serviço. Criaremos receitas e despesas, com novos *endpoints* para nossa API. E, claro, como vamos criar transações, haverá um impacto no saldo e isso significa que teremos que voltar ao código que concluímos aqui neste capítulo.

Nos vemos nas próximas páginas!

UMA API MAIS RICA COM TRANSAÇÕES

No capítulo anterior, vimos como nossa API foi ganhando mais corpo. Construímos nossa primeira rota, que até aqui ainda é bem simples. Colocamos um suporte bem bacana de testes. Por fim, demos um acabamento com a definição do cabeçalho de resposta.

Neste capítulo, vamos incrementar esta API: vamos permitir a criação de transações, com testes de aceitação que validem o comportamento da API, inclusive para lidar com requisições inválidas.

12.1 CRIANDO TRANSAÇÕES

Para que nossa API de gerenciamento de finanças pessoais faça algo útil, é preciso que ela permita a inserção de dados. No nosso domínio, serão transações de despesa ou receita. E, claro, o saldo deve mudar sempre que incluirmos uma nova transação (com valor diferente de zero). Acho que esta última frase explica bem o que pode ser um teste de aceitação legal: ao incluirmos uma receita de 10, devemos ter um novo saldo igual a 10. Que tal começarmos, então, por este teste de aceitação? Vamos precisar fazer o seguinte:

- submeter uma transação de tipo :receita ;
- consultar a rota /saldo ; e
- assegurar que o valor é 10 .

Vamos precisar alterar o arquivo `saldo_aceitacao_test.clj` , de modo que ele fique assim:

```
(ns financeiro.saldo-aceitacao-test
  (:require [midje.sweet :refer :all]
            [cheshire.core :as json]
            [financeiro.auxiliares :refer :all]
            ;; inclusão da dependência que faz a requisição HTTP
            [clj-http.client :as http]))

(against-background [(before :facts (iniciar-servidor
                                     porta-padrao))
                    (after :facts (parar-servidor))])

(fact "O saldo inicial é 0" :aceitacao
  (json/parse-string (conteudo "/saldo") true) => {:saldo 0})

;; o novo fato
(fact
  "O saldo é 10 quando a única transação é uma receita de 10"
  :aceitacao

  ;; aqui fazemos uma requisição do tipo POST
  ;; para a rota /transacoes
  ;; com um JSON {"valor" 10, "tipo" "receita"}
  (http/post (endereco-para "/transacoes")
    {:body (json/generate-string {:valor 10 :tipo "receita"})}))

;; e esperamos que o saldo seja igual a 10
(json/parse-string (conteudo "/saldo") true) => {:saldo 10}))
```

No início do arquivo, incluímos a dependência à biblioteca que faz requisições HTTP. Mais para o final, incluímos um novo fato, no começo do qual fazemos uso da função `http/post` para criar uma requisição do tipo `POST` . Esta requisição vai com um JSON com os seguintes dados: `{"valor" 10, "tipo" "receita"}` e é feita para a rota `/transacoes` , que não existe ainda. Vejamos o

que o Midje nos diz:

```
lein midje :filters aceitacao
# clojure.lang.ExceptionInfo: clj-http: status 403
# body: "<h1>Invalid anti-forgery token</h1>"
```

Ops! Recebemos uma mensagem de erro com o status 403, que quer dizer "acesso negado". Olhando um pouco mais para a frente na pilha de erro, vemos uma mensagem que diz `Invalid anti-forgery token`. Isso é uma proteção que frameworks para Web criam para prevenir um tipo de ataque chamado *Cross-Site Request Forgery*, CSRF. Mas esta prevenção só faz sentido quando falamos de formulários para páginas Web. O Compojure cria esta proteção por padrão quando criamos um projeto seguindo seu template. Vejamos de onde vem esta configuração, dentro do arquivo `handler.clj`:

```
(ns financeiro.handler
  (:require [compojure.core :refer :all]
            [compojure.route :as route]
            [cheshire.core :as json]
            [ring.middleware.defaults :refer [wrap-defaults
                                              site-defaults]]))

(defn saldo-como-json []
  {:headers {"Content-Type" "application/json; charset=utf-8"}
   :body (json/generate-string {:saldo 0})})

(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  (GET "/saldo" [] (saldo-como-json))
  (route/not-found "Recurso não encontrado"))

(def app
  ;; aqui, o site-defaults vvvvvvvvvvvvvv
  (wrap-defaults app-routes site-defaults))
```

Além da proteção a CSRF, várias outras configurações são feitas, sendo muitas desnecessárias para nós. Para nossa alegria,

podemos substituir `site-defaults` por `api-defaults`, que também é uma configuração provida pelo Compojure:

```
(ns financeiro.handler
  (:require [compojure.core :refer :all]
            [compojure.route :as route]
            [cheshire.core :as json]
            [ring.middleware.defaults :refer [wrap-defaults
                                              api-defaults]]))

;; mudamos a referência na importação aqui      ^^^^^^^^^^^^^^^

(defn saldo-como-json []
  {:headers {"Content-Type" "application/json; charset=utf-8"}
   :body (json/generate-string {:saldo 0})})

(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  (GET "/saldo" [] (saldo-como-json))
  (route/not-found "Recurso não encontrado"))

(def app
  ;; e também mudamos aqui      vvvvvvvvvvvvvv
  (wrap-defaults app-routes api-defaults))
```

Depois que atualizamos o `handler.clj` para utilizar `api-defaults`, este erro some:

```
lein midje :filters aceitacao
# clojure.lang.ExceptionInfo: clj-http: status 404
# body: "Recurso não encontrado"
```

Agora temos outro erro: a rota `/transacoes` não existe! Precisamos criar esta nova rota e, por enquanto, vamos fazer com que ela não faça nada. Em `handler.clj`:

```
(ns financeiro.handler
  (:require [compojure.core :refer :all]
            [compojure.route :as route]
            [cheshire.core :as json]
            [ring.middleware.defaults :refer [wrap-defaults
                                              api-defaults]]))
```

```

(defn saldo-como-json []
  {:headers {"Content-Type" "application/json; charset=utf-8"}
   :body (json/generate-string {:saldo 0})})

(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  (GET "/saldo" [] (saldo-como-json))
  ;; a nova rota
  (POST "/transacoes" [] {}))
(route/not-found "Recurso não encontrado"))

(def app
  (wrap-defaults app-routes api-defaults))

```

Não temos mais um erro, já que a rota é encontrada. Mas o teste ainda está falhando. O Midje nos diz que o esperado é que o saldo seja 10, mas nossa API retorna 0. Vamos precisar guardar em algum lugar a transação que foi criada para que uma consulta ao saldo encontre o valor apropriado.

O que me vem em mente é abstrair o gerenciamento dos dados para um outro namespace. Neste namespace podemos ter uma função que guarda a transação recém-criada e uma outra para expor o saldo. Não precisamos saber como elas fazem isso agora, pois podemos criar Mocks para os nossos testes unitários. Mas, em breve, precisaremos implementá-las para que nossos testes de aceitação passem. Vamos incluir uns fatos que tratam da criação de transações em `handler_test.clj` :

```

(facts "Registra uma receita no valor de 10"
  ;; cria um mock para a função db/registrar
  (against-background (db/registrar {:valor 10
                                      :tipo "receita"}))
  => {:id 1 :valor 10 :tipo "receita"})

(let [response
      (app (-> (mock/request :post "/transacoes")
               ;; cria o conteúdo do POST
               (mock/json-body {:valor 10

```

```

:tipo "receita"})))]

(fact "o status da resposta é 201"
  (:status response) => 201)

(fact "o texto do corpo é um JSON
      com o conteúdo enviado e um id"
  (:body response) =>
    "{ \"id\":1, \"valor\":10, \"tipo\": \"receita\" }"))

```

O código anterior traz duas novidades: uma é a construção do corpo de uma requisição e a introdução de *mocking*. O `against-background` voltou, e desta vez vamos utilizá-lo para definir um comportamento esperado de uma chamada a uma função, e não seu comportamento real. Neste uso de `against-background`, definimos a função que nos interessa (`db/registrar`), junto com os argumentos que devemos passar, e o resultado esperado (`{:id 1 :valor 10 :tipo "receita"}`).

A outra novidade é a chamada a `mock/json-body`. Requisições `POST` precisam que digamos o conteúdo a ser enviado à API, e a função `mock/json-body` nos ajuda a construir um JSON para o `POST` que o teste faz para `/transacoes`. Além disso, define o `content-type` do cabeçalho da requisição como `application/json`.

Como o namespace `db` não existe, o teste falha. Vamos criá-lo e referenciá-lo no arquivo de teste. Crie um arquivo chamado `db.clj` em `src/financeiro`. Seu conteúdo deve ser o seguinte:

```

(ns financeiro.db)

;; mais uma novidade
(declare registrar)

```

A macro `declare` é mais uma novidade. Ela permite que

indiquemos que há uma função (ou qualquer outra coisa, na real) com este nome, mas sem nenhuma implementação. Isto nos ajuda a deixar os detalhes da implementação para depois.

Precisamos referenciar o namespace db em handler_test.clj :

```
;; precisamos fazer a referência ao namespace db
(ns financeiro.handler-test
  (:require [midje.sweet :refer :all]
            [ring.mock.request :as mock]
            [cheshire.core :as json]
            [financeiro.handler :refer :all]
            [financeiro.db :as db]))
```

Ao rodarmos os testes novamente, encontramos 3 erros:

- um deles é do nosso teste de aceitação, cujo saldo esperado é 10, e ainda não atendemos o requisito;
- outro, em handler_test.clj , que diz que esperamos o código HTTP 201 (utilizado para indicarmos que um recurso foi criado); e
- mais um, também em handler_test.clj , que diz que o corpo da resposta não está conforme o esperado.

Vamos começar a fazer estes testes passarem. Primeiro, referenciamos o namespace db em handler.clj e ajustamos a definição da rota POST /transacoes :

```
(ns financeiro.handler
  (:require [compojure.core :refer :all]
            [compojure.route :as route]
            [cheshire.core :as json]
            [ring.middleware.defaults :refer [wrap-defaults
                                              api-defaults]]

    ;; novo namespace
    [financeiro.db :as db]))
```



```
;; ...
```

```
(POST "/transacoes" requisicao (db/registrar requisicao))
```

```
;; ...
```

Mudamos um pouco a definição da rota: trocamos `[]` por `requisicao`, para ter uma referência aos dados da requisição HTTP. Agora temos uns erros diferentes, e um deles, da execução do teste de aceitação, se deve à chamada a `db/registrar`:

```
java.lang.IllegalStateException: Attempting to call unbound fn:  
#'financeiro.db/registrar
```

Vamos preencher `db/registrar` com um conteúdo simples, para que possamos dar o próximo passo, removendo o `declare` e utilizando `defn`:

```
(defn registrar [transacao]  
  transacao)
```

Mas ainda não temos algo bom o bastante. Os testes unitários seguem falhando porque a chamada à função `db/registrar` não aconteceu como esperado:

```
You never said #'registrar would be called with these arguments:  
[{:protocol "HTTP/1.1", :remote-addr "localhost", :params {}, ..  
.]
```

O que acontece é que estamos passando a requisição completa, do jeito que chegou, para a função `db/registrar`. A requisição vem com cabeçalho (acessível via `:headers`) e corpo (acessível via `:body`) e precisamos apenas do que está no corpo. Assim:

```
;; ...
```

```
(POST "/transacoes" requisicao (db/registrar (:body requisicao)  
))
```

```
;; ...
```

O que ainda não é suficiente, porque o corpo da requisição é passado como uma instância de `java.io.ByteArrayInputStream`, conforme o erro:

```
You never said #'registrar would be called with these arguments:
[#object[java.io.ByteArrayInputStream 0x4e0a2a9e "java.io ...
```

É preciso indicar que o dado que recebemos é um JSON e também interpretar o corpo da requisição como tal.

Recebendo JSON

Para isso, precisamos alterar nossa API para que ela tente empacotar todas as requisições como JSON. Isso pode ser feito através de *wrappers* do Ring e serve tanto para as requisições quanto para as respostas. Vamos começar pelas requisições que a API recebe. Primeiro, devemos incluir a biblioteca `ring-json` em `project.clj`:

```
;; ...
```

```
:dependencies [[org.clojure/clojure "1.10.0"]
               [compojure "1.6.1"]
               [cheshire "5.8.1"]
               [ring/ring-defaults "0.3.2"]
               [ring/ring-json "0.4.0"]
               [clj-http "3.9.1"]]
```

```
;; ...
```

Depois incluímos a referência à biblioteca em `handler.clj`:

```
(ns financeiro.handler
  (:require [compojure.core :refer :all]
            [compojure.route :as route]
            [cheshire.core :as json]
```

```
[ring.middleware.defaults :refer [wrap-defaults
                                   api-defaults]]
;; aqui a gente referencia wrap-json-body
[ring.middleware.json :refer [wrap-json-body]]
[financeiro.db :as db]])

;; ...
```

E, ainda no mesmo arquivo, vamos mudar a definição de `app` de modo que as requisições venham empacotadas como JSON:

```
;; ...

(def app
  (-> (wrap-defaults app-routes api-defaults)
      (wrap-json-body)))
```

Por fim, vamos atualizar nosso teste de aceitação para que ele faça uma requisição com JSON. Em `saldo_aceitacao_test.clj`, o último fato fica assim:

```
(fact "0 saldo é 10 quando a única transação é uma receita de 10"
  :aceitacao

  (http/post (endereco-para "/transacoes")
    {:content-type :json
     :body (json/generate-string {:valor 10
                                   :tipo "receita"})})

  (json/parse-string (conteudo "/saldo") true) => {:saldo 10})
```

Fazendo isto, percebemos que um dos nossos erros muda para algo muito mais compreensível:

```
You never said #'registrar would be called with these arguments:
[{"valor" 10, "tipo" "receita"}]
```

Isso é o Midje nos dizendo que o que foi passado, de fato, para `db/registrar` é diferente do esperado. O próprio Ring cuidou de converter o corpo da requisição para JSON, mas as chaves estão

como string, enquanto nosso teste disse que seriam keywords. Para consertar isso, vamos atualizar `app` novamente:

```
(def app
  (-> (wrap-defaults app-routes api-defaults)
      (wrap-json-body {:keywords? true :bigdecimals? true})))
```

Agora estamos passando os dados da forma apropriada para `db/registrar` e as mensagens de falha dos testes unitários nos guiam para os próximos passos na rota `POST /transacoes`. Ela não retorna nada no corpo da resposta. Perceba que o retorno para `GET /saldo` é bem elaborado, construindo uma resposta com cabeçalho e corpo bem definidos. Precisamos fazer algo parecido. Então vamos extrair a construção do JSON de resposta no `handler.clj`:

```
;; o conteúdo agora é passado como argumento
(defn como-json [conteudo]
  {:headers {"Content-Type" "application/json; charset=utf-8"}
   :body (json/generate-string conteudo)})

(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  ;; o retorno da rota se adaptou
  (GET "/saldo" [] (como-json {:saldo 0}))
  ;; e reaproveitamos a construção de resposta aqui também
  (POST "/transacoes"
        requisicao (-> (db/registrar (:body requisicao))
                      (como-json)))
  (route/not-found "Recurso não encontrado"))
```

`saldo-como-json` virou `como-json`, e agora empacota algum conteúdo como uma resposta usando JSON. E é possível aproveitar para construir a resposta da criação de uma transação. Conseguimos fazer um dos fatos passar, já que o conteúdo é igual ao esperado. Falta ajustar o código de resposta. Vamos incrementar `como-json` para que ela possa incorporar um

código:

```
(defn como-json [conteudo & [status]]
  {:status (or status 200)
   :headers {"Content-Type" "application/json; charset=utf-8"}
   :body (json/generate-string conteudo)})

(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  ;; não diz qual é o código que deve retornar
  (GET "/saldo" [] (como-json {:saldo 0}))
  ;; aplica como-json com o código 201
  (POST "/transacoes"
    requisicao (-> (db/registrar (:body requisicao))
                  (como-json 201)))
  (route/not-found "Recurso não encontrado"))
```

Sucesso com os testes unitários! Agora falta o de aceitação passar. Vamos fazer com que `db/registrar` de fato registre algo para que possa ser consumido, começando pelos seus testes unitários.

Átomos para guardar transações

Vamos criar um arquivo `db_test.clj` em `test/unitarios/financeiro/`:

```
(ns financeiro.db-test
  (:require [midje.sweet :refer :all]
            [financeiro.db :refer :all]))

(facts "Guarda uma transação num átomo"
  (fact "a transação é o primeiro registro"
    (registrar {:valor 7 :tipo "receita"})
    => {:id 1 :valor 7 :tipo "receita"}))
```

O teste que criamos espera que `registrar` retorne a própria transação com um par chave-valor a mais: `:id`. Como é um primeiro registro, o valor deve ser `1`. Vejamos como fazer o teste

passar. Em `db.clj` :

```
(defn registrar [transacao]
  (merge transacao {:id 1}))
```

Apesar de ser o suficiente para o teste passar, é óbvio que é insuficiente para onde queremos chegar: guardar uma lista de transações de modo que possamos calcular o saldo computando todas elas. Mas vamos fazer um passo de cada vez. Podemos verificar agora que uma coleção de transações começa vazia e que passa a ter um elemento quando registramos uma transação. Vamos atualizar o nosso teste unitário com mais este fato:

```
(facts "Guarda uma transação num átomo"
  (fact "a coleção de transações inicia vazia"
    (count (transacoes)) => 0)

  (fact "a transação é o primeiro registro"
    (registrar {:valor 7 :tipo "receita"})
    => {:id 1 :valor 7 :tipo "receita"})
    (count (transacoes)) => 1))
```

E precisamos atualizar nosso código para incluir a função `transacoes` e a manipulação da coleção de transações. Em `db.clj` :

```
(ns financeiro.db)

;; o átomo será usado como banco de dados
(def registros
  (atom []))

(defn transacoes []
  @registros)

;; armazena uma transação no átomo
;; e retorna a transação argumento com a posição dela na coleção
(defn registrar [transacao]
  (let [colecacao-atualizada (swap! registros conj transacao)]
    (merge transacao {:id (count colecacao-atualizada)})))
```

Neste código, criamos uma coleção vazia e a associamos a um átomo. `registrar` atualiza o átomo, incluindo a transação na coleção. O retorno da chamada a `swap!` é guardado em `colecão-atualizada`, uma vez que `swap!` retorna o novo valor do átomo e podemos confiar que `(count coleção-atualizada)` retornará o valor correto, evitando dificuldades por causa de concorrência.

Nossos testes rodam, mas a contagem de elementos da coleção não está de acordo com o esperado. Precisamos limpar a coleção toda vez que os fatos rodarem. `against-background` nos ajuda com isso:

```
(facts "Guarda uma transação num átomo"
  (against-background [(before :facts (limpar))])

  (fact "a coleção de transações inicia vazia"
    (count (transacoes)) => 0)

  (fact "a transação é o primeiro registro"
    (registrar {:valor 7 :tipo "receita"})
    => {:id 1 :valor 7 :tipo "receita"})
    (count (transacoes)) => 1)))
```

E vamos criar uma função para limpar os registros em `db.clj`:

```
(defn limpar []
  (reset! registros []))
```

`reset!` substitui tudo o que estiver dentro do átomo pelo que for passado como argumento. Neste caso, temos um vetor vazio (`[]`).

E, assim, nossos testes unitários passam! Falta agora ter sucesso com o teste de aceitação falho. Ora, o código para a rota `GET /saldo` retorna um valor fixo, `{:saldo 0}`. Precisamos fazer

com que este código consulte o saldo diretamente em `db`. Vamos alterar `handler.clj` primeiro:

```
;; ...

;; cria um mapa com o valor resultante da chamada a db/saldo
(GET "/saldo" []) (como-json {:saldo (db/saldo)}))

;; ...
```

É preciso atualizar o teste unitário `handler_test.clj` primeiro, indicando que haverá uma chamada a um Mock da função `db/saldo`:

```
(facts "Saldo inicial é 0"
  (against-background [(json/generate-string {:saldo 0})
                       => "{\"saldo\":0}"]

    ;; cria mock da chamada a db/saldo
    (db/saldo) => 0])

  (let [response (app (mock/request :get "/saldo"))]
    (fact "o formato é 'application/json'"
      (get-in response [:headers "Content-Type"])
      => "application/json; charset=utf-8")

    (fact "o status da resposta é 200"
      (:status response) => 200)

    (fact
      "o texto do corpo é um JSON cuja chave é saldo e o valor é 0"
      (:body response) => "{\"saldo\":0}"))
```

Vamos criar agora os testes unitários para a função `saldo`, que ainda não existe, em `db_test.clj`:

```
(facts "Calcula o saldo dada uma coleção de transações"
  (against-background [(before :facts (limpar))])
  (fact "saldo é positivo quando só tem receita"
    (registrar {:valor 1 :tipo "receita"})
    (registrar {:valor 10 :tipo "receita"}))
```



```

(registrar {:valor 100 :tipo "receita"})
(registrar {:valor 1000 :tipo "receita"})

(saldo) => 1111)

(fact "saldo é negativo quando só tem despesa"
  (registrar {:valor 2 :tipo "despesa"})
  (registrar {:valor 20 :tipo "despesa"})
  (registrar {:valor 200 :tipo "despesa"})
  (registrar {:valor 2000 :tipo "despesa"}))

(saldo) => -2222)

(fact "saldo é a soma das receitas menos a soma das despesas"
  (registrar {:valor 2 :tipo "despesa"})
  (registrar {:valor 10 :tipo "receita"})
  (registrar {:valor 200 :tipo "despesa"})
  (registrar {:valor 1000 :tipo "receita"}))

(saldo) => 808)))

```

Aqui temos três fatos para validar nossa lógica: a de que o saldo é a soma das receitas menos a soma das despesas. Lembra do código com o qual terminamos o capítulo 7? Que recursivamente calculava o saldo? Vamos precisar dele aqui. Então, nossa implementação para a função `saldo`, em `db.clj`, fica assim:

```

(defn- despesa? [transacao]
  (= (:tipo transacao) "despesa"))

(defn- calcular [acumulado transacao]
  (let [valor (:valor transacao)]
    (if (despesa? transacao)
      (- acumulado valor)
      (+ acumulado valor))))

(defn saldo []
  (reduce calcular 0 @registros))

```

E nossos testes passam! Recapitulando o que este código faz, ele vai acumulando o valor do saldo a cada elemento da coleção. Se

for uma receita, ele adiciona o valor da transação ao valor acumulado. Se for uma despesa, ele reduz.

Acabamos nos antecipando um pouco e cobrindo os casos de lidar com despesas. Vamos criar um teste de aceitação para nos certificarmos que estamos, de fato, no caminho correto: que sabemos calcular o saldo quando há transações tanto do tipo

"despesa" quanto "receita". Em `saldo_aceitacao_test.clj`, crie o seguinte fato:

```
;; ...

(fact "O saldo é 1000 quando criamos duas receitas de 2000
      e uma despesa da 3000" :aceitacao
  (http/post (endereco-para "/transacoes")
    {:content-type :json
     :body (json/generate-string {:valor 2000
                                  :tipo "receita"})}))
  (http/post (endereco-para "/transacoes")
    {:content-type :json
     :body (json/generate-string {:valor 2000
                                  :tipo "receita"})}))
  (http/post (endereco-para "/transacoes")
    {:content-type :json
     :body (json/generate-string {:valor 3000
                                  :tipo "despesa"})}))

  (json/parse-string (conteudo "/saldo") true) => {:saldo 1000})

;; ...
```

Ops! Temos o mesmo problema com a limpeza dos registros antes de rodar os testes. Vamos atualizar o teste de aceitação para que ele os limpe a cada execução:

```
(ns financeiro.saldo-aceitacao-test
  (:require [midje.sweet :refer :all]
            [cheshire.core :as json]
            [financeiro.auxiliares :refer :all])
```

```
;; precisamos incluir a referência ao namespace db
[financeiro.db :as db]
[clj-http.client :as http]))

(against-background [(before :facts [(iniciar-servidor
                                     porta-padrao)
                                     (db/limpar)])]
                    (after :facts (parar-servidor))])
```

Sucesso! Mas até agora só temos lidado com requisições que estão de acordo com o que esperamos e sabemos tratar. Uma API robusta precisa saber lidar com requisições com conteúdo inválido. Imagine que clientes de nossa API enviem requisições sem o valor ou sem o tipo da transação, ou pior: com um tipo de transação diferente de "receita" e "despesa" . O mínimo que devemos fazer é indicar que a requisição é inválida.

Validação de dados (40X para quando falta :valor)

Vejamos que verificações temos que fazer para assegurar que uma transação é válida:

- Existe um valor para a transação e ele é positivo; e
- Existe um tipo de transação e ele é "receita" ou é "despesa" .

Se uma requisição não estiver de acordo com esses requisitos, devemos indicar para clientes da API que o corpo da requisição é inválido, retornando o código HTTP 422 Unprocessable Entity . Podemos fazer estas verificações nos testes de aceitação. Mas, antes de começar a fazer novos fatos, vamos remover as duplicações na criação de requisições em JSON. Vamos criar algumas novas funções em `auxiliares.clj` :

```
;; precisamos incluir a referência ao Cheshire
```

```
(ns financeiro.auxiliares
  (:require [ring.adapter.jetty :refer [run-jetty]]
            [financeiro.handler :refer [app]]
            [cheshire.core :as json]
            [clj-http.client :as http]))

;; ...

;; as novas funções
(defn conteudo-como-json [transacao]
  {:content-type :json
   :body (json/generate-string transacao)
   :throw-exceptions false})

(defn despesa [valor]
  (conteudo-como-json {:valor valor :tipo "despesa"}))

(defn receita [valor]
  (conteudo-como-json {:valor valor :tipo "receita"}))
```

Em `conteudo-como-json`, perceba que há um novo par chave-valor: `:throw-exceptions false`. Quando uma requisição HTTP retorna códigos excepcionais, como os 400 e 500, a biblioteca lança exceções e este não é um comportamento que desejamos. Se for preciso lançar uma exceção, nossos clientes precisarão lidar com isso.

E, em `saldo_aceitacao_test.clj`, os testes de aceitação para `POST /transacoes` ficam assim:

```
(fact "O saldo é 1000 quando criamos duas receitas de 2000
      e uma despesa da 3000" :aceitacao
  (http/post (endereco-para "/transacoes") (receita 2000))
  (http/post (endereco-para "/transacoes") (receita 2000))
  (http/post (endereco-para "/transacoes") (despesa 3000))

  (json/parse-string (conteudo "/saldo") true) => {:saldo 1000})
```

Agora podemos começar os testes de validação:

```

(fact "Rejeita uma transação sem valor" :aceitacao
  (let [resposta (http/post (endereço-para "/transacoes")
    (conteúdo-como-json {:tipo
                          "receita"}))])

  (:status resposta) => 422))

(fact "Rejeita uma transação com valor negativo" :aceitacao
  (let [resposta (http/post (endereço-para "/transacoes")
    (receita -100))])

  (:status resposta) => 422))

(fact "Rejeita uma transação com valor que não é um número"
  :aceitacao

  (let [resposta (http/post (endereço-para "/transacoes")
    (receita "mil"))])

  (:status resposta) => 422))

(fact "Rejeita uma transação sem tipo" :aceitacao
  (let [resposta (http/post (endereço-para "/transacoes")
    (conteúdo-como-json {:valor 70}))])

  (:status resposta) => 422))

(fact "Rejeita uma transação com tipo desconhecido" :aceitacao
  (let [resposta (http/post (endereço-para "/transacoes")
    (conteúdo-como-json
      {:valor 70
       :tipo "investimento"}))])

  (:status resposta) => 422))

```

Todos eles falham, claro, já que não fizemos, ainda, nenhuma validação. Vamos implementar uma função que nos indique se uma transação é válida. Que tal em um namespace separado? Vamos criar `transacoes.clj` e colocar nele a função `valida?`, guiando-nos pelos testes unitários em `transacoes_test.clj`:

```

(ns financeiro.transacoes-test
  (:require [midje.sweet :refer :all]
    [financeiro.transacoes :refer :all]))

(fact "Uma transação sem valor não é válida"

```

```

(valida? {:tipo "receita"}) => false)

(fact "Uma transação com valor negativo não é válida"
  (valida? {:valor -10 :tipo "receita"}) => false)

(fact "Uma transação com valor não numérico não é válida"
  (valida? {:valor "mil" :tipo "receita"}) => false)

(fact "Uma transação sem tipo não é válida"
  (valida? {:valor 90}) => false)

(fact "Uma transação com tipo desconhecido não é válida"
  (valida? {:valor 8 :tipo "investimento"}) => false)

(fact "Uma transação com valor numérico positivo
      e com tipo conhecido é válida"
  (valida? {:valor 230 :tipo "receita"}) => true)

```

E a implementação da verificação, em `transacoes.clj`, fica assim:

```

(ns financeiro.transacoes)

(defn valida? [transacao]
  (and (contains? transacao :valor)
        (number? (:valor transacao))
        (pos? (:valor transacao))
        (contains? transacao :tipo)
        (or (= "despesa" (:tipo transacao))
              (= "receita" (:tipo transacao)))))

```

Cobrimos todos os cenários previstos, e agora podemos fazer com que o `handler` chame esta validação. Caso a requisição seja válida, seguimos com a inclusão da transação nos registros. Caso contrário, retornamos uma resposta indicando que a requisição foi inválida. Vejamos como ficam as rotas em `handler.clj`:

```

(ns financeiro.handler
  (:require [compojure.core :refer :all]
             [compojure.route :as route]
             [cheshire.core :as json]
             [ring.middleware.defaults :refer [wrap-defaults]]

```

```

                                api-defaults]]
[ring.middleware.json :refer [wrap-json-body]]
[financeiro.db :as db]
;; vamos incluir a referência ao namespace transacoes
[financeiro.transacoes :as transacoes]])

;; ...

(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  (GET "/saldo" [] (como-json {:saldo (db/saldo)}))
  (POST "/transacoes" requisicao
    (if (transacoes/valida? (:body requisicao))
      (-> (db/registrar (:body requisicao))
        (como-json 201))
      (como-json {:mensagem "Requisição inválida"} 422)))
  (route/not-found "Recurso não encontrado"))

```

E os nossos testes de aceitação rodam com sucesso!

Vamos iniciar um servidor, criar umas transações e vê-las sendo retornadas pela nossa API? No terminal, rode o comando `lein ring server-headless`, para iniciar a API na porta 3000. Para incluir uma transação, com `cUrl`, execute o seguinte comando:

```

curl -X POST -d '{"valor": 700, "tipo": "despesa"}' \
  -H "Content-Type: application/json" localhost:3000/transacoes

```

Fique à vontade para se certificar de que o saldo é mesmo o que você espera.

12.2 CONCLUSÃO

Conseguimos progredir bastante na nossa API. Conseguimos fazer com que clientes da API consigam cadastrar transações, sejam elas despesas ou receitas. E esta é a base para algo mais complexo: consultas diversas. Veremos no próximo capítulo como

fazer consultas à API, filtrando por campos que as transações podem ter.

Até lá!

EXTRATO DAS TRANSAÇÕES E SEUS FILTROS

O capítulo anterior nos permitiu registrar transações, habilitando nossa API a fornecer mais funcionalidades, já que agora podemos fazer operações sobre os dados. Uma funcionalidade já está no ar: a consulta do saldo. Que tal também habilitarmos a busca por transações de apenas um determinado tipo? Ou de um rótulo ("feira" , "entretenimento" ou "educação" , por exemplo)? Que tal ambas?

Neste capítulo, veremos como criar APIs que lidem com parâmetros de busca (o conteúdo que vem depois de um ponto de interrogação) em uma requisição HTTP, e também veremos como criar um artefato `.jar` para publicação em servidores.

Vamos começar a aprimorar nosso controle financeiro começando pelo filtro de tipo de transação.

13.1 RECEITAS OU DESPESAS?

Tipos de transação, neste livro, são "receita" e "despesa" .

Nossa API poderá receber requisições HTTP GET da seguinte forma:

- /transacoes , para trazer todas as transações, sem filtros;
- /receitas , que traz só as transações cujo elemento :tipo for "receita" ; e
- /despesas , que faz o mesmo que a rota acima, mas considerando "despesa" como tipo.

OPINIÃO DO AUTOR

Poderíamos desenhar nossa API para que só tivéssemos a rota /transacoes e usássemos parâmetros de busca, como /transacoes?tipo=receita e /transacoes?tipo=despesa . Penso que a experiência de quem vai consumir a API conta bastante, e por isso acho muito válido dedicar uma rota para cada filtro. Outro motivo pelo qual esta *promoção* é válida é que, no domínio deste livro, não há nenhum tipo de transação além destes dois.

O mesmo não se aplicaria a rótulos de transações. Aqui temos um número muito variável de itens, "educação" , "lazer" , "aluguel" ... Estes são valores que são definidos por quem consome a API. Portanto, vamos disponibilizar o filtro por eles como parâmetros de consulta, como /transacoes?rotulos=lazer . Vamos nos debruçar sobre este tema mais para o final do capítulo.

Como praticamos nos capítulos anteriores, vamos começar

pelos testes de aceitação. Podemos registrar algumas transações de tipos variados e esperar que os cenários nos retornem apropriadamente as transações dos tipos certos. E teremos um cenário para rota não encontrada.

Vamos criar um novo arquivo para os testes de aceitação dos filtros. Dentro da pasta `test/aceitacao/financeiro`, crie um arquivo chamado `filtros_aceitacao_test.clj`. Seu conteúdo inicial será o seguinte:

```
(ns financeiro.filtros-aceitacao-test
  (:require [midje.sweet :refer :all]
            [cheshire.core :as json]
            [financeiro.auxiliares :refer :all]
            [clj-http.client :as http]
            [financeiro.db :as db]))

(against-background [(before :facts
                           [(iniciar-servidor porta-padrao)
                            (db/limpar)])
                    (after :facts (parar-servidor))])
```

As referências a bibliotecas e outras funções do programa são iguais às que encontramos em `saldo_aceitacao_test.clj`. O mesmo vale para o `against-background`.

Vamos criar nossos primeiros cenários:

```
(against-background [(before :facts
                           [(iniciar-servidor porta-padrao)
                            (db/limpar)])
                    (after :facts (parar-servidor))])

(fact "Não existem receitas" :aceitacao
  (json/parse-string (conteudo "/receitas") true)
  => {:transacoes '()})

(fact "Não existem despesas" :aceitacao
  (json/parse-string (conteudo "/despesas") true)
  => {:transacoes '()})
```

```
(fact "Não existem transacoes" :aceitacao
  (json/parse-string (conteudo "/transacoes") true)
  => {:transacoes '()})
```

Se rodarmos os testes agora com `lein midje`, veremos que ambos falham porque a rota não foi encontrada. Claro, não indicamos para a nossa aplicação que vamos ter a opção de filtrar transações. Antes de alterarmos as definições de rota em `handler.clj`, vamos criar mais uns cenários que nos certifiquem de que concluímos a nossa funcionalidade de filtrar transações por tipo.

Vejamos uma nova versão do teste de aceitação:

```
;; algumas transações para os nossos testes
(def transacoes-aleatorias
  '({:valor 7.0M :tipo "despesa"}
    {:valor 88.0M :tipo "despesa"}
    {:valor 106.0M :tipo "despesa"}
    {:valor 8000.0M :tipo "receita"}))

(against-background [(before :facts
                           [(iniciar-servidor porta-padrao)
                            (db/limpar)])
                    (after :facts (parar-servidor))])

(fact "Não existem receitas" :aceitacao
  (json/parse-string (conteudo "/receitas") true)
  => {:transacoes '()})

(fact "Não existem despesas" :aceitacao
  (json/parse-string (conteudo "/despesas") true)
  => {:transacoes '()})

(fact "Não existem transações" :aceitacao
  (json/parse-string (conteudo "/transacoes") true)
  => {:transacoes '()})

;; antes destes fatos começarem, limpamos os registros e
;; cadastramos algumas transações
```

```
;; quando acabarem, apagamos todas as transações
(against-background
  [(before :facts (doseq [transacao transacoes-aleatorias]
    ;; ^^^^^ este doseq é novidade!
    (db/registrar transacao)))
   (after :facts (db/limpar))])

(fact "Existem 3 despesas" :aceitacao
  (count (:transacoes (json/parse-string
    (conteudo "/despesas") true))) => 3)

(fact "Existe 1 receita" :aceitacao
  (count (:transacoes (json/parse-string
    (conteudo "/receitas") true))) => 1)

(fact "Existem 4 transações" :aceitacao
  (count (:transacoes (json/parse-string
    (conteudo "/transacoes") true))) => 4)))
```

Nesta versão, criamos algumas transações para a configuração dos nossos cenários de teste. Depois, incluímos mais um `against-background` para incorporar três cenários, que precisam ter registros cadastrados antes de começarem. Perceba que há uma novidade: o uso da macro `doseq`, que permite varrer uma coleção para executar algum procedimento. Aqui, varremos a coleção `transacoes-aleatorias`.

Para cada iteração nesta varredura, o elemento em questão recebe o nome `transacao` e fica disponível para um ou mais procedimentos dentro de `doseq`. No nosso caso, apenas guardamos a transação. Quando utilizamos `doseq`, normalmente esperamos que aconteça algum efeito colateral e não recebamos uma lista de volta. Além disso, limpamos os registros depois que estes 3 cenários rodarem.

Por fim, temos 3 fatos: um que verifica que temos 3 transações do tipo `"despesa"`, outro que verifica que só há uma transação

do tipo "receita" , e mais um que assegura que há 4 transações em /transacoes .

Transações por tipo

Vamos criar a rota, então? Começando pelos testes unitários, vamos alterar o arquivo `handler_test.clj` para incluir os seguintes fatos:

```
(facts "Existe rota para lidar com filtro de transação por tipo"
  (against-background
    [(db/transacoes-do-tipo "receita")
     => '({:id 1 :valor 2000 :tipo "receita"})]
    (db/transacoes-do-tipo "despesa")
     => '({:id 2 :valor 89 :tipo "despesa"})]
    (db/transacoes)
     => '({:id 1 :valor 2000 :tipo "receita"}
          {:id 2 :valor 89 :tipo "despesa"})])
  (fact "Filtro por receita"
    (let [response (app (mock/request :get "/receitas"))]
      (:status response) => 200
      (:body response) => (json/generate-string
                           { :transacoes '({:id 1 :valor 2000 :tipo "receita"}) })]))

  (fact "Filtro por despesa"
    (let [response (app (mock/request :get "/despesas"))]
      (:status response) => 200
      (:body response) => (json/generate-string
                           { :transacoes '({:id 2 :valor 89 :tipo "despesa"}) })]))

  (fact "Sem filtro"
    (let [response (app (mock/request :get "/transacoes"))]
      (:status response) => 200
      (:body response) => (json/generate-string
                           { :transacoes '({:id 1 :valor 2000 :tipo "receita"}
                                             {:id 2 :valor 89 :tipo "despesa"}) }) ))))
```

Nossos testes unitários nem compilam porque não há uma função chamada `transacoes-do-tipo` em `db.clj` . Vamos

criar uma que retorne apenas uma lista vazia. Em `db.clj` , inclua o seguinte:

```
(defn transacoes-do-tipo [tipo]
  '())
```

Agora os testes compilam e falham, como esperado, porque as rotas não existem. Vamos criá-las. Em `handler.clj` , vamos editar a definição das rotas em `defroutes` :

```
(defroutes app-routes
  (GET "/" [] "Olá, mundo!")
  (GET "/saldo" [] (como-json {:saldo (db/saldo)}))
  (POST "/transacoes" requisicao
    (if (transacoes/valida? (:body requisicao))
      (-> (db/registrar (:body requisicao))
        (como-json 201))
      (como-json {:mensagem "Requisição inválida"} 422)))

;; novas rotas
(GET "/transacoes" []
  (como-json {:transacoes (db/transacoes)}))
(GET "/receitas" []
  (como-json {:transacoes (db/transacoes-do-tipo "receita")}))
(GET "/despesas" []
  (como-json {:transacoes (db/transacoes-do-tipo "despesa")}))
(route/not-found "Recurso não encontrado"))
```

Agora nossos testes unitários passam e os de aceitação falham. Isto acontece porque não implementamos a função `db/transacoes-do-tipo` . Vamos implementá-la! Em `db_test.clj` , inclua os seguintes fatos, responsáveis por verificar que os filtros são aplicados conforme esperamos:

```
(facts "filtra transações por tipo"
  (def transacoes-aleatorias '({:valor 2 :tipo "despesa"}
                                {:valor 10 :tipo "receita"}
                                {:valor 200 :tipo "despesa"}
                                {:valor 1000 :tipo "receita"}))

  (against-background [(before :facts
```

```

[ (limpar)
  (doseq [transacao transacoes-aleatorias]
    (registrar transacao)))]

(fact "encontra apenas as receitas"
  (transacoes-do-tipo "receita")
  => '({:valor 10 :tipo "receita"}
      {:valor 1000 :tipo "receita"}))

(fact "encontra apenas as despesas"
  (transacoes-do-tipo "despesa")
  => '({:valor 2 :tipo "despesa"}
      {:valor 200 :tipo "despesa"})))

```

Agora, em `db.clj`, vamos implementar o filtro por tipo:

```

(defn transacoes-do-tipo [tipo]
  (filter #(= tipo (:tipo %)) (transacoes)))

```

Aqui temos a aplicação da função `filter`, que recebe como argumento uma função anônima e a aplica sobre as transações cadastradas. Esta função anônima compara o `:tipo` da transação com o `tipo` fornecido como filtro. O resultado de `filter`, então, vai ser uma lista que contém apenas os elementos de `(transacoes)` que retornarem `true` na aplicação da função anônima.

Ao rodarmos nossos testes, temos o sucesso da suíte de testes de aceitação! Para ver tudo funcionando, rode mais uma vez o comando `lein ring server-headless` para subir um servidor na porta 3000.

Jar autossuficiente

Enquanto estamos desenvolvendo, é confortável e prático rodar o Leiningen para subir um servidor. Mas o Leiningen é uma ferramenta para auxílio do desenvolvimento e não a encontramos

nos servidores em que nossa API vai rodar. Para ter a API no servidor de produção, por exemplo, precisamos enviar uma versão empacotada dela. No mundo Java, temos os arquivos `.jar`, `.war` e `.ear`. Outras linguagens têm seus artefatos também, como os pacotes `npm`, `gems` do Ruby, `eggs` do Python...

Em Clojure, criamos arquivos `.jar` que são autossuficientes: o artefato inclui todas as dependências necessárias para que a API rode em um servidor. Os arquivos `.jar` autossuficientes são conhecidos como *fat jars*, ou *jars gordos*. Eu prefiro alternativas, como autossuficientes, autocontidos ou autônomos. Os `jars` autossuficientes podem ser gerados nos nossos computadores enquanto desenvolvemos, mas o mais recomendado é que sejam criados dentro de um processo de integração contínua, e que este artefato nunca mais seja modificado, até que chegue em produção um dia.

Para gerar nosso `jar` autossuficiente, executamos o seguinte comando:

```
lein ring uberjar
```

Você verá que os arquivos `financeiro-0.1.0-SNAPSHOT.jar` e `financeiro-0.1.0-SNAPSHOT-standalone.jar` foram criados na pasta `target`. O nosso `jar` autossuficiente é o que contém `standalone` no nome. O nome do arquivo é composto pelo nome do projeto e um número de versão que segue o formato descrito no modelo de versionamento semântico (<https://semver.org/lang/pt-BR/>). Neste caso, a versão é `0.1.0-SNAPSHOT`, conforme podemos encontrar descrito na primeira linha do arquivo `project.clj`.

Podemos alterar o nome do arquivo criado configurando o

arquivo `project.clj` para ter a seguinte definição:

```
(defproject financeiro "0.1.0-SNAPSHOT"
  ;; ...

  :uberjar-name "financeiro.jar"
  ;; ^^ configura o nome do jar que será criado

  ;; ...
  :test-paths ["test/unitarios" "test/aceitacao"])
```

Para gerar um novo arquivo, desta vez com o nome desejado, vamos limpar o que já existe na pasta `target` e executar novamente o comando de criação do `jar` :

```
lein clean # apaga o conteúdo da pasta target
lein ring uberjar
```

Dessa vez, a pasta `target` inclui o arquivo `financeiro.jar` .

Agora que temos o artefato autossuficiente, vamos rodá-lo. Para isso, execute o seguinte comando na raiz do projeto:

```
java -jar target/financeiro.jar
```

Como resultado, temos nossa API rodando na porta 3000. Para validar as funcionalidades da API, vamos incluir algumas transações com o `cUrl` :

```
curl -X POST -d '{"valor": 20, "tipo": "despesa"}' \
-H "Content-Type: application/json" localhost:3000/transacoes
curl -X POST -d '{"valor": 700, "tipo": "despesa"}' \
-H "Content-Type: application/json" localhost:3000/transacoes
curl -X POST -d '{"valor": 3000, "tipo": "receita"}' \
-H "Content-Type: application/json" localhost:3000/transacoes
```

Com estes comandos, indicamos para o `cUrl` que vamos fazer uma requisição do tipo `POST` com o `-X POST` ; com os dados sendo fornecidos com o argumento `-d` , em `-d '{"valor":`

3000, "tipo": "receita"}}' ; com o cabeçalho indicando que o tipo de conteúdo é JSON (-H "Content-Type: application/json"); e a URL é localhost:3000/transacoes . Então teremos 3 transações registradas, e as rotas /transacoes , /receitas e /despesas devem funcionar como esperado.

Filtro por rótulos

Já implementamos o primeiro incremento da API que nos propusemos a fazer no começo do capítulo. Agora podemos partir para o próximo, que é filtrar transações por um determinado rótulo. Para esta funcionalidade, nossa rota pode ser assim:

```
/transacoes?rotulo=jogo
```

Podemos pensar na possibilidade de filtrar por múltiplos rótulos ao mesmo tempo. Talvez você queira listar as despesas com a viagem para a Paraíba, mais precisamente as cidades de João Pessoa e Cabaceiras, a Hollywood nordestina. Nesse caso, poderíamos buscar por transações que tenham rótulos "joao pessoa" ou "cabaceiras" . Segundo a RFC 6570 (<https://tools.ietf.org/html/rfc6570/>), na seção 3.2.8, podemos escolher uma das seguintes formas:

```
/transacoes?rotulos=joao%20pessoa,cabeceiras  
/transacoes?rotulos=joao%20pessoa&rotulos=cabeceiras
```

Vale lembrar que %20 é um espaço em branco. É o que se utiliza na composição de URLs em vez do espaço em si.

A primeira opção tem uma legibilidade legal, mas exige uma

pequena complexidade para clientes comporem a URL, uma vez que é preciso avaliar os parâmetros já fornecidos para decidir se precisa, ou não, colocar rotulos ou já anexar direto com , . A segunda parece oferecer menos complexidade, tendo em vista que basta sempre anexar rotulos=...& . Vejamos como o Compojure (aquela biblioteca que nos ajuda a lidar com rotas) pode nos ajudar na escolha.

Fazendo um experimento rápido, vamos colocar uma rota GET no handler.clj :

```
(defroutes app-routes
  ;; ...
  (GET "/"q" {params :params} (str params))
  ;;      ^^^^^^^^^^^^^^^^^^^^^ aqui são guardados os parâmetros
  ;;                               utilizados na busca
)
```

Se consultarmos a URL `http://localhost:3000/q?rotulos=joao%20pessoa,cabaceiras` , o resultado é o seguinte:

```
{:rotulos "joao pessoa,cabaceiras"}
```

Precisaremos quebrar a string a cada vírgula para separar os rótulos.

Por um outro lado, consultando a URL `http://localhost:3000/q?rotulos=joao%20pessoa&rotulos=cabaceiras` , o resultado é:

```
{:rotulos ["joao pessoa" "cabaceiras"]}
```

O retorno é um array com cada rótulo separado devidamente. Então vamos com esta opção. E, claro, precisamos apagar a rota /q que acabamos de criar.

Vamos começar pela edição do teste de aceitação de filtros para incluir a nova rota. Em `filtros_aceitacao_test.clj` :

```
;; vamos colocar alguns rótulos nas transações de teste
(def transacoes-aleatorias
  '({:valor 7.0M :tipo "despesa"
     :rotulos ["sorvete" "entretenimento"]}
    {:valor 88.0M :tipo "despesa"
     :rotulos ["livro" "educação"]}
    {:valor 106.0M :tipo "despesa"
     :rotulos ["curso" "educação"]}
    {:valor 8000.0M :tipo "receita"
     :rotulos ["salário"]}))

;; ...

(against-background
  [(before :facts [(db/limpar)
                   (doseq [transacao transacoes-aleatorias]
                     (db/registrar transacao)))]
   (after :facts (db/limpar))])

(fact "Existem 3 despesas" :aceitacao
  (count (:transacoes (json/parse-string
                       (conteudo "/despesas") true))) => 3)

(fact "Existe 1 receita" :aceitacao
  (count (:transacoes (json/parse-string
                       (conteudo "/receitas") true))) => 1)

(fact "Existem 4 transações" :aceitacao
  (count (:transacoes (json/parse-string
                       (conteudo "/transacoes") true))) => 4)

;; aqui começam os novos cenários

(fact "Existe 1 receita com rótulo 'salário'"
  (count (:transacoes (json/parse-string
                       (conteudo "/transacoes?rotulos=salário") true))) => 1)

(fact "Existem 2 despesas com rótulo 'livro' ou 'curso'"
  (count (:transacoes (json/parse-string
                       (conteudo "/transacoes?rotulos=livro&rotulos=curso")
                       true))) => 2)
```

```
(fact "Existem 2 despesas com rótulo 'educação'"
  (count (:transacoes (json/parse-string
    (conteudo "/transacoes?rotulos=educação") true))) => 2)))
```

Quando rodamos os testes, eles falham, todos dizendo que o resultado foi 4 , diferente do esperado. Quando eles passarem, teremos mais confiança de que concluímos a funcionalidade.

Agora vamos implementar os testes unitários para introduzir o uso dos parâmetros de busca. Em `handler_test.clj` :

```
(facts "Filtra transações por parâmetros de busca na URL"
  (def livro {:id 1 :valor 88 :tipo "despesa"
    :rotulos ["livro" "educação"]})
  (def curso {:id 2 :valor 106 :tipo "despesa"
    :rotulos ["curso" "educação"]})
  (def salario {:id 3 :valor 8000 :tipo "receita"
    :rotulos ["salário"]})

  (against-background
    [(db/transacoes-com-filtro {:rotulos ["livro" "curso"]})
     => [livro curso]
     (db/transacoes-com-filtro {:rotulos "salário"})
     => [salario]]
    (fact "Filtro múltiplos rótulos"
      (let [response (app (mock/request
        :get "/transacoes?rotulos=livro&rotulos=curso"))]
        (:status response) => 200
        (:body response) => (json/generate-string
          {:transacoes [livro curso]})))))

    (fact "Filtro com único rótulo"
      (let [response (app (mock/request
        :get "/transacoes?rotulos=salário"))]
        (:status response) => 200
        (:body response) => (json/generate-string
          {:transacoes [salario]})))))
```

`db.clj` precisa de pelo menos um esqueleto de `transacoes-com-filtro` para que nossos testes compilem:

```
(defn transacoes-com-filtro [filtros]
  '())
```

Agora nossos testes rodam e indicam que o resultado encontrado (todas as transações) é diferente do esperado. Conforme consta na rota `/transacoes`, todas elas são retornadas mesmo. O que precisamos é avaliar se há filtro para ser aplicado e, se sim, aplicá-lo. Vamos fazer com que `handler.clj` aplique a função `transacoes-com-filtro` na rota `/transacoes`:

```
(defroutes app-routes
  ;; ...
  (GET "/transacoes" {filtros :params}
    (como-json {:transacoes
      (if (empty? filtros)
        (db/transacoes)
        (db/transacoes-com-filtro filtros))}))
  ;; ...
)
```

Os testes para o handler agora são bem-sucedidos. Do ponto de vista de saber lidar com rotas, tudo certo. Falta saber filtrar, e é por isso que os testes de aceitação falham. Vamos implementar a filtragem. Em `db_test.clj`:

```
;; ...

(facts "filtra transações por rótulo"
  (def transacoes-aleatorias
    '({:valor 7.0M :tipo "despesa"
      :rotulos ["sorvete" "entretenimento"]}
      {:valor 88.0M :tipo "despesa"
      :rotulos ["livro" "educação"]}
      {:valor 106.0M :tipo "despesa"
      :rotulos ["curso" "educação"]}
      {:valor 8000.0M :tipo "receita"
      :rotulos ["salário"]}))

  (against-background
    [(before :facts [(limpar)
```

```

(doseq [transacao transacoes-aleatorias]
  (registrar transacao)))]
(fact "encontra a transação com rótulo 'salário'"
  (transacoes-com-filtro {:rotulos "salário"})
  => '({:valor 8000.0M :tipo "receita"
        :rotulos ["salário"]}))

(fact "encontra as 2 transações com rótulo 'educação'"
  (transacoes-com-filtro {:rotulos ["educação"]})
  => '({:valor 88.0M :tipo "despesa"
        :rotulos ["livro" "educação"]}
      {:valor 106.0M :tipo "despesa"
        :rotulos ["curso" "educação"]}))

(fact "encontra as 2 transações com rótulo 'livro' ou 'curso'"
  (transacoes-com-filtro {:rotulos ["livro" "curso"]})
  => '({:valor 88.0M :tipo "despesa"
        :rotulos ["livro" "educação"]}
      {:valor 106.0M :tipo "despesa"
        :rotulos ["curso" "educação"]}))

```

E, em db.clj :

```

(defn transacoes-com-filtro [filtros]
  (let [rotulos (->> (:rotulos filtros)
    ;;      ^^^ a macro thread-last está de volta
    (conj [])
    (flatten)
    (set)))]
    (filter #(some rotulos (:rotulos %)) (transacoes))))

```

Todos os nossos testes são bem-sucedidos! Vejamos o que o código de filtragem faz. Perceba que utilizamos a macro `thread-last : ->>` . Para lembrar, o que ela faz é pegar o resultado de cada aplicação de função e passá-lo como último argumento para cada função seguinte.

Primeiro, pegamos o conteúdo do mapa cuja chave é `:rotulos` , imaginando que os rótulos que aplicamos são `"livro"` e `"curso"` :


```
(:rotulos filtros)
;; ["livro" "curso"]
```

E o resultado é passado como último argumento para `(conj [])`. Se o resultado de `(:rotulos filtros)` for `["salário"]`, é como se acontecesse o seguinte:

```
(conj [] ["livro" "curso"])
```

Como `conj` já está recebendo um argumento, `[]`, `["livro" "curso"]` deve ser passado como último argumento. É por isso que utilizamos `->>` neste caso, em vez de `->`, que passaria `["livro" "curso"]` como primeiro argumento. O resultado em si teria pouca diferença; só a ordem dos elementos mudaria. E por que usamos `(conj [])`? Esta função será um tanto mais robusta se ela souber lidar com `(:rotulos filtros)` retornando tanto um array quanto uma única string, que é o caso de haver só um rótulo. No caso de vir apenas uma string, com `(conj [])`, colocamos a string dentro de um array; no caso de já vir uma coleção, continuamos com um array. Neste último cenário, no entanto, ficamos com um array dentro de outro, e por isso o uso de `(flatten)`:

```
(:rotulos filtros)
;; ["livro" "curso"]

(conj [] ["livro" "curso"])
;; [["livro" "curso"]] <- Um array dentro de outro

(flatten [["livro" "curso"]])
;; ["livro" "curso"] <- flatten simplificou o array
```

O que `(flatten)` faz é colocar num mesmo nível todos os elementos de uma coleção. Por exemplo:

```
(def feira [[:banana :abacaxi]
             [:alface :cenoura :beterraba :rabanete]
```

```

[:chocolate [:caixa :amargo :organico]]])

(flatten feira)
;; (:banana :abacaxi :alface :cenoura :beterraba :rabanete
;; :chocolate :caixa :amargo :organico)

```

Perceba que temos arrays dentro de arrays em `feira`. Quando aplicamos `flatten` a `feira`, temos uma lista onde todos os elementos estão no mesmo nível. Depois transformamos esta lista em um conjunto, removendo quaisquer elementos duplicados:

```

(set ["livro" "curso"])
;; #{"livro" "curso"}

```

Logo veremos outro motivo especial para transformar o resultado de `flatten` em um conjunto.

Agora que separamos quais os critérios para filtragem, podemos usar a função `filter` para nos trazer as transações que nos interessam. `filter` espera uma função que diga como verificar se pega ou não pega um elemento. No nosso caso, queremos saber se algum elemento da coleção de rótulos que veio dos filtros está na coleção de rótulos de uma transação. Se tiver, `filter` traz esta transação. Se não tiver, `filter` a descarta.

A função que verifica se uma coleção contém itens de uma outra é `some`. Podemos usar `some` para verificar se algum elemento de uma coleção cumpre um determinado requisito. Por exemplo, livros por até R\$ 20,00:

```

(def livros [{:titulo "Poemas dos Becos de Goiás"
               :por "Cora Coralina" :preco 32.99M}
             {:titulo "Morte e Vida Severina"
               :por "João Cabral de Melo Neto" :preco 33.9M}
             {:titulo "A Hora da Estrela"
               :por "Clarice Lispector" :preco 13.9M}

```

```

{:titulo "Eu e Outras Poesias"
 :por "Augusto dos Anjos" :preco 14.9M}})

(some #(< (:preco %) 20) livros)
;; true

(some #(< (:preco %) 10) livros)
;; nil

```

Aqui, primeiro declaramos uma coleção de livros cujos preços podem ser vistos através da chave `:preco`. Depois verificamos se existe algum livro com preço menor que 20, e há algum. Em seguida, se há algum com preço menor que 10, e não há nenhum.

Para comparar coleções, teríamos que varrer uma das coleções para ver se o item da coleção A é igual a algum item da coleção B.

`some` já faz isso para nós, se o primeiro argumento que passamos para ela for um conjunto. Por isso, vale a pena convertermos o resultado do `flatten` em um conjunto. Então, podemos comparar coleções como no exemplo a seguir:

```

(def frutas-essenciais #{:banana :abacaxi :laranja})

(def feira [:banana :abacaxi :alface :cenoura :beterraba
            :rabanete :chocolate :caixa :amargo :organico])

(some frutas-essenciais feira)
;; :banana

```

O comportamento de `some` é retornar o primeiro resultado verdadeiro. Vale lembrar que, em Clojure, `true` e `:banana` podem ser considerados verdadeiros.

Voltando para o nosso domínio, o uso de `filter` ficou da seguinte forma:

```

(filter #(some rotulos (:rotulos %)) (transacoes))
;;          ^^^^^^ o conjunto que criamos no let

```

Já vimos como o `some` funciona. Temos, então, uma função anônima que usa `some` pegando os rótulos de cada elemento resultante de `(transacoes)`. Se `some` retornar algo verdadeiro (neste caso, alguma string que corresponda a um rótulo encontrado), `filter` vai guardar o elemento em questão da coleção de transações.

Vamos ver tudo funcionando com a API no ar? Suba um servidor:

```
lein ring server-headless
```

E use os seguintes comandos, com `curl`, para criar dados de exemplo:

```
curl -X POST -d \
  '{"valor": 2700, "tipo": "despesa", "rotulos": ["computador"]}' \
  -H "Content-Type: application/json" localhost:3000/transacoes
curl -X POST -d \
  '{"valor": 9200, "tipo": "despesa", "rotulos": ["salário"]}' \
  -H "Content-Type: application/json" localhost:3000/transacoes
curl -X POST -d \
  '{"valor":20,"tipo":"despesa","rotulos":["livro","educação"]}' \
  -H "Content-Type: application/json" localhost:3000/transacoes
curl -X POST -d \
  '{"valor":40,"tipo":"despesa","rotulos":["livro","educação"]}' \
  -H "Content-Type: application/json" localhost:3000/transacoes
curl -X POST -d \
  '{"valor":400,"tipo":"despesa","rotulos":["curso","educação"]}' \
  -H "Content-Type: application/json" localhost:3000/transacoes
```

Para ver as transações:

```
curl localhost:3000/transacoes
# {"transacoes":[{"valor":2700,"tipo":"despesa","rotulos":...
```

```
curl "localhost:3000/transacoes?rotulos=livro&rotulos=curso"
# {"transacoes":[{"valor":20,"tipo":"despesa","rotulos":["livro".
..

curl "localhost:3000/transacoes?rotulos=inexistente"
# {"transacoes":[]}]

curl "localhost:3000/transacoes?rotulos="
# {"transacoes":[]}]
```

Ops! Quando não colocamos nenhum valor para o parâmetro `rotulos`, nenhuma transação é retornada, mesmo que você inclua uma transação sem rótulo ou com uma lista vazia de rótulos. Podemos fazer com que, em um cenário como este, as transações sem rótulo sejam retornadas. Que tal você implementar este cenário como um desafio? Não se esqueça de fazer os testes unitários e funcionais!

13.2 CONCLUSÃO

Neste capítulo, vimos como lidar com parâmetros de busca nas URLs, enriquecendo nossa API. Incluímos novas rotas e vimos novas funções, como `doseq` e `flatten`. Veja o código desta API em <https://gitlab.com/programacaofuncional/financeiro>

Espero que você tenha gostado de experimentar o desenvolvimento de software com Clojure! Encerramos por aqui a introdução de novos conceitos. O próximo capítulo trará algumas considerações finais e sugestões de melhorias para a API caso você queira continuar o aprendizado.

CONSIDERAÇÕES FINAIS

Ainda que tenhamos concluído diversas funcionalidades da API, existem alguns cenários que você pode pensar em implementar como forma de exercício ou buscar novos aprendizados.

14.1 FERRAMENTAS PARA O DIA A DIA DA PROGRAMAÇÃO

Clojure é uma linguagem muito poderosa. Como qualquer linguagem de programação, é importante que existam boas ferramentas para que seu uso seja significativo no cotidiano de quem a adota. Ambientes integrados de desenvolvimento (IDEs) e ferramentas de análise estática são peças fundamentais para o ecossistema de qualquer linguagem. Para Clojure temos algumas opções legais.

IDEs

Se você prefere um ambiente de desenvolvimento robusto, há um plugin muito legal para o IntelliJ, chamado Cursive (<https://cursive-ide.com>).

O Emacs é muito popular entre as pessoas que programam

Clojure, em partes por oferecer um dialeto de Lisp para ampliar as capacidades do editor.

Eu costumo utilizar o Vim, com plugins como o `paredit` e o `parinfer`.

Atom e Visual Studio Code também têm plugins para Clojure.

Análise estática

O `cljfmt` (<https://github.com/weavejester/cljfmt>) ajuda a formatar seu código para uma forma mais idiomática. Recomendo bastante para superar as limitações de caracteres por linha nos códigos deste livro.

O `Eastwood` (<https://github.com/jonase/eastwood>) é uma ferramenta que varre o seu código procurando por potenciais problemas.

14.2 OPORTUNIDADES DE REFATORAÇÃO

No código que criamos, existe uma ligação muito forte entre `handler.clj` e `db.clj`. Há uma boa oportunidade de separação de lógica de dentro de `handler.clj` para uma camada intermediária. Criamos um arquivo chamado `transacoes.clj` e este é um bom candidato!

E ainda que tenhamos criado `db.clj`, não há uma conexão com um sistema de gerenciamento de bancos de dados, como MySQL ou PostgreSQL. Clojure tem um ótimo suporte a eles graças ao uso do JDBC, do Java.

O `HoneySQL` (<https://github.com/jkk/honeysql/>) é uma boa

ferramenta para manipulação de dados com SQL, e o Migratus (<https://github.com/yogthos/migratus/>) é uma solução simples, mas boa o bastante, para migração de esquema de bancos de dados.

14.3 FUNCIONALIDADE A IMPLEMENTAR

Enquanto este livro foi escrito, várias ideias de funcionalidades para API surgiram, mas que nos fariam desviar do foco do livro se as implementássemos aqui. Mas, caso queira continuar a experiência, eis algumas sugestões:

- Remover e alterar transações.
- Paginação de resultados: quando a quantidade de transações for muito alta, é importante evitar que muitos registros trafeguem de uma vez. Com paginação, podemos, inclusive, reduzir o impacto nos servidores.
- Implementar a especificação do HAL (http://stateless.co/hal_specification.html/).
- Documentação da API com Swagger (<https://swagger.io/specification/>).
- Inclusão de logs e tratamento de exceções.

14.4 OUTRAS FORMAS DE TESTES

Vimos como criar testes unitários e de aceitação. Mas o mundo de automação de testes não se limita a estas opções.

No contexto de APIs, há uma crescente busca por acelerar a descoberta da quebra de contrato entre quem consome e quem provê uma API, e daí surgem os testes de contrato. Uma

abordagem que tem ganhado cada vez mais popularidade é a definição de contrato dirigida por quem consome (*Consumer Driven Contracts*) e eu recomendo fortemente o Pact (<https://docs.pact.io/>).

Um outro tipo de testes que tem se tornado popular por causa da popularização de linguagens de Programação Funcional são os testes baseados em propriedades. Não é um conceito novo, mas veio à tona recentemente já que ferramentas como QuickCheck, do Haskell, e ScalaCheck, para Scala, vêm crescendo em adoção. Nestes testes, você descreve uma propriedade e a ferramenta testa esta propriedade para dados diversos que ela própria vai gerar. Para Clojure temos o `test.check` (<https://github.com/clojure/test.check/>).

14.5 PORTS & ADAPTERS (HEXAGONAL) E FUNCIONAL

Em termos de arquitetura de software, a ideia de portas e adaptadores (ou arquitetura hexagonal) combina muito bem com linguagens de Programação Funcional. Sugiro uma pesquisa pelo tema. Você verá que o foco em imutabilidade nos ajuda a manter o núcleo do nosso software puro, e as bordas (como acesso a bancos de dados, leitura de arquivo, requisição HTTP) ficam muito bem segregadas deste núcleo.

14.6 DESPEDIDA

É isso! MUITÍSSIMO obrigado pela leitura. Espero que você tenha gostado do que viu por aqui e que o futuro lhe seja brilhante.

Valeu!